

A PROJECT REPORT ON SKA AIRLINES SOLUTIONS – WEB-BASED AIRLINE RESERVATION AND MANAGEMENT SYSTEM

SKA Flight Ticket Booking System Documentation

Overview

The SKA Flight Ticket Booking System is a comprehensive, web-based airline management application developed using Flask (Python). It delivers a robust platform supporting airline administration, user bookings, and guest interactions. The system is modularized into three main segments for Admin, User, and Guest roles, ensuring scalability, maintainability, and clear separation of concerns.

DECLARATION

I hereby declare that the project report titled “SKA FLIGHT TICKET BOOKING SYSTEM” is original work done by Adepu Sai Kiran during the period June 2024 to August 2025 , to the best of my knowledge and belief.

Abstract

The SKA Flight Ticket Booking System is a web-based application designed to manage flight bookings and passenger journey histories. This project demonstrates the core functionalities of an airline reservation system, aiming to simplify and streamline ticket booking processes for users. The system is developed using technologies such as Flask (Python), HTML, CSS, JavaScript, and MySQL for database management.  

It delivers efficient flight scheduling, fare management, ticketing, and user feedback functionalities. The system replaces traditional manual booking methods, saving time  and increasing operational efficiency . Proper requirements analysis and system design ensure the platform meets customer needs while maintaining compliance and scalability.  

SKA FLIGHT BOOKING SYSTEM INDEX PAGE

1. Introduction ✈️
2. AIM of The Project 🎯
3. The Application Main Purpose 🔧
4. THE PROJECT ARCHITECTURE ✈️ / PROJECT HIERACHY 📊
5. The Index Page of SKA Flight Ticket Booking System 🛫
6. Application Modules 📄
7. Flight Admin Management Module 🛫 ⚡
8. Registered User Module 🗑 ⚡
9. New User Registration Module 📄 ⚡
10. Guest User Module 👤
11. SYSTEM REQUIREMENTS ✈️ 💻
12. SYSTEM DESIGN 📊
13. SKA FLIGHT BOOKING SYSTEM ARCHITECTURE DIAGRAM ✈️
14. Flight Reservation Booking Terms ✈️ 📄
15. Flight Types / Terms ✈️ ✈️
16. Flight Seats Layout System 🖱 ✈️
17. Customer's Obligations 👤 💻 🔒
18. THE SKA FLIGHT TICKET BOOKING SYSTEM HIERARCHY / PROCESS 🛫 📁
19. PROJECT SCHEDULING ⌚ 📅
20. Benefits of Online Flight Ticket Booking ✈️ 💻
21. SKA FLIGHT TICKET BOOKING SYSTEM APPLICATION DATABASE/TABLES 🛫
22. THE SKA FLIGHT BOOKING SYSTEM APPLICATION CALLS/FLOW 🛫
23. THE SKA FLIGHT BOOKING SYSTEM ENTIRE APPLICATION ENVIRONMENT 🛫
24. THE SKA FLIGHT BOOKING SYSTEM VERSIONS 🛫
25. Issues Encountered and Resolutions ⚠️ ✅
26. Conclusion 🎯 ✈️
27. Future Look of SKA Flight Ticket Booking System 🚀 ⚡
28. Author's Note 🖋
29. Contact Information 📧
30. Acknowledgements 🙏

Introduction

The SKA Flight Ticket Booking System project is a model internet-based flight ticket reservation platform. This system enables customers/users to book flight tickets conveniently from their office or home using a PC or laptop. The system provides users the ability to create an account, search and book flights, and view reports of their entire travel history. Customers can log in to their accounts to manage bookings and access detailed flight information according to their preferences.

With internet booking, the traditional flight booking structure transforms into an online booking model, giving the concept of virtual booking a real shape. Today's booking is no longer limited to physical airline counters or offices. E-booking facilitates flight reservations by customers 24/7 globally. Anyone who registers as a user can become a member of the SKA Flight Booking System by providing personal details through a simple registration form.

To maintain customer trust and satisfaction, there is a vital need for seamless management of flight reservations that operates with ease and comfort. Efficient management positively impacts both customers and staff, empowering the management committee to make informed decisions for future enhancements. Managing flight bookings manually can be tedious and time-consuming, which makes this software essential for improving efficiency and speed in booking processes.

All transactions are securely conducted online, allowing for quick and hassle-free payment processing. This software aims to eliminate the drawbacks of manual booking systems by offering a fast, reliable, and user-friendly platform.  

Synopsis

The SKA Flight Ticket Booking System maintains a comprehensive daily record as a complete airline booking solution. It manages user account registrations, flight bookings, feedback submissions, mobile number updates, and password changes. This system is designed to streamline flight booking operations and enhance user experience with secure and efficient transaction handling.   

AIM of The Project

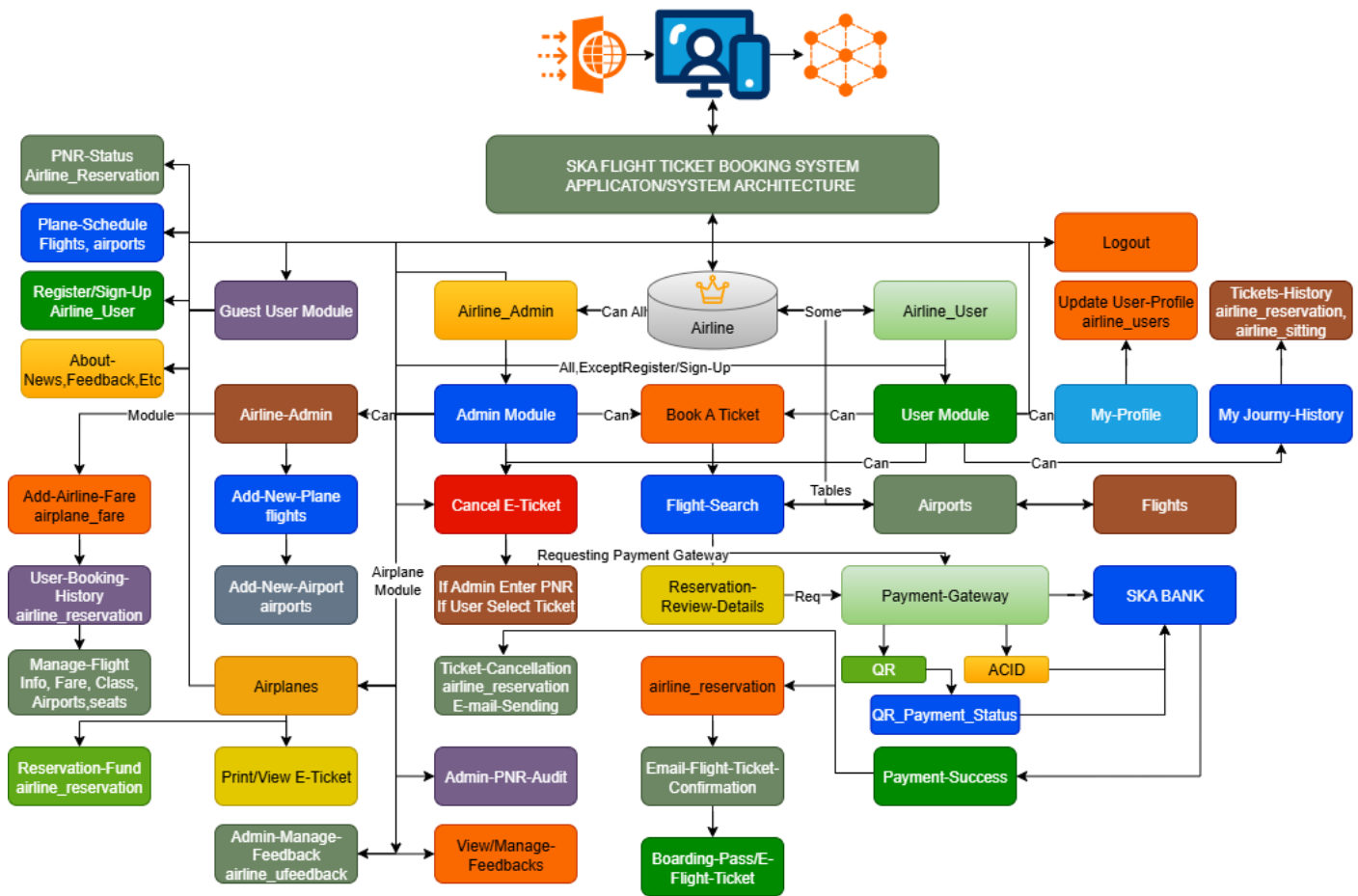
The main aim of designing and developing this Flight Ticket Booking System using Flask (Python) and MySQL is to provide secure and efficient online booking facilities to airline customers over the internet. The application runs on a web server and allows customers to log in through a secured webpage using their user ID, username, and password. Users can access all essential features such as booking flights, viewing journey history, and submitting feedback, ensuring a seamless travel booking experience.   

The Main Purpose

Traditionally, maintaining user details and booking flights required users to visit airline counters physically, which could be inconvenient and time-consuming for both passengers and airline administrators. This project aims to automate the entire flight booking process online, providing real-life simulation of an airline reservation system and the activities performed by various roles.

The SKA Flight Booking System captures and manages booking activities, passenger details, payment processing, and administrative tasks, all through an intuitive internet-based platform. This automation enhances the efficiency and accuracy of flight reservation management while reducing manual workload and improving customer satisfaction.  

THE PROJECT ARCHITECTURE ✈️ / PROJECT HIERACHY 🏗️



Main Modules:

Admin: Airlines ← Admin-Dashboard ← Admin.Routes(Python) ← Admin.Routes(Html)

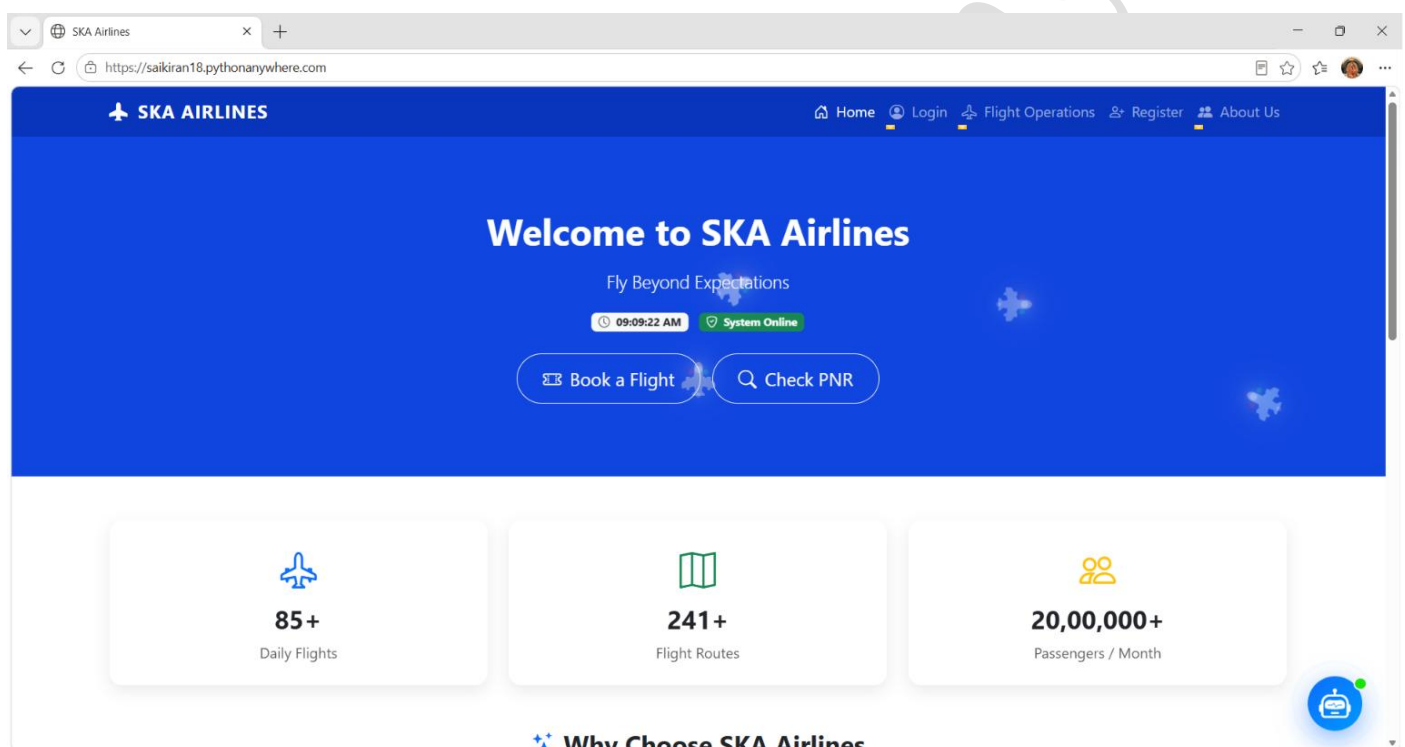
User: Airlines ← User-Dashboard ← User.Routes(Python) ← User.Routes(Html)

Guest User: Airlines ← Guest-Dashboard ← Guest.Routes(Py) ← Guest.Routes(Html)

The Index Page of SKA Flight Ticket Booking System

The Index Page of the SKA Flight Ticket Booking System serves as the main entry point for users visiting the website. It provides easy navigation and access to key features such as user login, guest access, and flight search. The page is designed to be user-friendly, allowing registered users to quickly log in and manage bookings, while guests can explore available flights, schedules, and register for new accounts.

The index page also offers essential information and links to support pages like help, feedback, and contact details, creating a smooth and intuitive experience for all users. Its clean layout and responsive design ensure accessibility across various devices.



Main Modules of SKA Flight Ticket Booking System

The system is primarily divided into four main modules, each serving different user roles and functionalities:

1. Flight Admin Module

Responsible for managing all airline operations including flight scheduling, fare management, ticket administration, and handling feedback. Admins have secure access to control and monitor the entire system.

2. Registered User Module

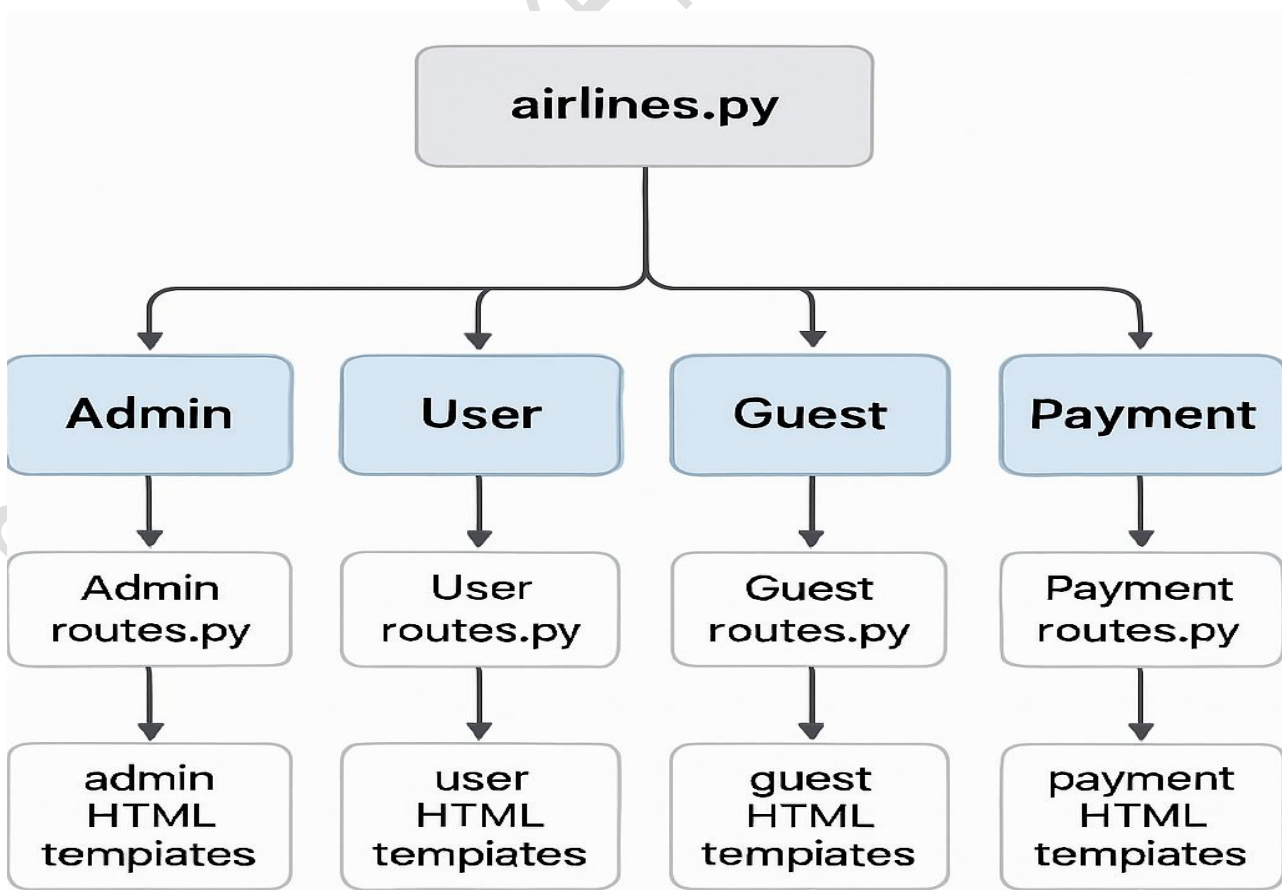
Allows registered customers to log in, book flights, view journey history, manage their profiles, make payments, and provide feedback.

3. New User Registration Module

Handles the onboarding of new users by enabling account creation with secure authentication, allowing effortless access to flight booking services.

4. Guest User Module

Caters to non-registered users by offering flight schedule browsing, PNR status checking, and access to general information without the need for logging in.



What to Expect / Modules of SKA Flight Ticket Booking System

The system is designed with multiple modules, each packed with features tailored to different user roles. Below is a detailed overview of the key modules and functionalities you can expect in this platform:

1. Flight Admin Module

The Flight Admin is the primary user responsible for managing airline operations. Admins log in securely with their username and password, which are validated against the flight booking database. After successful authentication, the admin can perform a wide range of critical tasks to ensure the smooth operation of the airline's booking system.

Main Features & Submodules for Flight Admin:

Flight Management: Add and schedule new flights, assign aircraft to routes, and update flight details.

Airport Management: Add new airports and maintain existing airport information.

Fare Management: Define, update, and modify fare structures for various flights and routes.

Ticketing: Administer ticket issuance, cancellations, and downloads.

PNR Enquiries & E-ticket Management: Lookup passenger name records and issue/manage electronic tickets.

User Booking History: Access comprehensive user booking records for tracking and support.

Feedback Management: Review and respond to customer feedback efficiently.

Seat Availability Monitoring: Monitor and manage seat inventory in real-time.

Payment Gateway Oversight: Supervise payment processing, including ticket cancellations and refunds, **Additional Controls:** Manage administrative logins, notifications, and other system settings.

These tasks are organized into submodules powered by Flask Blueprints, such as:

`new_flight_bp` for flight scheduling, `add_airport_bp` for airport data

`add_fare_bp` and `change_fare_bp` for fare control, `ticket_admin_bp` for ticket management

`user_booking_history_bp` for booking tracking ...and many more.

This modular design enhances flexibility, maintainability, and scalability of the admin functionalities.  

2. Registered User Module

Users who register on the platform can log in to:

Search and book flights

Cancel bookings when necessary

Make payments via integrated gateways including QR code options

Update personal information such as mobile numbers and passwords

Submit feedback and view journey history

3. New User Registration Module

New users can easily create accounts by filling out the registration form. Upon successful validation with the OTP sent to email, they gain access to all user functionalities, enabling smooth transitions from guests to registered customers.

4. Guest User Module

Non-registered users or guests can:

Browse flight schedules and availability

Check for the flight operational status for the flights which are ON_TIME

Check their PNR (Passenger Name Record) status

Browse flights by the Airport basis

View airline news and events

Submit anonymous feedback

This comprehensive modular architecture brings together admins, users, and guests, offering an efficient, secure, and user-friendly flight booking ecosystem.  

1. Flight Admin Management Module ✈️ ✨

The Flight Admin Management module is the core of the SKA Flight Ticket Booking System's administration. It empowers administrators with full control over airline operations, ensuring smooth scheduling, fare management, and ticketing. Admins securely log in with their credentials maintained in the airline database. 🔒 👤 ✈️

Key Responsibilities and Functionalities 🛠️

After successful login, the Flight Admin can perform the following crucial tasks, organized into several submodules:

1. Flight Operations Management ✈️

Add New Flights: Create and schedule flights, assigning airplanes to routes (new_flight_bp)

Manage Flight Details: Update, modify, or cancel flights as needed ↺

View Plane-Airport Mapping: Manage relationships between airplanes and airports (airplane_airports_bp) ✂️ 📌

2. Airport Management 🏢

Add Airports: Register new airports into the system (add_airport_bp) +

Maintain Airport Details: Update existing airport information 🛠️

3. Fare Management 💰

Set New Fares: Define fares for flights and routes (add_fare_bp) 💵

Modify Existing Fares: Update or change fares based on requirements (change_fare_bp) ✎

4. Ticket Administration 🎫

Ticket Issuance & Management: Handle booking confirmations, issuance, and ticket views (ticket_admin_bp) ✅

Ticket Downloads: Enable downloading of tickets (ticket_download_admin_bp) ⬇

Boarding Pass: The Boarding pass issues on the time of the departure before 24 hrs to 20 min departure time

Ticket Cancellation: Manage cancellation requests and refunds (admin_cancel_ticket_bp, payment_gateway_ticket_cancel_admin_bp) ❌ ↺

5. PNR Enquiries 🔍

Lookup and manage Passenger Name Records to assist passengers 🖨️

6. User Booking History 📅

Access detailed user booking records (user_booking_history_bp) 📁

7. User AC Control: Admin can control the users accounts and maintained in the system

8. Feedback Management 📧

Review and respond to customer feedback for service improvement

(admin_view_feedback.py, admin_delete_feedback.py) 💬 👍

9. Seat Availability Monitoring 📊

Monitor and manage seat inventory in real time (seat_avl_bp) 📈

10. Payment Gateway Oversight 💳

Oversee payment transactions including ticket purchases and cancellations

(payment_gateway_tkt_can_admin_bp) 🔒 💰

11. Wallet Operations 💰

Admin manages the wallet bookings

12. Additional Admin Controls ⚙️

Manage admin login/logout (admin_login_bp, admin_logout_bp) 🔑

Oversee airline funds and finances (airline_fund_bp) 💼

Configure system notifications and alerts 🔔

Route connection management (admin_routes.py, admin_bp) 🔄

Architecture & Implementation 🧩

These functionalities are implemented in a modular fashion using Flask Blueprints, encouraging maintainability and scalability. Some important files include:

new_flight.py, add_airport.py, add_fare.py, change_fare.py, ticket_admin.py, ticket_download_admin.py, user_booking_history.py, admin_can_ticket.py

payment_gateway_tkt_can_admin.py, airline_fund.py

...and more

Summary 🎲

The Flight Admin Management module offers a centralized, secure, and efficient system for airline administrators to handle backend operations, supporting seamless flight management and customer service excellence. ✈️🔧🛡️

Flight Admin Module 🛫

└─ Admin Section

| └─ admin_login_bp <-- Login functions

| └─ admin_logout_bp <-- Logout logic

| └─ admin_dashboard_bp <-- HTML renderings (see below)

| └─ admin_bp <-- Route connectors

|

| admin_routes.py -----> Imports blueprints and submodules:

| |

| └─ new_flight_bp --> new_flight.html

| └─ add_airport_bp --> add_airport.html

| └─ add_fare_bp --> add_fare.html

| └─ change_fare_bp --> change_fare.html

| └─ ticket_admin_bp --> ticket_admin.html, ticket_admin1.html, ...

| └─ ticket_download_admin_bp --> ticket_download_admin.html

| └─ user_booking_history_bp --> user_booking_history.html

| └─ airplane_airports_bp --> airline_airports_su.html

| └─ admin_can_ticket_bp --> admin_can_ticket.html, admin_can_ticket1.html, ...

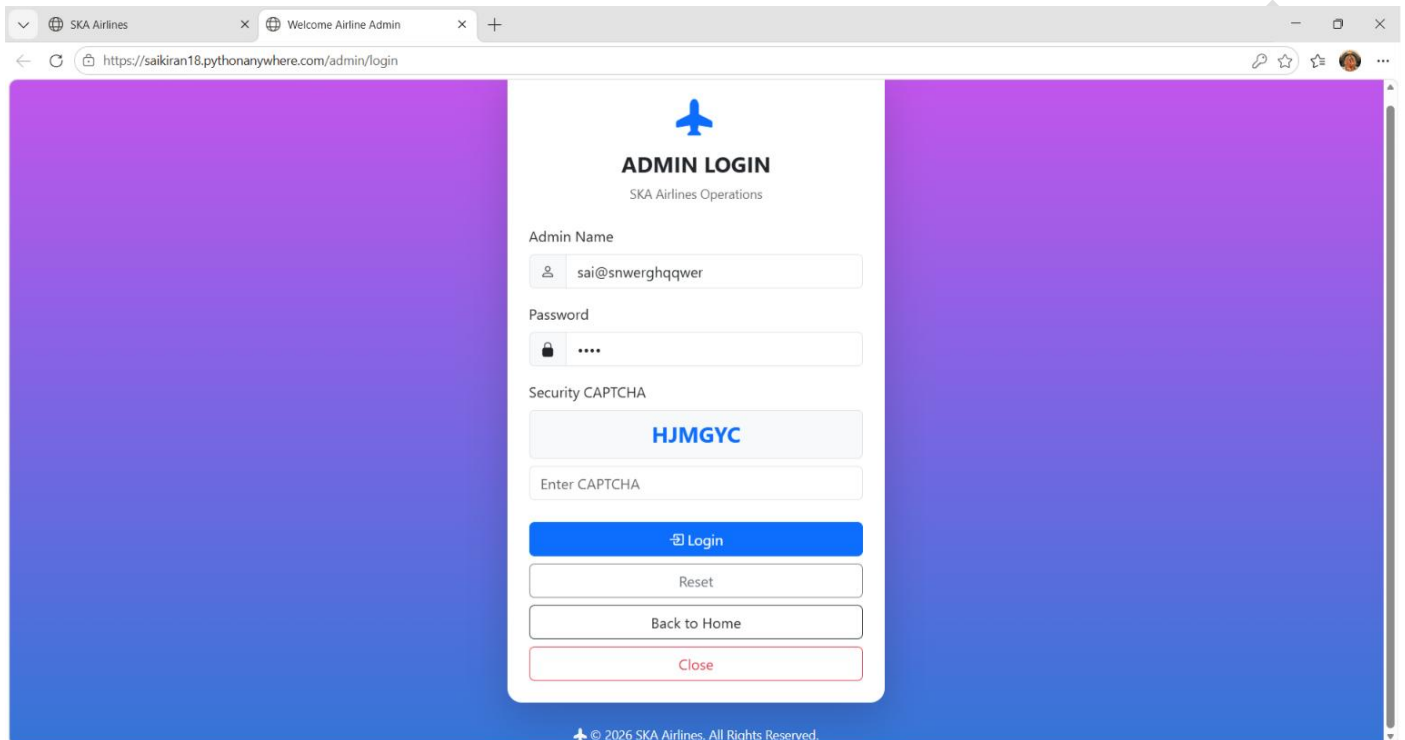
| └─ payment_gateway_tkt_can_admin_bp

| └─ airline_fund_bp --> airline_fund_su.html

| (other submodules...)

Flight Admin Screenshots

ADMIN LOGIN PAGE Protected by Password hashing and security captcha



SKA Airlines

Welcome Airline Admin

https://saikiran18.pythonanywhere.com/admin/login

ADMIN LOGIN
SKA Airlines Operations

Admin Name
sai@snwerghqwer

Password
.....

Security CAPTCHA
HJMGYC

Enter CAPTCHA

Login

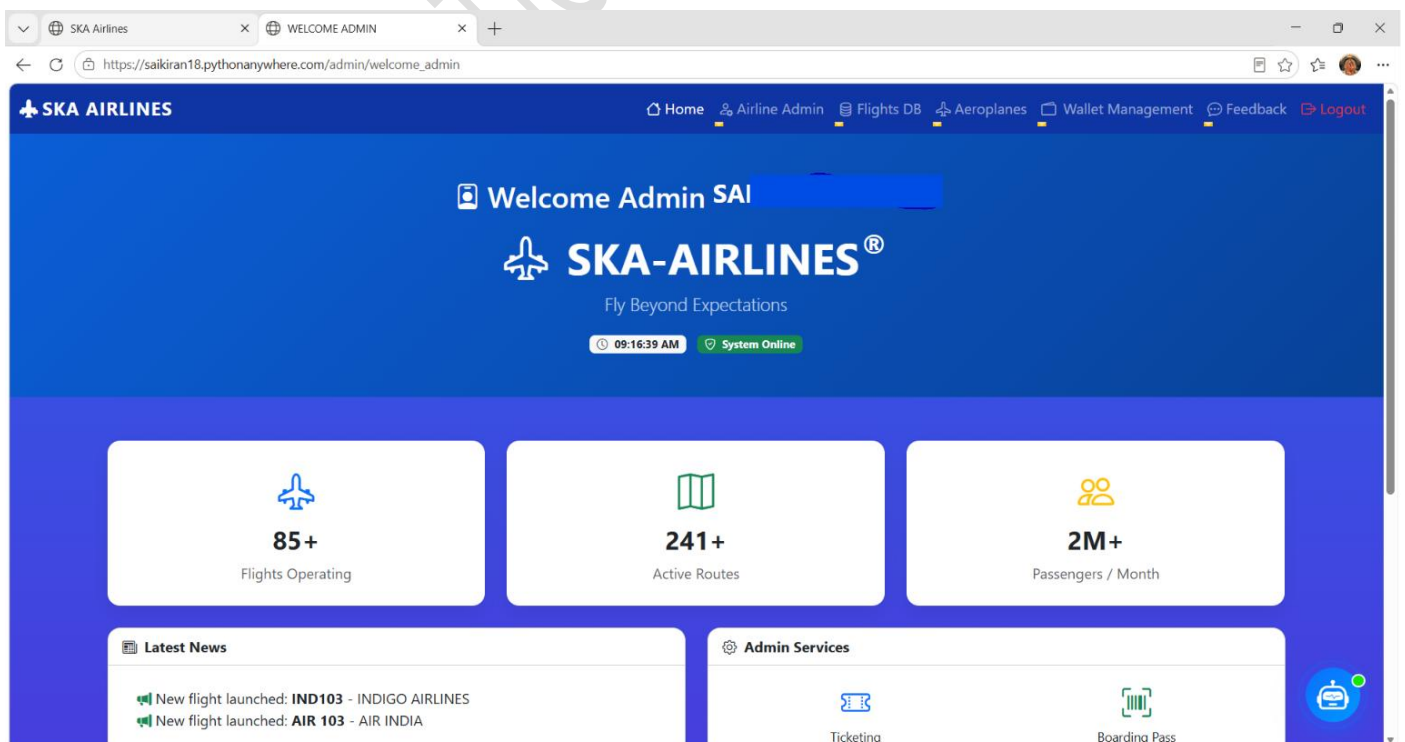
Reset

Back to Home

Close

© 2026 SKA Airlines. All Rights Reserved.

INDEX PAGE OF FLIGHT ADMIN



SKA AIRLINES

Home Airline Admin Flights DB Aeroplanes Wallet Management Feedback Logout

Welcome Admin SAI

SKA-AIRLINES®
Fly Beyond Expectations

09:16:39 AM System Online

85+
Flights Operating

241+
Active Routes

2M+
Passengers / Month

Latest News

New flight launched: **IND103** - INDIGO AIRLINES
New flight launched: **AIR 103** - AIR INDIA

Admin Services

Ticketing Boarding Pass

SKA Airlines | WELCOME ADMIN | Admin User Account Control | SKA Airlines

https://saikiran18.pythonanywhere.com/admin/admin_user_account_control

Admin User Account Control

Secure Passenger Status Monitoring & Restriction Management System

Airline User Number

Passenger Username

Current User Status

Change Account Status

Select Account Status

Passenger Status Message

Select account status to view the passenger notification message.

SKA Airlines | WELCOME ADMIN | New Flight Register | SKA Airlines

https://saikiran18.pythonanywhere.com/admin/add_new_plane

Admin Instructions

- Airports & fares auto-generated (₹12/KM).
- Flight number is automatically generated at the backend using the flight name, ensuring a unique alphanumeric format.
- Update arrival & departure times after adding flight.
- If the airport dropdown does not work, please refresh the page.
- After adding a flight, please refresh this page to enable the airport dropdown.

[Flight Timings](#) [Flight Fares](#)

NEW FLIGHT REGISTER

Flight Name *

Flight Type *

Select Airport

[Add Airport](#)

Selected Airports *

Base Fare *

Aircraft Seat Layout Information

- Controls aircraft seat capacity.
- Used during ticket booking.
- Used for seat allocation.
- Used during boarding pass generation.
- Used for seat availability calculations.
- Only Seat Count is editable.

 Search Aircraft Type, Class Type or Remarks

ID	Aircraft Type	Class Type	Seat Count	Remarks	Action
10	AIRBUS A320	BUSINESS CLASS	12	PREMIUM ECONOMY = 150	 Update
12	AIRBUS A320	ECONOMY	138	0	 Update
9	AIRBUS A320	FIRST CLASS	12	BUSINESS CLASS = 100	 Update
11	AIRBUS A320	PREMIUM ECONOMY	24	ECONOMY = 550	 Update
14	AIRBUS A321	BUSINESS CLASS	25	0	 Update
16	AIRBUS A321	ECONOMY	156	0	 Update

Book A Flight

 Origin	 Destination
NEW DELHI-NDL	NEW YORK-NY
 Journey Date	 Return Date
26-06-2026	dd-mm-yyyy
 Plane Class	 Ticket Type
FIRST CLASS	GENERAL
 Trip Mode	
ONE WAY	
SEARCH	RESET
	CLOSE

 © 2026 SKA Airlines. All Rights Reserved.

✈️ Modify Search Results

📍 Origin

NEW DELHI-NDL

📍 Destination

NEW YORK-NY

📄 Class

First Class

📅 Journey Date

dd-mm-yyyy

📄 Ticket Type

General

🔄 Trip Mode

One Way

📅 Return Date

dd-mm-yyyy

🔍 Modify Search

🔄 Reset

🚫 Close

Origin ✈️

Indira Gandhi Int Airport
NEW DELHI-NDL, INDIA-IND



Destination ✈️

John F. Kennedy Int Airport
NEW YORK-NY, UNITED STATES-USA

✈️ Available Flights

✈️ Flight No	✈️ Flight Name	✈️ Aircraft	🕒 Departure	🕒 Arrival	🕒 Duration	₹ Fare	🕒 Flight Schedule
SKA100	SKA AIRLINES	BOEING 777	20:40 26/Jun/2026	17:35 27/Jun/2026	20H 55M	950	✈️ Flight Schedule
SOW100	SOWMYA AIRLINES	AIRBUS A320	01:04 26/Jun/2026	21:04 26/Jun/2026	20H 0M	900	✈️ Flight Schedule



AIRLINE RESERVATION NEXT STEPS

📄 Passenger details must match your Government issued ID

📍 Origin:

Indira Gandhi Int Airport
NEW DELHI-NDL, INDIA-IND

🕒 Departure: 11:00

26/Jun/2026

📄 Return: None

📍 Destination:

John F. Kennedy Int
Airport
NEW YORK-NY, UNITED STATES-USA

🕒 Arrival: 06:20

27/Jun/2026

✈️ Flight No: UNI100

✈️ Flight Name: UNITED
AIRLINES

✈️ Type: BOEING 777

📄 Class: FIRST CLASS

📄 Quota: GENERAL

🕒 Fly Time: 19H 20M

✈️ Flight Charges: ₹4100.00

✈️ Class Charges: ₹6500.00

✈️ Quota Discount: -
₹0.00

✈️ Trip Discount: -
₹0.00

💰 Total Fare:
₹151660.00

👤 Passenger Details

👤 Name

Sai Kiran

👤 Age

26

👤 Gender

Male

👤 Passengers

1

0 Kids

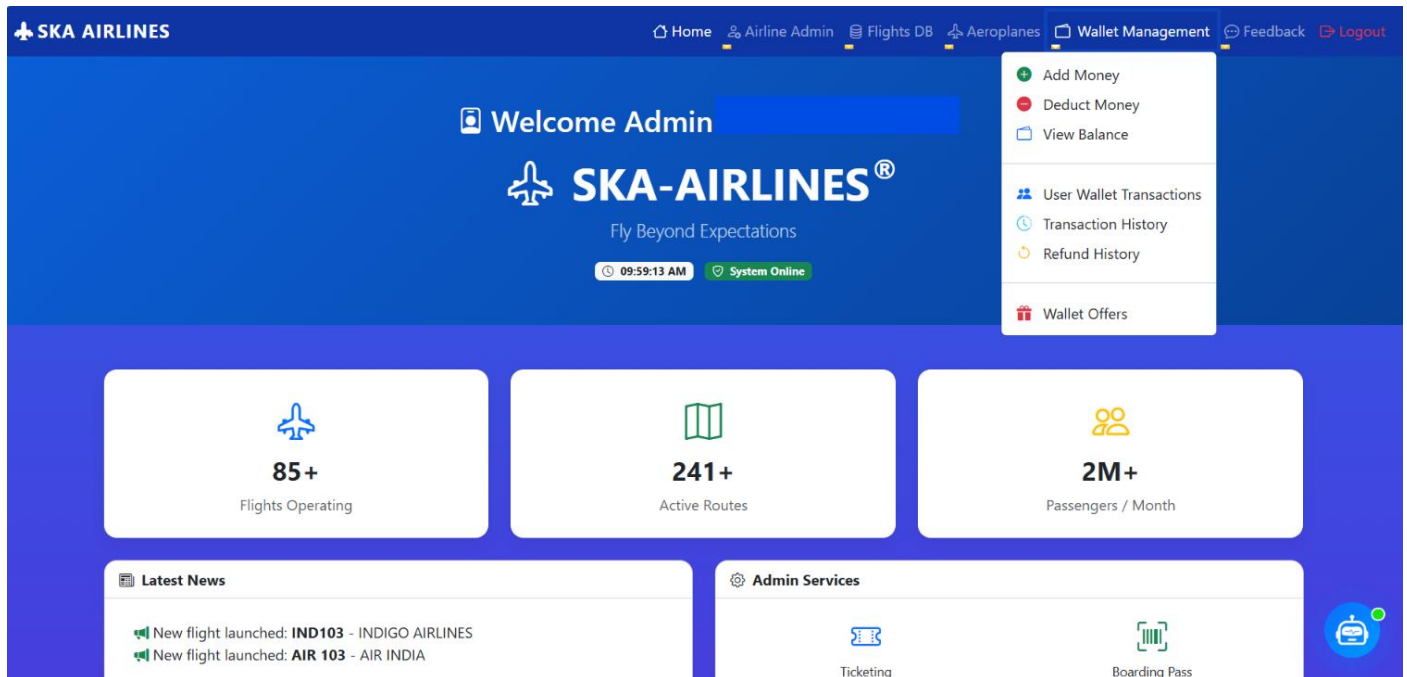
📄 Seats: 12

+ Add Person

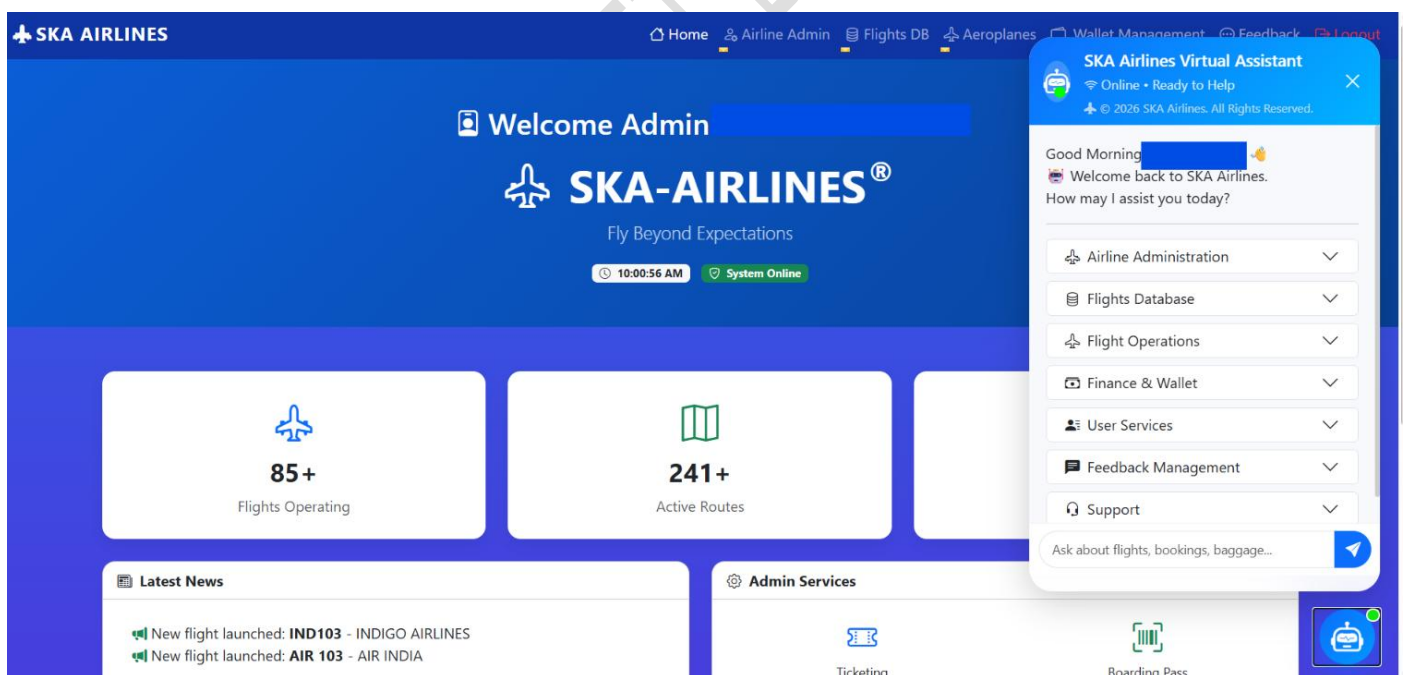
BOARDING PASS

17

Admin Wallet Operations & Chatbot



Admin Chatbot: The chatbot can able to answer to the all questions that related to the project which is easy to maintain the system smooth



SKA AIRLINES

HomeAirline AdminFlights DBAeroplanesWallet ManagementFeedbackLogout

Welcome Admin

SKA-AIRLINES®

Fly Beyond Expectations

10:01:25 AMSystem Online

85+

Flights Operating

241+

Active Routes

Latest News

New flight launched: **IND103** - INDIGO AIRLINES

New flight launched: **AIR 103** - AIR INDIA

Admin Services

Ticketing

Boarding Pass

SKA Airlines Virtual Assistant

Online • Ready to Help

© 2026 SKA Airlines. All Rights Reserved.

Hi

Good Morning

Welcome to SKA Airlines.

I'm the SKA Airlines Virtual Assistant

I can assist you with:

Flight Status

Search FlightsFlight Schedule

Airport ScheduleMy Booking

WalletContact Support

Please select a suggestion button or type your request.

Ask about flights, bookings, baggage...

SKA AIRLINES

HomeAirline AdminFlights DBAeroplanesWallet ManagementFeedbackLogout

Welcome Admin

SKA-AIRLINES®

Fly Beyond Expectations

10:01:54 AMSystem Online

85+

Flights Operating

241+

Active Routes

Latest News

New flight launched: **IND103** - INDIGO AIRLINES

New flight launched: **AIR 103** - AIR INDIA

Admin Services

Ticketing

Boarding Pass

SKA Airlines Virtual Assistant

Online • Ready to Help

© 2026 SKA Airlines. All Rights Reserved.

Hi

I can assist you with:

Flight Status

Search FlightsFlight Schedule

Airport ScheduleMy Booking

WalletContact Support

Please select a suggestion button or type your request.

Priv status

Hello

Please enter your PNR number
Example: 279

Ask about flights, bookings, baggage...

2. Registered User Module

The Registered User Module provides customers with a secure and interactive platform to manage their flight bookings, personal profiles, payments, and feedback efficiently. It ensures a seamless and personalized experience tailored to each user's needs.  

Key Features & Functionalities

Secure Authentication 

User Registration & Login: Secure sign-up and login with unique credentials (username, password).

Logout: Ensures safe termination of sessions for privacy.

Flight Booking & Management

Flight Search: Quick searches based on destination, dates, and preferences.

Ticket Booking: Easy booking with real-time seat availability and fare display.

Multiple Payment Modes:

Bank Account Payment: Users can pay through their bank account IDs (ACID).

QR Code Payment: Convenient QR-based payment integration for fast processing.

Ticket Cancellation: Ability to cancel bookings online subject to airline policies.

Ticket Status & Notifications

After booking, ticket status and e-ticket details are sent to users via email, keeping them informed throughout the journey.  

Payment Processing

Integrated Payment Gateway: Supports secure transactions via bank accounts and QR code payments (payment_gateway_bp, qr_payment_bp).

Transaction Security: Ensures encrypted and validated payments protecting financial data.

Profile & Account Management

Update Personal Info: Users can update mobile numbers (user_update_mbl_num_bp) and passwords (user_update_password_bp).

Journey History: Detailed access to past and upcoming bookings (user_journey_history_bp).

User Feedback & Support

Users can provide valuable feedback to improve service quality (user_feedback_bp).

Ticket Handling

E-ticket Access: Download, print, or view e-tickets at any time (ticket_user_bp).

Cancellation Requests: Manage cancellations and track status (user_can_ticket_bp).

and e-ticket details are sent to users via email, keeping them informed throughout the journey.  

User Experience & Interface

The module features intuitive HTML templates for login, dashboards, booking, payment, feedback, and profile management pages. The design is fully responsive for desktop and mobile devices, ensuring smooth user experience everywhere.  

Security & Data Integrity

Strong user authentication, encrypted communication, and session management protect privacy and ensure integrity of user data.

Summary

The Registered User Module offers a secure, flexible, and user-friendly environment with multiple payment options, ticket status email notifications, and comprehensive booking controls. This fosters confidence and satisfaction among air travellers.   

Flight Registered User Module

└─ User Section

| └─ user_login_bp

| └─ user_logout_bp

| └─ user_dashboard_bp

| └─ user_bp

| user_routes.py -----> Handles: |

| └─ ticket_user_bp --> ticket_user.html, ticket_user1.html, ...

| └─ payment_gateway_bp --> payment_gateway_su.html

| └─ ticket_userdb_bp

| └─ user_can_ticket_bp --> user_can_ticket.html, user_can_ticket1.html, ...

| └─ user_journey_history_bp --> user_journey_history_su.html, ...

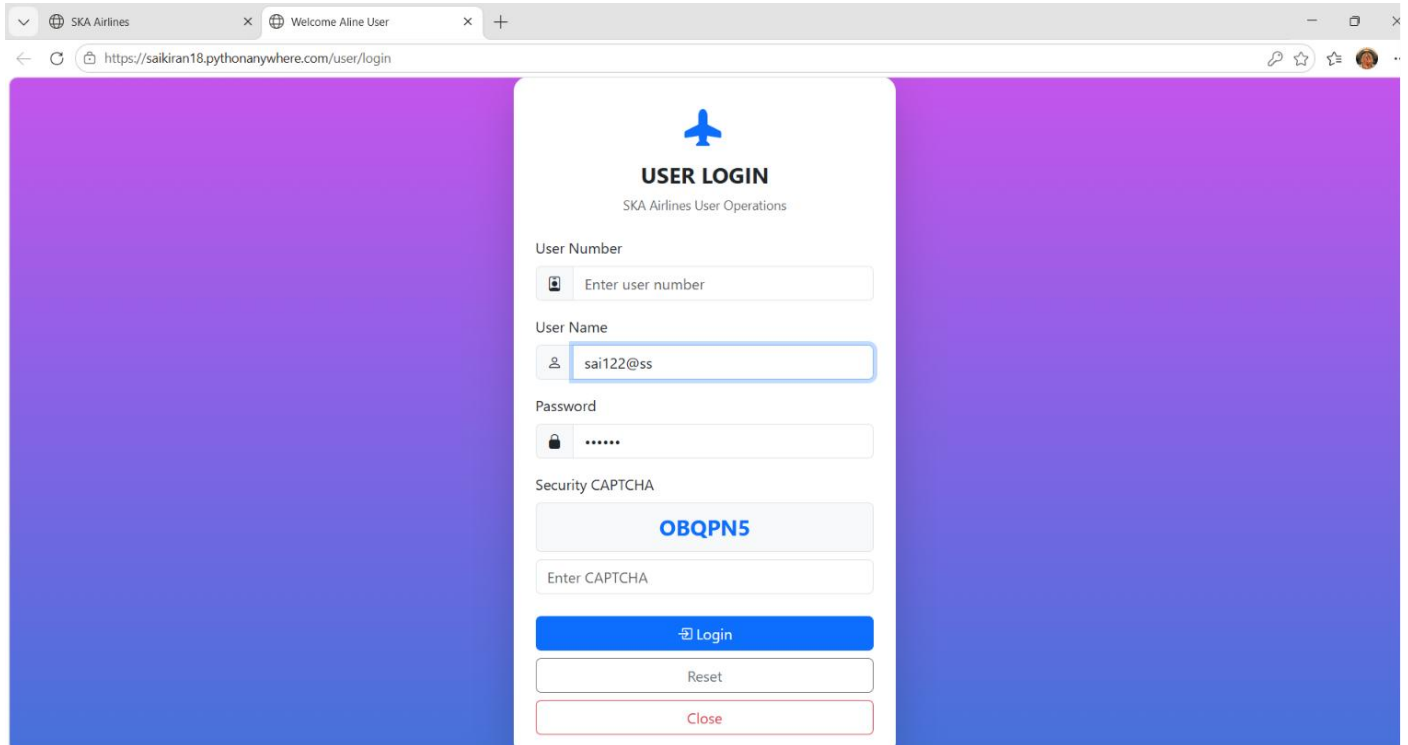
| └─ user_feedback_bp --> user_feedback.html, user_feedback_su.html

| └─ user_update_mbl_num_bp --> user_update_mbl_num.html, ...

| └─ user_update_password_bp --> user_update_password.html, ...

| └─ qr_payment_bp --> qr_payment_su.html

Flight Registered User Module Screenshots



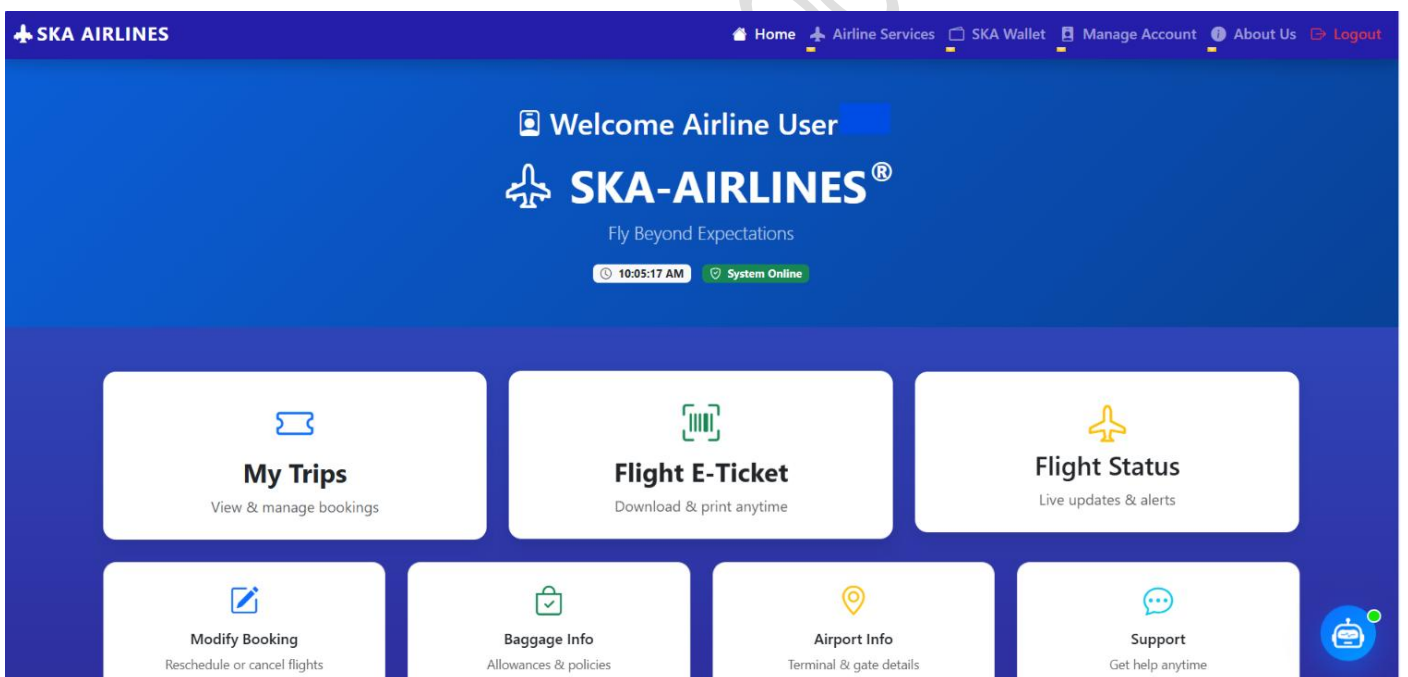
USER LOGIN
SKA Airlines User Operations

User Number

User Name

Password

Security CAPTCHA



SKA AIRLINES Home Airline Services SKA Wallet Manage Account About Us Logout

Welcome Airline User

SKA-AIRLINES®
Fly Beyond Expectations

10:05:17 AM System Online

My Trips
View & manage bookings

Flight E-Ticket
Download & print anytime

Flight Status
Live updates & alerts

Modify Booking
Reschedule or cancel flights

Baggage Info
Allowances & policies

Airport Info
Terminal & gate details

Support
Get help anytime

✈️ Modify Search Results

📍 Origin

NEW DELHI-NDL

↔

📍 Destination

TOKYO-TKO

✈️ Class

First Class

📅 Journey Date

dd-mm-yyyy

📄 Ticket Type

General

🔄 Trip Mode

One Way

📅 Return Date

dd-mm-yyyy

🔍 Modify Search

🔄 Reset

✖ Close

Origin ✈️

Indira Gandhi Int Airport
NEW DELHI-NDL, INDIA-IND



Destination ✈️

Tokyo Int Airport
TOKYO-TKO, JAPAN-JPN

✈️ Available Flights							
✈️ Flight No	✈️ Flight Name	✈️ Aircraft	🕒 Departure	🕒 Arrival	🕒 Duration	₹ Fare	📅 Flight Schedule
JAP100	JAPAN AIRLINES	AIRBUS A320	05:20 25/Jun/2026	13:30 25/Jun/2026	8H 10M	2600	✈️ Flight Schedule
ASI101	ASIANA AIRLINES	AIRBUS A380	19:30	05:30	10H 0M	2500	✈️ Flight Schedule

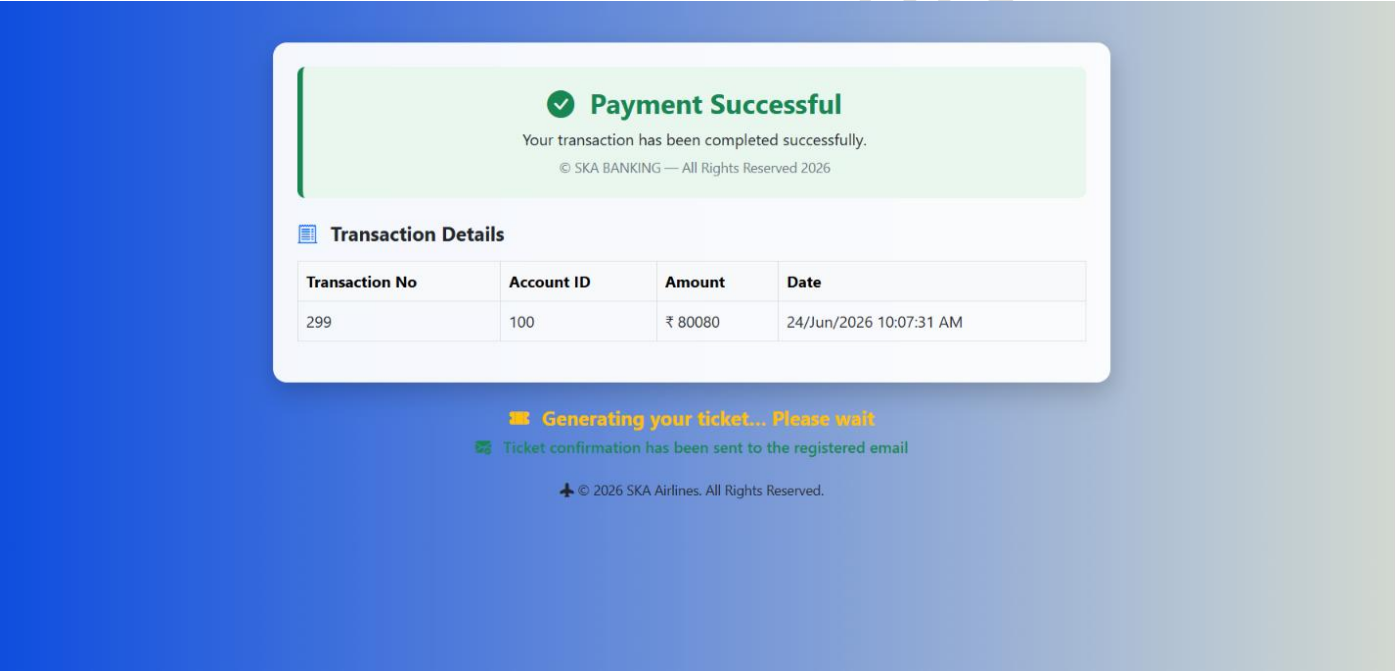
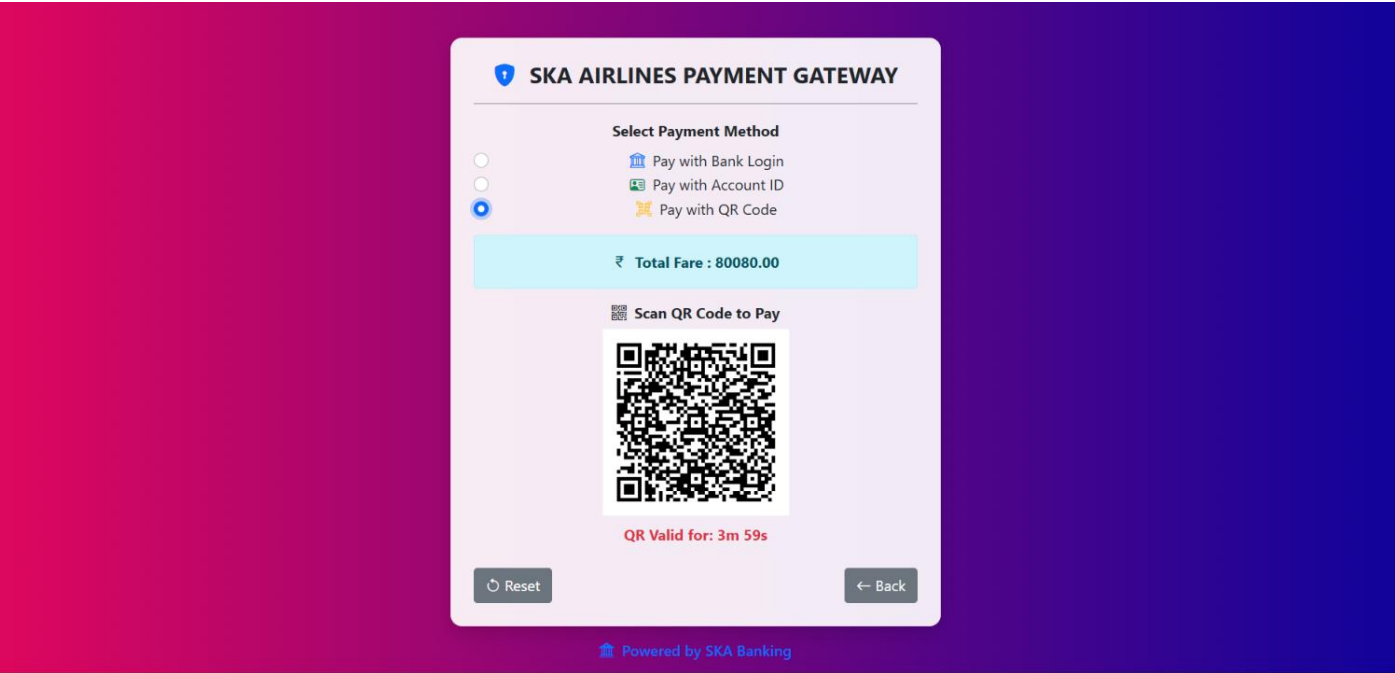
✈️ Flight No	✈️ Flight Name	✈️ Aircraft	🕒 Departure	🕒 Arrival	🕒 Duration	₹ Fare	📅 Flight Schedule
JAP100	JAPAN AIRLINES	AIRBUS A320	05:20 25/Jun/2026	13:30 25/Jun/2026	8H 10M	2600	✈️ Flight Schedule

Seat Availability: JAP100 - JAPAN AIRLINES

Class: FIRST CLASS

25/Jun/2026	26/Jun/2026	27/Jun/2026	28/Jun/2026	29/Jun/2026	30/Jun/2026		
25/Jun/2026 12 Seats	26/Jun/2026 12 Seats	27/Jun/2026 12 Seats	28/Jun/2026 12 Seats	29/Jun/2026 12 Seats	30/Jun/2026 12 Seats		
ASI101	ASIANA AIRLINES	AIRBUS A380	19:30 25/Jun/2026	05:30 26/Jun/2026	10H 0M	2500	✈️ Flight Schedule
AIR101	AIR FRANCE	AIRBUS A380	09:00 25/Jun/2026	23:10 25/Jun/2026	14H 10M	2600	✈️ Flight Schedule
TRA100	TRANSAVIA FRANCE	BOEING 737	07:00 25/Jun/2026	21:20 25/Jun/2026	14H 20M	2000	✈️ Flight Schedule
VUE100	VUELING AIRLINES	AIRBUS A320	05:18 25/Jun/2026	17:50 25/Jun/2026	12H 32M	2150	✈️ Flight Schedule
ITA100	ITA AIRWAYS	AIRBUS A320	06:00 25/Jun/2026	17:35 25/Jun/2026	11H 35M	2200	✈️ Flight Schedule
ITA101	ITA AIRWAYS	AIRBUS A380	05:15 25/Jun/2026	16:21 25/Jun/2026	11H 6M	2300	✈️ Flight Schedule
NEO100	NEOS AIRLINES	AIRBUS A380	05:00 25/Jun/2026	13:32 25/Jun/2026	8H 32M	2600	✈️ Flight Schedule

Payment Gateway:



ORIGIN

Indira Gandhi Int Airport
NEW DELHI-NDL, INDIA-IND
Departure: 05:20 25/Jun/2026



Fly Time: 8H 10M

DESTINATION

Tokyo Int Airport
TOKYO-TKO, JAPAN-JPN
Arrival: 13:30 25/Jun/2026

Journey Details

JAPAN AIRLINES # JAP100 GENERAL FIRST CLASS

PNR Number: 284	Airplane Number: JAP100	Airplane Name: JAPAN AIRLINES	Booking Date: 24/Jun/2026 10:07:38 AM
Return Date: None	Class: FIRST CLASS	Trip Type: GENERAL	Fly Time: 8H 10M

Passenger Details

Passenger Name	Age	Gender	Gate	Current Status	Booking Status
SAI KIRAN	26	Male	Check Monitors	CNF / FIRST CLASS / 1A	CNF / FIRST CLASS / 1A

Fare Summary

Boarding QR Code



10:07:57 AM

- Book A Ticket
- Search A Flight
- PNR Status
- Plane Schedule
- Flight Operational Status
- Airport Schedule
- Web Check-in / Boarding Pass
- Cancel Flight E-Ticket



My Trips

View & manage bookings



Flight E-Ticket

Download & print anytime



Flight Status

Live updates & alerts



Modify Booking

[Reschedule or cancel flights](https://saikiran18.pythonanywhere.com/guest/plane_schedule)



Baggage Info

Allowances & policies



Airport Info

Terminal & gate details



Support

Get help anytime



Your Flight E-Tickets Ready to Generate Boarding Pass/Web-Check-in

Select a Flight E-Ticket row to Generate Boarding Pass/Web-Check-in.

① Row-click enabled

Confirmed E-Ticket List Ready for Boarding Pass/Web-Check-in

 Click any row to Generate Boarding Pass/Web-Check-in

PNR NUMBER	TICKET STATUS	ORIGIN	DESTINATION	FLIGHT NUMBER	FLIGHT NAME	FLIGHT CLASS	DEPARTURE	JOURNEY STATUS	FLIGHT TYPE
284	CNF	Indira Gandhi Int Airport NEW DELHI-NDL, INDIA-IND	Tokyo Int Airport TOKYO-TKO, JAPAN-JPN	JAP100	JAPAN AIRLINES	FIRST CLASS	05:20 25/Jun/2026	IP	AIRBUS A320

Confirmed Flight E-Ticket Ready for Boarding Pass/Web-Check-in Action

ⓘ **Note:** You must select a flight row first.

 Generate Boarding Pass/Web-Check-in

 Reset

 Close

Guides and Observations for Web-Check-in / Boarding Pass Generation

- ⓘ **Note:** Web Check-in and Boarding Pass issuance are available only for **confirmed** e-tickets.
-  **Note:** Web Check-in is available only for flights departing **today** or **tomorrow** (based on the PNR).
- ⌚ **Note:** Web Check-in opens **48 hours** prior to scheduled departure and **20 minutes** before departure.

User Web-Check-in

Your Flight E-Tickets Ready

Select a Flight E-Ticket row to Generate Boarding Pass/Web-Check-in.

saikiran18.pythonanywhere.com says

Are you sure you want to generate the Boarding Pass?

SECURITY NOTICE:

Passengers are strictly prohibited from carrying weapons, firearms, explosives, sharp objects, flammable or hazardous materials.

All baggage is subject to airport security screening.
Violations may lead to denied boarding or legal consequences.

OK

Cancel

Web-Check-in

① Row-click enabled

Confirmed E-Ticket List Ready for Boarding Pass/Web-Check-in

 Click any row to Generate Boarding Pass/Web-Check-in

PNR NUMBER	TICKET STATUS	ORIGIN	DESTINATION	FLIGHT NUMBER	FLIGHT NAME	FLIGHT CLASS	DEPARTURE	JOURNEY STATUS	FLIGHT TYPE
284	CNF	Indira Gandhi Int Airport NEW DELHI-NDL, INDIA-IND	Tokyo Int Airport TOKYO-TKO, JAPAN-JPN	JAP100	JAPAN AIRLINES	FIRST CLASS	05:20 25/Jun/2026	IP	AIRBUS A320

Confirmed Flight E-Ticket Ready for Boarding Pass/Web-Check-in Action

ⓘ **Note:** You must select a flight row first.

 Generate Boarding Pass/Web-Check-in

 Reset

 Close

Guides and Observations for Web-Check-in / Boarding Pass Generation

- ⓘ **Note:** Web Check-in and Boarding Pass issuance are available only for **confirmed** e-tickets.
-  **Note:** Web Check-in is available only for flights departing **today** or **tomorrow** (based on the PNR).
- ⌚ **Note:** Web Check-in opens **48 hours** prior to scheduled departure and **20 minutes** before departure.

SKA AIRLINES®

Secure Boarding Pass

PNR: 284

Indira Gandhi Int Airport

NEW DELHI-NDL, INDIA-IND

Origin

Tokyo Int Airport

TOKYO-TKO, JAPAN-JPN

Destination

FLIGHT NO

JAP100

FLIGHT NAME

JAPAN AIRLINES

DEPARTURE

05:20 25/Jun/2026

ARRIVAL

13:30 25/Jun/2026

Passenger Details

CNF - FIRST CLASS

NAME	AGE	GENDER	STATUS / CLASS / SEAT
SAI KIRAN	26	Male	CNF/FIRST CLASS/1A

Boarding begins 2 hours prior to departure. Gate closes 20 minutes before departure.

© 2026 SKA Airlines. All Rights Reserved.

BOARDING PASS

JAP100

Scan at Gate

Gate

Check Monitors

Aircraft

AIRBUS A320

Trip Type

GENERAL

Distance

5915 KM

Fly Time

8H 10M

© SKA AIRLINES® 2026

Download Boarding Pass

Close

SKA AIRLINES

Home

Airline Services

SKA Wallet

Manage Account

About Us

Logout

Welcome Airline User S

Fly Beyond Expectations

10:08:37 AM

System Online

Add Money

View Balance

Transaction History

Refund History

Wallet Offers

My Trips

View & manage bookings

Flight E-Ticket

Download & print anytime

Flight Status

Live updates & alerts

Modify Booking

Baggage Info

Allowances & policies

Airport Info

Terminal & gate details

Support

Get help anytime

https://saikiran18.pythonanywhere.com/user/welcome_user#

SKA FLIGHT

User Wallet & Chatbot:

SKA AIRLINES

Home Airline Services **SKA Wallet** Manage Account About Us Logout

Welcome Airline User

SKA-AIRLINES

Fly Beyond Expectations

10:08:37 AM System Online

- Add Money
- View Balance
- Transaction History
- Refund History
- Wallet Offers

My Trips
View & manage bookings

Flight E-Ticket
Download & print anytime

Flight Status
Live updates & alerts

Modify Booking
Edit your flight details

Baggage Info
Allowances & policies

Airport Info
Terminal & gate details

Support
Get help anytime

https://saikiran18.pythonanywhere.com/user/welcome_user#

Wallet Transaction History
SKA Airlines Secure Financial Ledger

Wallet Dashboard Add Money Close Window

WALLET ACCOUNT USERNAME AVAILABLE BALANCE ₹ 80,080

Transaction Ledger
Complete Airline Wallet Financial Activity 46 Transactions

Search transaction, amount, account, remarks...

Cr User E-Flight Booking
Transaction No : 299 ₹ 80,080
24/Jun/2026 10:07:31 AM

SKA AIRLINES

Home

Airline Services

SKA Wallet

Manage Account

About Us

Logout

Welcome Airline User

 SKA-AIRLINES®

Fly Beyond Expectations

10:17:49 AM

System Online



My Trips

View & manage bookings



Flight E-Ticket

Download & print anytime



Modify Booking

Reschedule or cancel flights



Baggage Info

Allowances & policies



Airport Info

Terminal & gate details



Support

Get help anytime

SKA Airlines Virtual Assistant

Online • Ready to Help

© 2026 SKA Airlines. All Rights Reserved.

Good Morning 🌞

Welcome back to SKA Airlines.
How may I assist you today?

Flight Information

Booking Services

Account & Wallet

Support

Ask about flights, bookings, baggage...

SKA AIRLINES

Home

Airline Services

SKA Wallet

Manage Account

About Us

Logout

Welcome Airline User

 SKA-AIRLINES®

Fly Beyond Expectations

10:18:13 AM

System Online



My Trips

View & manage bookings



Flight E-Ticket

Download & print anytime



Modify Booking

Reschedule or cancel flights



Baggage Info

Allowances & policies



Airport Info

Terminal & gate details



Support

Get help anytime

SKA Airlines Virtual Assistant

Online • Ready to Help

© 2026 SKA Airlines. All Rights Reserved.

Of course, I remember you! 😊

You're 🇧🇪 🌞

Welcome aboard SKA Airlines 🛫❤️
I'm SKA Airlines Assistant 🛩️, your virtual travel companion designed to make your journey easier and more enjoyable. ❤️

Flight Bookings

Status, Schedules

Wallet Services

Web Check-in

Booking History

Support & Assistance

Where would you like to fly today? 🌍🛩️
Feel free to ask me anything related to your travel experience. 🛩️

Ask about flights, bookings, baggage...

30

Welcome Airline User

SKA-AIRLINES®

Fly Beyond Expectations

10:18:41 AM System Online



My Trips

View & manage bookings



Flight E-Ticket

Download & print anytime



Modify Booking

Reschedule or cancel flights



Baggage Info

Allowances & policies



Airport Info

Terminal & gate details



Support

Get help anytime

SKA Airlines Virtual Assistant

Online • Ready to Help

2026 SKA Airlines. All Rights Reserved.

Booking History Support & Assistance

Where would you like to fly today? Feel free to ask me anything related to your travel experience.

[i want my tickets](#)

Hello

You can view, download and print your booked tickets anytime.

Manage your upcoming and past bookings below:

[My Bookings](#)

Ask about flights, bookings, baggage...

3. New User Registration Module

The New User Registration Module provides a simple, secure, and efficient way for new users to join the SKA Flight Ticket Booking System. This module is designed to onboard new customers smoothly, enabling them to become registered users and unlock all the benefits of the platform.  

Key Features & Functionalities

User-Friendly Registration Form: New users fill out a comprehensive yet easy-to-use registration form to create their accounts. The form collects essential personal information necessary for account setup.  

Data Validation & Verification: The entered information undergoes thorough validation checks to ensure accuracy and prevent duplicate accounts. This includes email verification, mobile number checks, and secure password criteria enforcement.  

Secure Account Creation: Upon successful validation, accounts are securely created and users receive confirmation notifications, often via email, to activate their profiles.  

Access to Full User Functionalities: Registered users can immediately access all features such as flight booking, payment options, journey history, and profile management, making their transition from guests seamless and efficient.  

Privacy & Security: Personal data is handled with confidentiality, and secure protocols protect user information during registration.

User Experience & Interface

The registration page is designed with clean, intuitive layouts compatible with desktops and mobile devices alike, ensuring a smooth and hassle-free signup experience. Form validation provides instant feedback to users for a user-friendly onboarding process.  

Summary

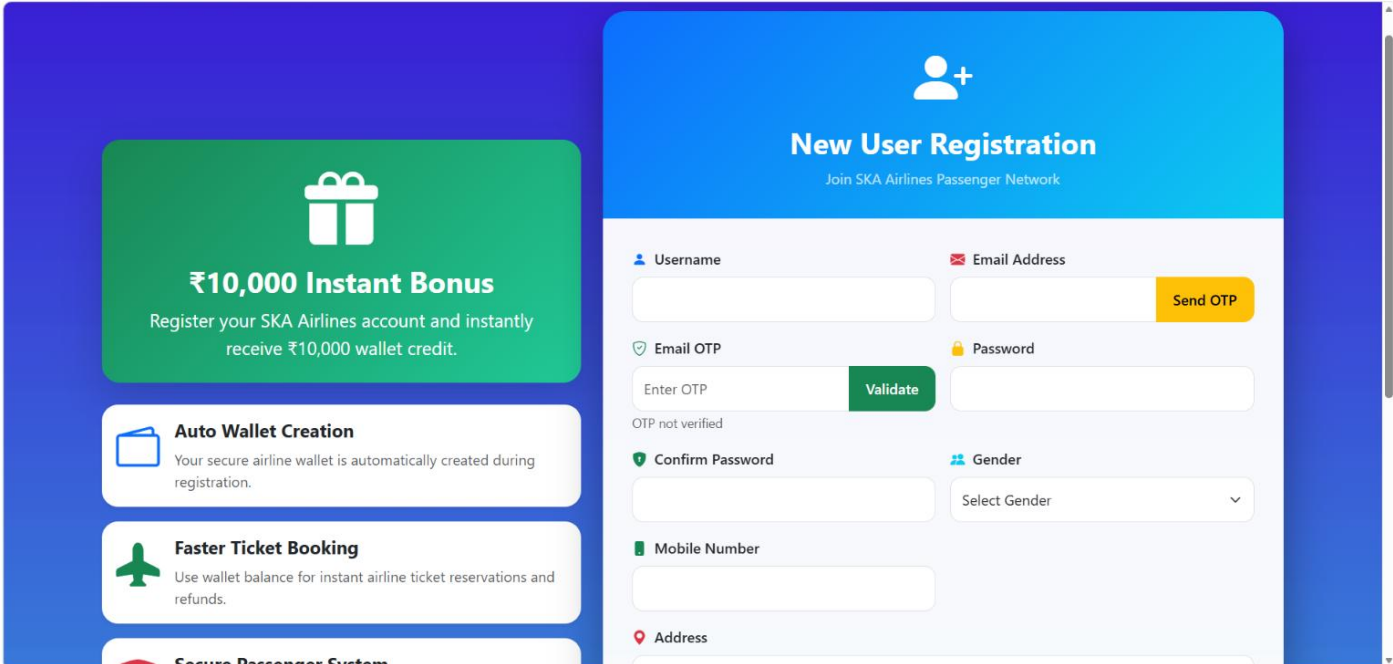
The New User Registration Module is a gateway to the SKA Flight Ticket Booking System, transforming visitors into valued registered customers through a secure, quick, and convenient signup process. This module lays the foundation for personalized and efficient flight booking experiences.   

Flight New-User/Register/Sign-Up Module

| guest_routes.py -----> Handles:

| |—— user_register_bp --> user_register.html, register_success.html

Flight New-User/ Sign-Up Module Screenshots



₹10,000 Instant Bonus
Register your SKA Airlines account and instantly receive ₹10,000 wallet credit.

Auto Wallet Creation
Your secure airline wallet is automatically created during registration.

Faster Ticket Booking
Use wallet balance for instant airline ticket reservations and refunds.

Secure Passenger System

New User Registration
Join SKA Airlines Passenger Network

Username

Email Address **Send OTP**

Email OTP **Validate**
OTP not verified

Confirm Password

Gender

Mobile Number

Address

4. Guest User Module

The Guest User Module caters to users who have not registered or logged in but still want to interact with the SKA Flight Ticket Booking System. This module provides essential access and functionalities to non-registered visitors, ensuring they can explore the system and its services before deciding to sign up.   

Key Features & Functionalities

View Flight Schedules & Availability: Guests can browse available flights, schedules, and routes without needing an account, helping them plan their travel efficiently.  

PNR Status Checking: Visitors can check their Passenger Name Record (PNR) status to track bookings made previously without logging in, providing convenient access to important information.  

User Registration Access: Guests are guided to create new accounts easily via clear prompts and links, facilitating smooth transition to registered user status.   

View News & Events: Access to the latest airline news, announcements, and special events keeps guests informed and engaged.  

Submit Anonymous Feedback: Guests can provide feedback or report issues anonymously, helping improve the service without the need for registration.  

User Experience & Interface

The Guest Module features clean, navigable pages designed to encourage exploration and easy access to information. Responsive design ensures seamless browsing across devices, including desktops, tablets, and smartphones.  

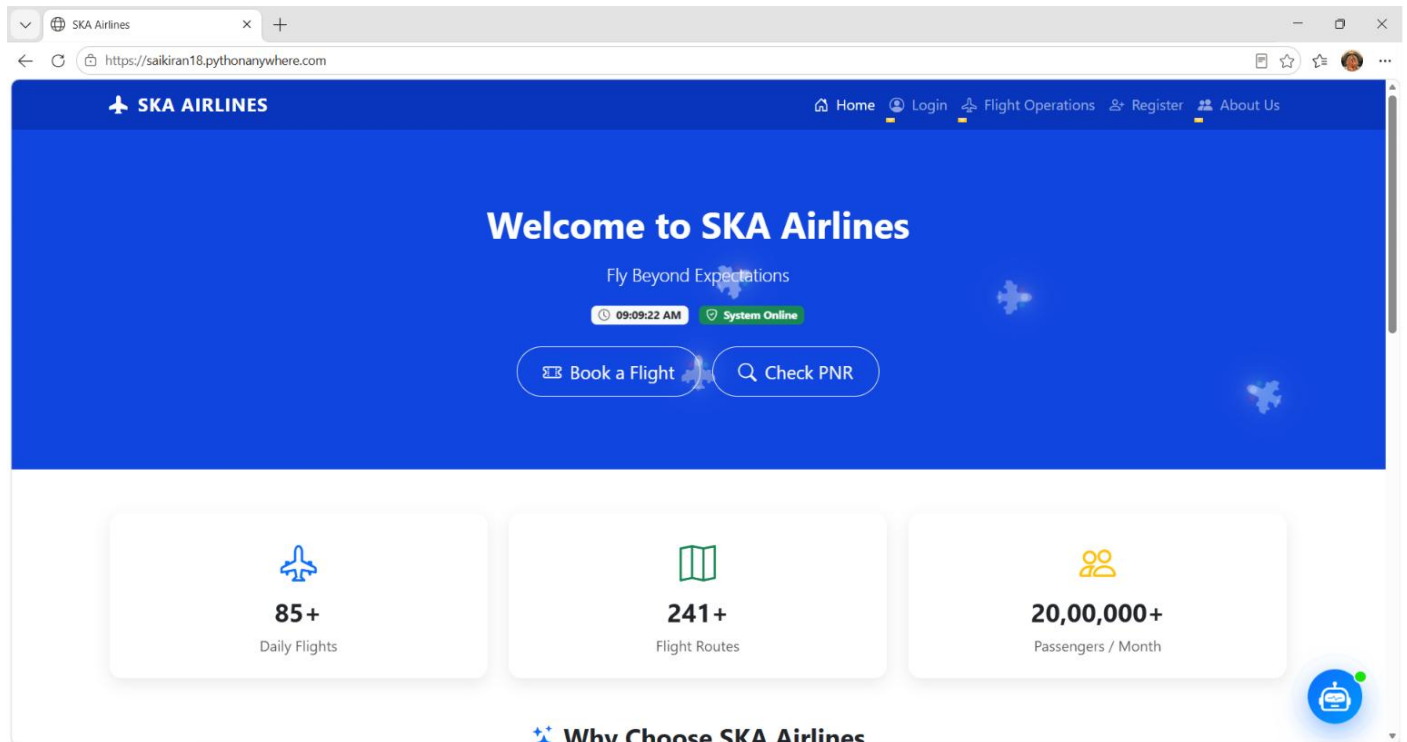
Summary

The Guest User Module offers a welcoming and informative gateway to the SKA Flight Ticket Booking System, allowing visitors to access essential travel information and interact anonymously before registering. This fosters engagement and builds trust with potential users.   

Flight Guest-User Module

```
|
|   |-- Guest Section
|   |   |-- guest_dashboard_bp
|   |   |-- guest_bp
|   |
|   |   guest_routes.py -----> Handles:
|   |   |
|   |   |-- user_register_bp      --> user_register.html, register_success.html
|   |   |-- feedback_index_bp    --> feedback_index.html, feedback_index_su.html
|   |   |-- plane_schedule_bp    --> plane_schedule.html, plane_schedule_su.html
|   |   |-- pnr_index_bp         --> pnr_index.html, pnr_index_su.html
|   |   |-- ticket_index_bp      --> ticket_index.html, ticket_index_su.html
|   |
|   |   guest_dashboard_bp: also maps routes directly to HTML
|   |   |-- airline_admins.html
|   |   |-- instructions.html
|   |   |-- newsevents_index.html
|   |
|   |-- Other Modules
|   |   |-- seat_avl_bp          --> seat_avl.html
|   |   |-- payment_qr_bp
|   |   |-- (future modules...)
```

Flight Guest-User Module Screenshots



The screenshot shows the SKA Airlines homepage. The browser address bar displays "https://saikiran18.pythonanywhere.com". The navigation bar includes links for Home, Login, Flight Operations, Register, and About Us. The main banner features the text "Welcome to SKA Airlines" and "Fly Beyond Expectations". Below this, there is a status bar showing "09:09:22 AM" and "System Online". Two buttons, "Book a Flight" and "Check PNR", are prominently displayed. The statistics section highlights "85+ Daily Flights", "241+ Flight Routes", and "20,00,000+ Passengers / Month". A "Why Choose SKA Airlines" section is partially visible at the bottom.

SKA AIRLINES

Home Login Flight Operations Register About Us

Welcome to SKA Airlines

Fly Beyond Expectations

09:09:22 AM System Online

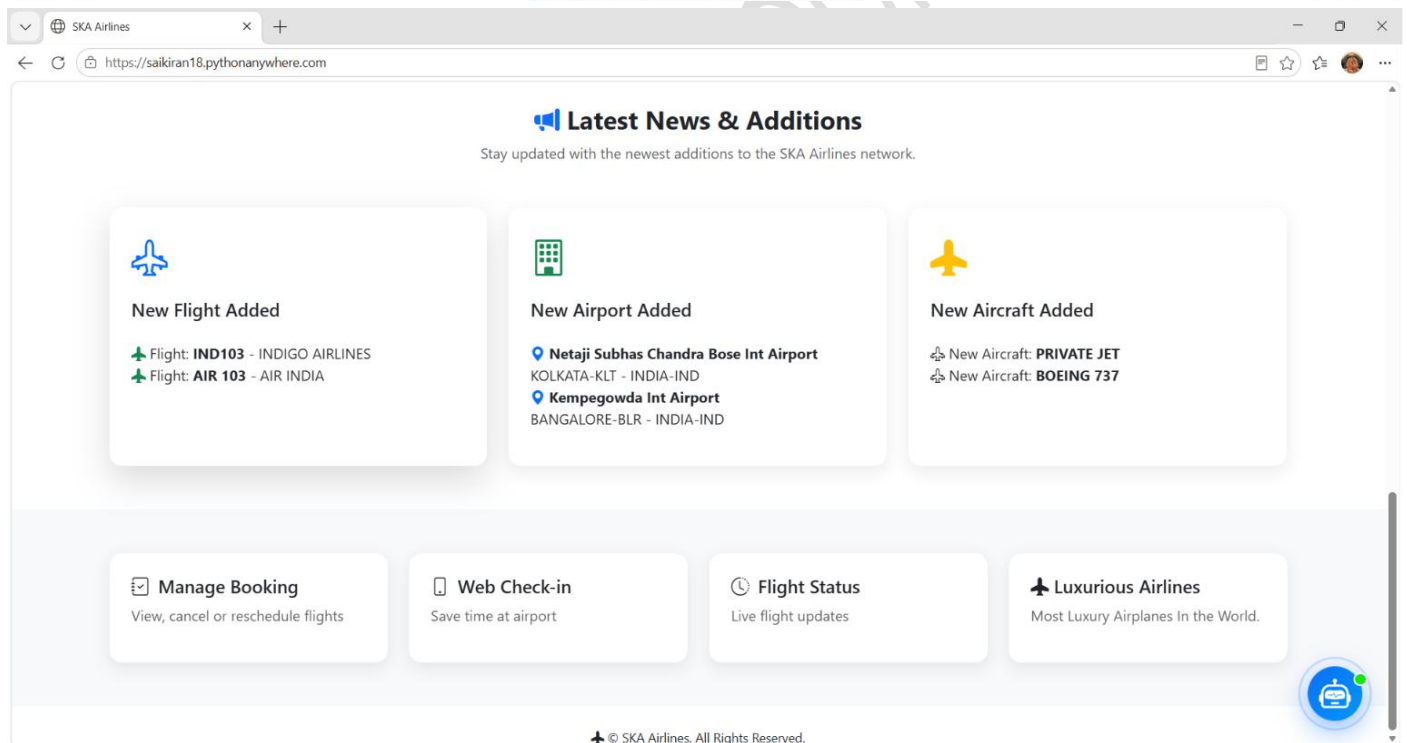
Book a Flight Check PNR

85+ Daily Flights

241+ Flight Routes

20,00,000+ Passengers / Month

Why Choose SKA Airlines



The screenshot shows the "Latest News & Additions" section of the SKA Airlines website. The browser address bar displays "https://saikiran18.pythonanywhere.com". The section is titled "Latest News & Additions" with the subtitle "Stay updated with the newest additions to the SKA Airlines network." It features three main news items: "New Flight Added" (listing IND103 - INDIGO AIRLINES and AIR 103 - AIR INDIA), "New Airport Added" (listing Netaji Subhas Chandra Bose Int Airport and Kempegowda Int Airport), and "New Aircraft Added" (listing PRIVATE JET and BOEING 737). Below these, there are four service tiles: "Manage Booking", "Web Check-in", "Flight Status", and "Luxurious Airlines". The footer includes the text "© SKA Airlines. All Rights Reserved."

Latest News & Additions

Stay updated with the newest additions to the SKA Airlines network.

New Flight Added

- Flight: **IND103** - INDIGO AIRLINES
- Flight: **AIR 103** - AIR INDIA

New Airport Added

- Netaji Subhas Chandra Bose Int Airport**
KOLKATA-KLT - INDIA-IND
- Kempegowda Int Airport**
BANGALORE-BLR - INDIA-IND

New Aircraft Added

- New Aircraft: **PRIVATE JET**
- New Aircraft: **BOEING 737**

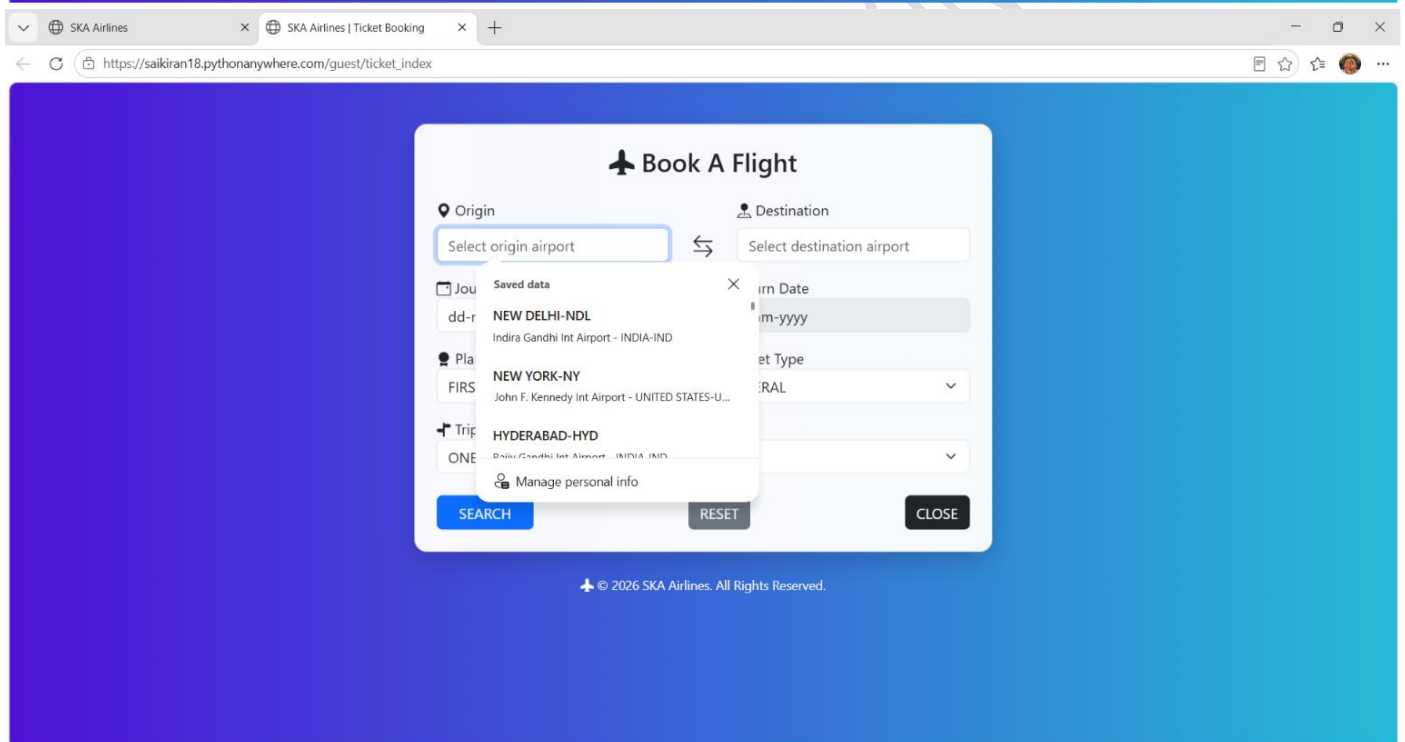
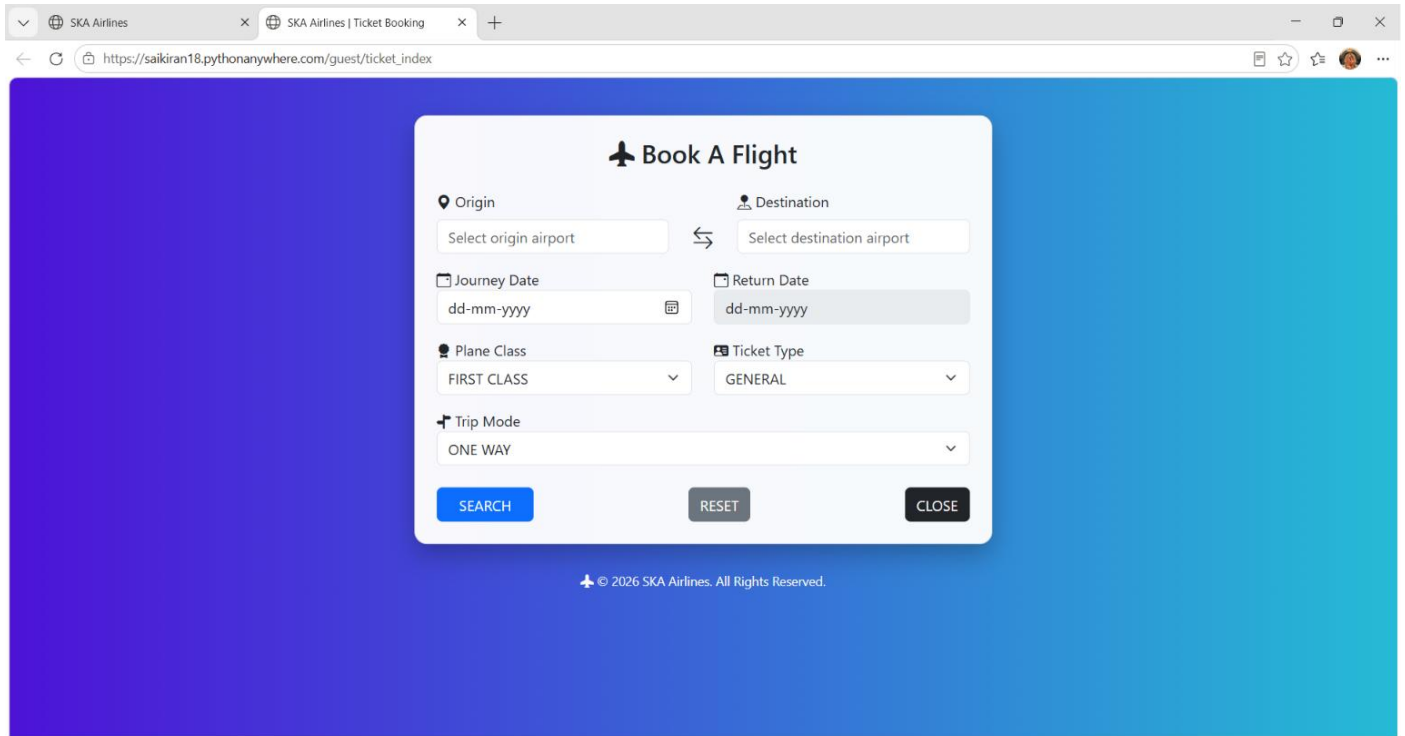
Manage Booking
View, cancel or reschedule flights

Web Check-in
Save time at airport

Flight Status
Live flight updates

Luxurious Airlines
Most Luxury Airplanes In the World.

© SKA Airlines. All Rights Reserved.



SKA Airlines

SKA Airlines | Ticket Booking

Flight Search Results

https://saikiran18.pythonanywhere.com/guest/guest/ticket_index

✈️

Modify Search Results

📍 Origin

NEW DELHI-NDL

✕

📍 Destination

NEW YORK-NY

✕

📅 Class

First Class

▼

📅 Journey Date

dd-mm-yyyy

📅

📄 Ticket Type

General

▼

🔄 Trip Mode

One Way

▼

📅 Return Date

dd-mm-yyyy

🔍 Modify Search

🔄 Reset

✖ Close

Origin ✈️

Indira Gandhi Int Airport
NEW DELHI-NDL, INDIA-IND

Destination ✈️

John F. Kennedy Int Airport
NEW YORK-NY, UNITED STATES-USA

✈️ Available Flights

✈️ Flight No	✈️ Flight Name	✈️ Aircraft	🕒 Departure	🕒 Arrival	⌚ Duration	₹ Fare	🕒 Flight Schedule
SKA100	SKA AIRLINES	BOEING 777	20:40 26/Jun/2026	17:35 27/Jun/2026	20H 55M	950	✈️ Flight Schedule
SOW100	SOWMYA AIRLINES	AIRBUS A320	01:04 26/Jun/2026	21:04 26/Jun/2026	20H 0M	900	✈️ Flight Schedule

SKA Airlines

PNR Enquiry | SKA Airlines

https://saikiran18.pythonanywhere.com/guest/pnr_index

☰

PNR Enquiry

Check your reservation details instantly

📄 PNR Number

279

🔍 Enquiry

🔄 Reset

✖ Close

← Back

📄 Public users can view limited reservation details only

✈️ © 2026 SKA Airlines. All Rights Reserved.

38

SKA Airlines

PNR Status | SKA Airlines

+

https://saikiran18.pythonanywhere.com/guest/guest/pnr_index

SKA AIRLINES®

Passenger Name Record (PNR) Status

Chhatrapati Shivaji Maharaj Int Airport

MUMBAI-MBI, INDIA-IND

Origin

John F. Kennedy Int Airport

NEW YORK-NY, UNITED STATES-USA

Destination

Journey Details

PNR

279

Flight

LUF100 - LUFTHANSA AIRLINES

Passengers

3

Aircraft

AIRBUS A380

Departure

08:00 05/Jun/2026

Class

FIRST CLASS

Return

None

Trip Mode

ONE WAY

Booking Status

Seat	Booking	Journey Status	Distance (KM)
1C	CNF	COMPLETED	12538 KM
1B	CNF	COMPLETED	12538 KM
1A	CNF	COMPLETED	12538 KM

SKA Airlines

Home Login Flight Operations Register About Us

Flight Schedule

Flight Operational Status

Airport Schedule

Welcome to SKA Airlines

Fly Beyond Expectations

09:14:24 AM System Online

Book a Flight Check PNR

85+

Daily Flights

241+

Flight Routes

20,00,000+

Passengers / Month

https://saikiran18.pythonanywhere.com/#

Why Choose SKA Airlines

39

SKA Airlines Flight Operational Status | SKA Airlines

https://saikiran18.pythonanywhere.com/guest/flight_operational_status

Flight Operational Status

SKA Airlines Public Flight Operations Monitoring System

✖ Cancelled Flight Monitoring

🕒 Delayed & Restricted Operations

✈ 1 Flights Affected

🔍 Search flight number, flight name, status, reason...

**AIR FRANCE**
✈ Flight Number : AIR 100
📄 Operational Reason
Technical Fault

📅 Affected From
19/Jun/2026

📅 Expected Until
26/Jun/2026

CANCELLED

✈ View Flight Schedule

SKA Airlines

Home Login Flight Operations Register About Us

Welcome to SKA Airlines

Fly Beyond Expectations

🕒 09:15:13 AM 🟢 System Online

🛫 Book a Flight

🔍 Check PNR

✈
85+
Daily Flights

🗺
241+
Flight Routes

👥
20,00,000+
Passengers / Month

Airline Admins

Instructions

News & Events

Feedback

SKA Wallet Offers

✈ Why Choose SKA Airlines

SYSTEM REQUIREMENTS

To operate efficiently, all software requires specific hardware components and software resources on the computer. These prerequisites are called system requirements and typically serve as guidelines rather than absolute rules. Most software defines two categories of system requirements: Minimum and Recommended.

With increasing demands for higher processing power and resources in newer software versions, system requirements tend to rise over time. Industry analysts suggest this trend drives more system upgrades than technological innovations themselves. Additionally, in a broader context, system requirements refer to the design specifications needed for a system or sub-system.

SOFTWARE REQUIREMENTS

Front End / Language: HTML5, CSS3, JavaScript

Backend Framework / Domain: Python (Flask)

Database: MySQL (or compatible SQL database)

Web Server: Gunicorn or Apache HTTP Server (for production), Flask built-in server for development

Server Logs: Standard logging formats (e.g., text, JSON)

Operating System: Windows 7, 10, 11, Linux, or macOS

HARDWARE REQUIREMENTS

Processor: Intel Core 2 Duo or later generations (i3, i5, i7, i9)

Main Memory (RAM): Minimum 4 GB

Hard Disk (Storage): Minimum 256 GB

Monitor: Colour Monitor (size as per user preference)

Mouse: Optical Mouse

FUNCTIONAL REQUIREMENTS

Functional requirements specify the actions the system must perform to fulfill the business needs of admins and users. These requirements are based on user needs and task analysis of the existing system.

Users must have a valid Admin/User ID and password to log in to the system. 

If an admin or user enters an incorrect password three consecutive times, the server will mark the attempts as unauthorized and restrict access. 

The system's behaviour is defined by the relationship between inputs (credentials, commands) and outputs (access granted/denied, system responses).

If an admin or user attempts to enter wrong credentials or access the system by directly copying the URL of an active session without proper login, the server will treat such attempts as unauthorized access.  

NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define how the system performs rather than what functions it performs. For the SKA Flight Ticket Booking System, these requirements ensure reliability, security, and quality user experience. Key non-functional requirements include:

Security: Protect confidential data such as admin and user credentials, booking information, and payment details through encryption and secure access controls. 

Performance: Ensure fast response times during booking, searches, and data retrieval to handle multiple concurrent users efficiently. 

Safety: Maintain data integrity and prevent unauthorized access or data loss.

Software Quality Attributes: Maintain reliability, usability, scalability, maintainability, and availability to offer a seamless and robust service. 

SYSTEM DESIGN

The SKA Flight Ticket Booking System's design is structured into logical and physical phases to deliver a scalable, secure, and user-friendly online flight reservation service.

1) System Design for Registered Users

The user requests access by entering the URL of the SKA Flight Booking Server. 

On the login page, the user enters their credentials (User ID and PIN). The system validates these credentials. 

If valid, the user is granted access to the main menu, where options include View Journey History, Book a Flight, View Flight Schedule, Update Profile, and more. 

For booking a flight, the user inputs Source (From), Destination (To), Travel Dates, and Class preferences, then submits the details

The system displays flight options; upon selecting a flight, the user confirms the booking by entering their secure PIN to finalize the transaction. 

Upon logout, users are recommended to clear browser cache for security reasons.  

2) System Design for Flight Admins

The admin requests the URL of the SKA Flight Booking Server. 

Admins input their credentials (Admin ID and PIN) on the login page; the system checks their validity. , If valid, admins access the admin dashboard featuring options like Add New Flight, Add New Airport, Modify Fare, View PNR Status, Manage Reservations, View Flight Schedules, and Print/Download Tickets. 

For example, when using the PNR enquiry menu, admins input a PNR number and submit it; the system retrieves and displays detailed booking information for confirmation.  

3) Development Phases

The system design is developed through two major phases:

a) Logical Design

Analyze current project requirements including data flows, file contents, volume, and frequencies, Define output specifications such as report formats and frequencies.

Specify input formats, content, and functions.

Establish edit, security, and control processes

Prepare and review an information flow walkthrough including inputs, outputs, controls, and implementation plans., Evaluate benefits, costs, timelines, and system constraints.

b) Physical Design

Develop the actual working system based on the logical design.

Specify input/output devices and media formats.

Design and implement the database with backup and recovery protocols

Plan and design the physical information flow and system walk-through.

Schedule system implementation and prepare conversion and training plans.

Define test procedures and specify any additional required hardware or software.

Update project benefits, costs, milestone dates, and constraints accordingly.

This comprehensive design approach ensures the SKA Flight Ticket Booking System is secure, reliable, user-friendly, and scalable to meet modern airline reservation needs efficiently.   

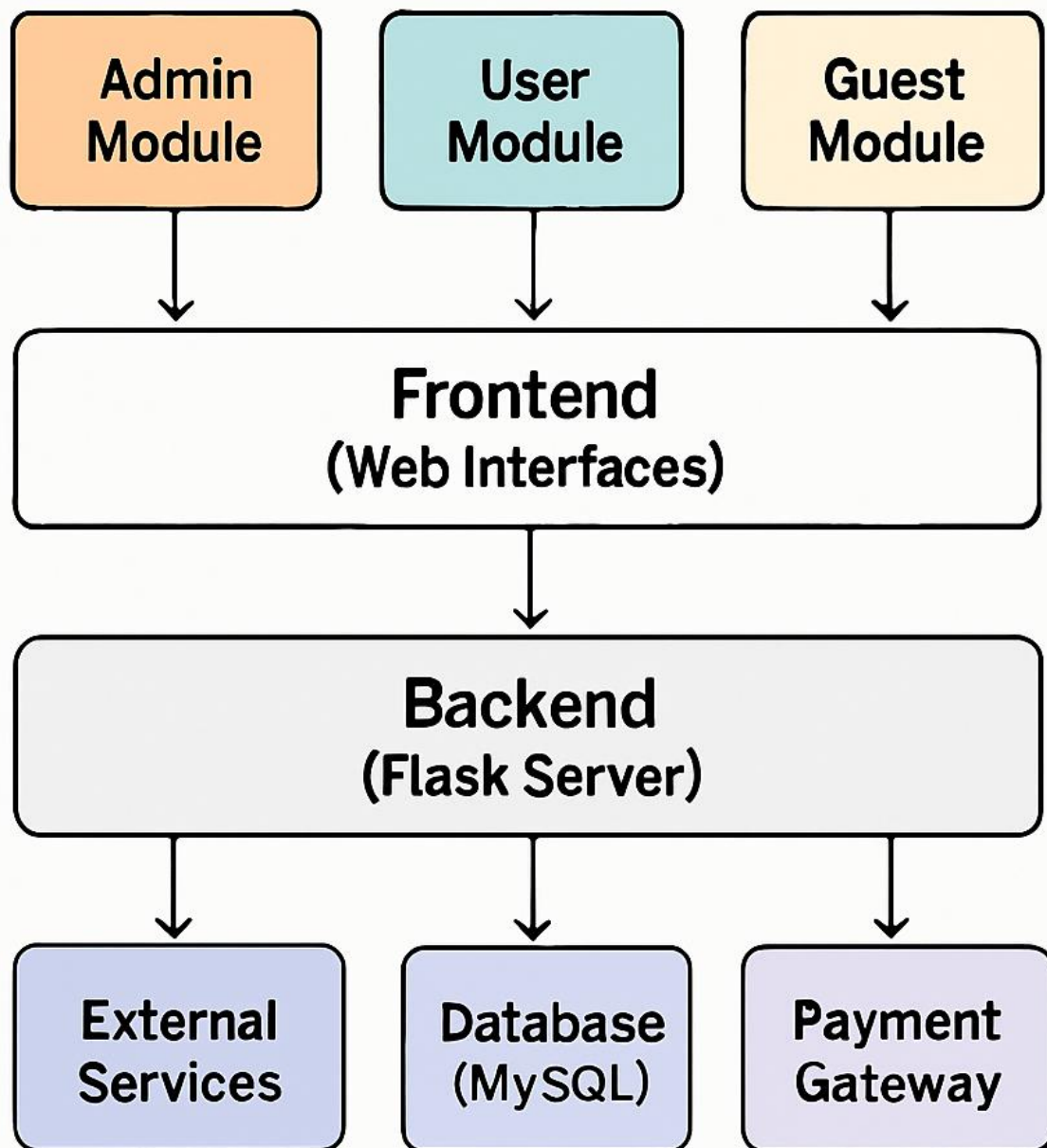
Flowchart: Flight Admin System Flow

1. Start
2. Admin requests URL of SKA Flight Booking System
3. Display Admin Login Page (Admin enters Admin ID and PIN)
4. Verify Admin ID and PIN
 - If Yes:
 - Show Admin Dashboard with options:
 - Add New Flight
 - Add New Airport
 - Modify Fare
 - View PNR Status
 - Manage Reservations
 - Print/Download Tickets
 - If No: Return to Login Page
5. If View PNR selected:
 - Input PNR Number
 - Submit request and display details
6. Logout
7. End

Flowchart: Registered User System Flow

1. Start
2. User requests the URL of SKA Flight Booking System
3. Display Login Page (User enters User ID and PIN)
4. Check if User ID and PIN are valid
 - If Yes:
 - Show Main Menu with options:
 - Book Flight
 - View Journey History
 - Update Profile
 - Other features
 - If No: Return to Login Page
5. If Book Flight selected:
 - Input Source, Destination, Travel Dates, and Class
 - Submit booking request
 - Confirm booking by entering secure PIN
6. Logout and clear browser cache for security
7. End

SKA FLIGHT BOOKING SYSTEM ARCHITECTURE DIAGRAM ✈️



Flight Reservation Booking Terms ✈️ 📋

- **Reservation:** The process of booking a seat on a specific flight for a particular date and route.
- **PNR (Passenger Name Record):** A unique identifier code generated for each booking that contains passenger details, itinerary, and ticket information. Used for tracking and managing bookings. 🔍
- **E-ticket (Electronic Ticket):** A digital version of a flight ticket issued electronically without a physical paper ticket. It contains all travel information and is proof of purchase. 📄
- **Fare:** The price paid by the passenger for a flight ticket, which can vary based on class, time, and other factors. 💰
- **Booking Confirmation:** The official acknowledgment by the airline that a reservation has been successfully made, often sent via email. ✅ ✉️
- **Cancellation:** The process by which a booked ticket is voided. Cancellation policies, fees, and refund eligibility depend on the fare rules. ❌ ↺
- **No-Show:** A passenger who does not show up for the flight without prior cancellation or notification. This may lead to forfeiture of fare. 🚫
- **Class of Service:** The travel class selected by the passenger, e.g., Economy, Business, or First Class, affecting fare and amenities. 🛋️
- **Connection / Layover:** A stop or break between flights during a journey, where passengers may switch planes. ⌚ ✈️
- **Seat Availability:** The current number of free seats on a flight for booking, updated in real-time. 📺
- **Payment Gateway:** A service that processes online payments securely when booking tickets, supporting multiple payment modes like bank transfer or QR code. 💳 🔒
- **Booking Reference:** Another term sometimes used interchangeably with PNR, referring to the unique code for a booking.

The Below table can be quick reference and user clarity.

Term	Definition
Reservation	The process of booking a seat on a specific flight for a particular date and route.
PNR (Passenger Name Record)	A unique booking code containing passenger and itinerary details, used to track/manage bookings. 🔍
E-ticket	An electronic version of a flight ticket; digital proof of purchase with travel details. 📄
Fare	The price paid for a flight ticket, which varies by class, timing, and other factors. 💰
Booking Confirmation	Official acknowledgment from the airline that the booking was successful, often via email. ✅✉️
Cancellation	Voiding a booked ticket; refund policies depend on fare ❌
No-Show	Passenger who misses the flight without cancellation, risking loss of fare. 🚫
Class of Service	Travel class chosen by passenger (Economy, Business, First), affecting price and amenities. 🛋️
Connection / Layover	A stop between flights allowing transfer to another flight. ⌚✈️
Seat Availability	Number of free seats on a flight updated in real-time. 🪑
Payment Gateway	Secure online service processing payments (bank transfers, QR codes). 💳🔒
Booking Reference	Another term for PNR; unique code used to identify a booking.

Flight Types / Terms

Term	Description
Domestic Flight	Flights operating within the same country, connecting cities or regions domestically.
International Flight	Flights that cross international borders, connecting different countries.
Direct Flight	A flight with no change of plane but may include stopovers for refueling or passenger boarding.
Non-Stop Flight	A flight that goes from origin to destination without any stops in between.
Connecting Flight	A journey with one or more stopovers requiring passengers to change planes at an intermediate airport.
Charter Flight	A non-scheduled flight arranged for specific groups or purposes, not part of the regular airline schedule.
Red-Eye Flight	Overnight flights typically departing late at night and arriving early morning.
Code Share Flight	A flight operated by one airline but marketed by one or more other airlines under their own flight number.
Round-Trip Flight	Travel from the origin to a destination and back to the origin on a single booking.
One-Way Flight	Travel from the origin to the destination only, without a return ticket.

Term	Description
Standby Flight	A flight where passengers wait for available seats, usually after missing an earlier flight.
Seasonal Flight	Flights operated only during specific seasons or periods (e.g., holiday seasons or summer).
Code Share Flight	A collaborative flight sharing arrangement where multiple airlines sell seats on the same flight.

This table can help users better understand the types of flights available and the terminology used in airline booking systems  .

1. Emirates Airbus A380 (International Long-Haul Flight)

- First Class: Private suites with doors for complete privacy, located at the front of the aircraft offering maximum comfort and luxury, Business Class: Spacious 2-2-2 seating with lie-flat seats, designed for long-haul travel comfort, Economy Class: High-density 3-4-3 layout maximizing capacity
- Flight Type: International, long-haul flights connecting major global hubs

2. Boeing 777 (International and Some Domestic Flights)

- Business Class: 2-3-2 seating arrangement, balancing space and capacity for medium to long-haul flights, Economy Class: Standard 3-3-3 layout with moderate legroom typical of many international and high-capacity domestic routes.
- Flight Type: Used commonly for international long-haul flights & busy domestic routes

3. Regional Jet (Primarily Domestic or Short-Haul Flights)

- Economy Class: Compact 1-2 or 2-2 seating arrangements designed for short flights with quick turnarounds.
- Flight Type: Domestic or regional flights connecting smaller airports and shorter distances, focusing on efficient use of space and quick service.

Flight Seats Layout System

Airlines use various seat layout configurations depending on aircraft type, flight duration, and service class. Understanding seat layouts helps users select preferred seats during booking.

Common Seat Layout Types

Layout Type	Description	Typical Configuration
Single Aisle (Narrow-body)	Aircraft with one central aisle. Common in short-haul flights.	3-3 (six seats per row, e.g., A-B-C aisle D-E-F)
Dual Aisle (Wide-body)	Aircraft with two aisles, used mainly for long-haul international flights.	2-4-2, 3-3-3, or 3-4-3 seat arrangements
Regional Jet	Smaller jets with fewer seats, suitable for short regional routes.	Typically 1-2 or 2-2 seat configurations

Seat Classes & Layouts

Class	Description	Layout Characteristics
Economy Class	Standard seating, higher seat density, basic legroom and amenities.	Rows of seats in tight configuration, usually 17-18 inches wide seats.
Premium Economy	Enhanced economy seats with more legroom and amenities.	Wider seats, more legroom, sometimes recline feature.
Business Class	Spacious seating with enhanced amenities, priority boarding, and meals.	Lie-flat or angled seats, 1-2-1 or 2-2-2 layouts common.
First Class	Most luxurious seating, private suites, top-tier service.	Private suites or enclosed seats, spacious, can include beds.

Seat Selection Features

- Window Seat: Located by the aircraft window, preferred for views and privacy.
- Aisle Seat: Located by the aisle, preferred for easier access.
- Exit Row: Extra legroom but may have restrictions on passengers (e.g., must be able to assist in emergencies).
- Bulkhead Seat: Located at cabin row dividing classes or sections, extra legroom but sometimes no under-seat storage.
- Seat Map Display: Interactive visual of seat availability shown during booking.
- Seat Availability Status: Indicated via colors (available, reserved, blocked).

Example Seat Map Notation

- A-F: Seat letters typically represent seats from left to right starting from the window.
- Row numbers: Numbered front to back of the cabin. E.g., row 12 A-F means seats in row 12 from window-left (A) to window-right (F).
-

Including detailed seat layout information enhances user experience by helping travelers choose preferred seats confidently during booking.

Let me know if you want me to create sample seat maps or diagrams! Here is an overview of common Seat Layout Systems used in airline reservation platforms, including their typical configurations and terminology, for inclusion in your SKA Flight Ticket Booking System documentation:

Seat Layout Systems

Airlines use various seat layout configurations depending on aircraft type, flight duration, and service class. Understanding seat layouts helps users select preferred seats during booking.

Common Seat Layout Types

Layout Type	Description	Typical Configuration
Single Aisle (Narrow-body)	Aircraft with one central aisle. Common in short-haul flights.	3-3 (six seats per row, e.g., A-B-C aisle D-E-F)
Dual Aisle (Wide-body)	Aircraft with two aisles, used mainly for long-haul international flights.	2-4-2, 3-3-3, or 3-4-3 seat arrangements
Regional Jet	Smaller jets with fewer seats, suitable for short regional routes.	Typically 1-2 or 2-2 seat configurations

Seat Classes & Layouts

Class	Description	Layout Characteristics
Economy Class	Standard seating, higher seat density, basic legroom and amenities.	Rows of seats in tight configuration, usually 17-18 inches wide seats.
Premium Economy	Enhanced economy seats with more legroom and amenities.	Wider seats, more legroom, sometimes recline feature.
Business Class	Spacious seating with enhanced amenities, priority boarding, and meals.	Lie-flat or angled seats, 1-2-1 or 2-2-2 layouts common.
First Class	Most luxurious seating, private suites, top-tier service.	Private suites or enclosed seats, spacious, can include beds.

Seat Selection Features

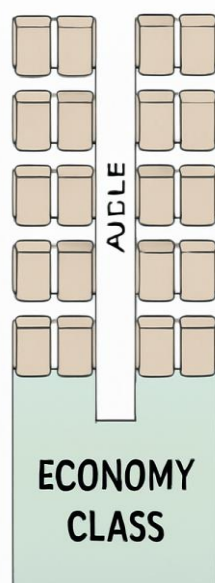
- Window Seat: Located by the aircraft window, preferred for views and privacy.
- Aisle Seat: Located by the aisle, preferred for easier access.
- Exit Row: Extra legroom but may have restrictions on passengers (e.g., must be able to assist in emergencies).
- Bulkhead Seat: Located at cabin row dividing classes or sections, extra legroom but sometimes no under-seat storage.
- Seat Map Display: Interactive visual of seat availability shown during booking.
- Seat Availability Status: Indicated via colors (available, reserved, blocked).

Example 1 Seat Map Notation

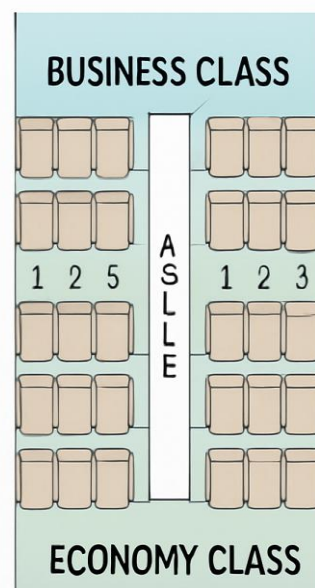
- A-F: Seat letters typically represent seats from left to right starting from the window.
- Row numbers: Numbered front to back of the cabin. E.g., row 12 A-F means seats in row 12 from window-left (A) to window-right (F).

Including detailed seat layout information enhances user experience by helping travelers choose preferred seats confidently during booking.

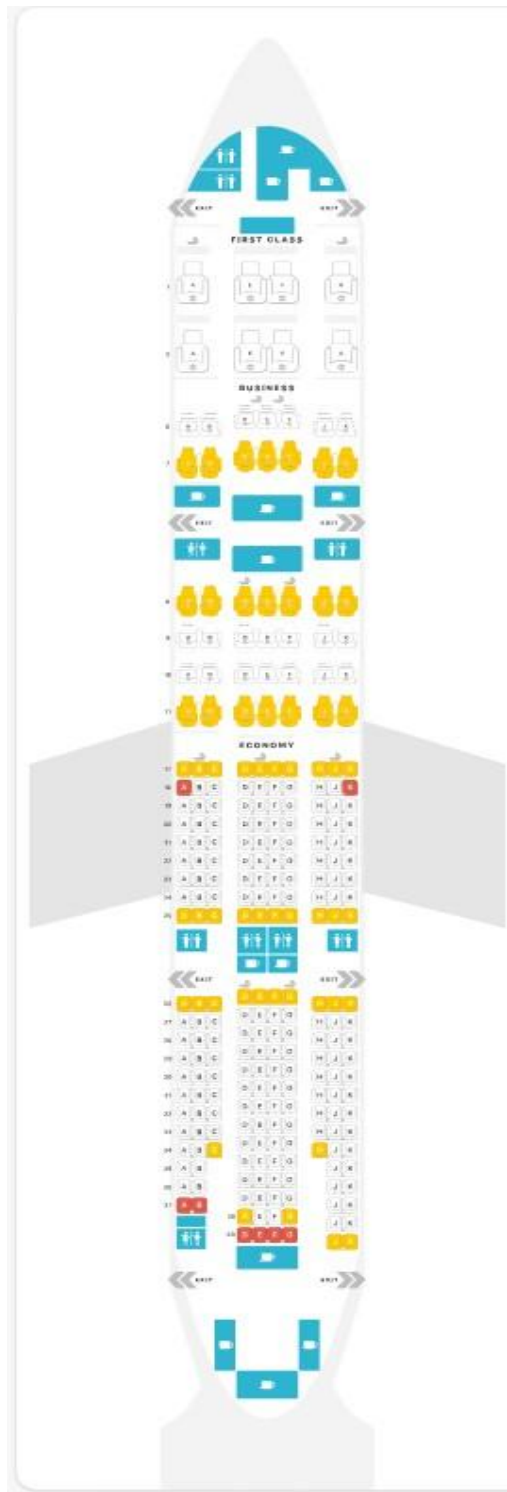
**TYPICAL DOMESTIC
FLIGHT LAYOUT**



**TYPICAL INTERNATIONAL
FLIGHT LAYOUT**



Example 2 Seat Map Notation



	A	B	AISLE	C	D
1	A	B	C	C	C
2	B	B	B	A	D
3	B	B	E	A	D
4	B	B	E	A	D
5	B	B	E	A	D
6	B	B	E	A	D
7	B	B	E	A	D
8	B	B	E	A	D
9	B	B	E	A	D
10	B	B	E	A	D
11	B	B	E	A	D
12	B	B	E	A	D
13	B	B	E	A	D
14	B	B	E	A	D
15	A	B	E	F	F
	A	B	C	C	F

Customer's Obligations

1. The customer must keep their Username and Password confidential. The bank assumes that login with valid credentials represents a session initiated by the customer.
 2. Transactions executed through a valid session are legally binding on the customer as originating from them. 
 3. The customer must never attempt or allow others to access the SKA Flight Ticket Booking System through unlawful means. 
- While registering on the website, users must note down their User Number, Username, and Password.
 - Tickets booked under PHC (Physically Handicapped Card) or Ladies Quota require passengers to carry valid identity proof. 
 - When changing passwords, users must enter their old password, then the new password with confirmation. 
 - PNRs with full waiting list status are automatically dropped and refunded. 
 - Never purchase e-tickets from unauthorized persons. 
 - SKA or its affiliates will never ask for your personal bank or security details. 
 - If a flight delay exceeds 4 hours, refunds are admissible.  
 - In case of flight cancellation, the full refund is automatically credited to the user's account.  

Dos & Don'ts

1. Keep your User ID and password strictly confidential. Any loss due to negligence is the customer's responsibility; SKA Flight Booking is not liable. 
2. Choose strong, unique passwords avoiding easily guessable personal information like name, birthdate, or phone number. Memorize passwords rather than writing them down.  
3. Never leave your computer unattended during an active session, as others may access your account.  

Safe Online Flight Booking Tips

- Check that the URL in your browser begins with “https”, where “s” means secured. 
- Look for the padlock icon in the browser’s address bar and verify the security certificate by clicking it. 
- Do not enter sensitive information in any pop-up windows. 
- A green address bar indicates a secured website. 

Beware of Phishing Attacks

- Phishing is a fraudulent attempt via email, phone calls, or SMS to steal your confidential information.  
- SKA or its representatives never request personal information, passwords, or one-time SMS (OTP) passwords via email, SMS, or phone calls.  
- Such communications are attempts to fraudulently withdraw money or access your account. Never respond! 
- Change your internet booking and banking passwords regularly. 
- Always check your last login date and time on the secure post-login page. 

Important Reminder

Scammers often send fake emails or texts pretending to be trusted companies like banks or utilities. They trick users into clicking malicious links or opening attachments to steal passwords or personal info.

Stay vigilant and protect your information!  

THE SKA FLIGHT TICKET BOOKING SYSTEM HIERARCHY / PROCESS

The SKA Flight Ticket Booking System is designed with a clear hierarchy of modules and a smooth process flow that ensures efficient flight reservation and management for users, admins, and guests.

System Hierarchy Overview

1. User Interface Layer (Frontend)

- Web pages and dashboards accessed by
 - Registered Users 
 - Flight Admins 
 - Guest Users 

2. Application Layer (Backend)

- Flask framework handling business logic including:
 - User authentication and authorization
 - Flight search and booking
 - Payment processing (bank transfers, QR payments)
 - Ticket management and cancellations
 - Feedback collection
 - Admin operations (flight/airport/fare management, PNR enquiries)

3. Data Layer (Database)

- MySQL database storing:
 - User profiles and credentials
 - Flight schedules and aircraft info
 - Booking and ticket details (PNRs, e-tickets)
 - Payments and refunds
 - Feedback and logs

4. External Services Integration

- Payment gateways for secure transactions
- Email services for ticket confirmations and notifications

System Process Flow

1. User / Guest Access

- Request website URL and reach landing page.
- Register (new user), log in (registered user/admin), or browse as guest.

2. Authentication & Authorization

- User/Admin submits credentials.
- System validates via backend and grants access.

3. Main Operations

- Users search flights by criteria (source, destination, date).
- Users select preferred flight and proceed to book.
- Multiple payment options offered: bank accounts, QR code.
- Tickets booked and e-tickets generated. Ticket status emailed.

4. Booking Management

- Users can view journey history, update profile, cancel bookings.
- Admins manage flights, fares, PNR enquiries, cancellations, refunds.

5. Security & Monitoring

- Secure sessions maintained; improper access attempts logged.
- Password and data confidentiality enforced strictly.

This hierarchy and process ensure a secure, efficient, and user-friendly flight booking experience for all stakeholders across the SKA Flight Ticket Booking System. ✈️ 🌐 ✅

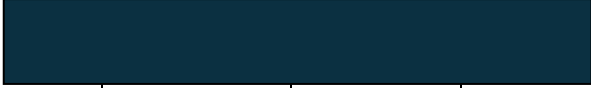
PROJECT SCHEDULING

I estimated the time and resources needed for the SKA Flight Ticket Booking System project while planning the schedule. All project activities must be arranged in a coherent sequence, meaning tasks are organized logically and clearly for easy understanding and smooth execution.

Initial project estimates are often optimistic, assuming favorable conditions with no major threats or problems during development.

Project Timeline

- Duration: June 2025 to August 2026
- The project will be implemented in 4 major phases to ensure systematic development and deployment.

Months	JUN-JUL-AUG	SEP-OCT	NOV-DEC	JAN-FEB	MAR-APR-MAY	JUN-JUL
Information Gathering						
Requirements Gathering						
Planning						
Analysis						
Design						
Coding						
Testing						
Implementation						
Documentation						

Benefits of Online Flight Ticket Booking ✈️ 💻

Many of us lead busy lives. Some wake up before dawn, preparing ourselves and our families for the day ahead. We rush to work, get the kids to school, and at the end of the day, rush home only to prepare for the next day. After such a hectic schedule, the last thing anyone wants is to spend time waiting in long lines at the airport or ticket counters.

That's where Online Flight Ticket Booking with SKA comes in. Many benefits of booking your flight tickets online are obvious:

- You don't have to wait in line. ⌚ ❌
- You don't need to plan your day around the airport or ticket office hours. 🕒
- You can check your flight information anytime, not just when you receive a statement. 📱

There are also some hidden benefits:

- If you are new to flight booking, using the SKA system helps you manage your account and travel plans while observing your spending patterns. 📊
- Online booking allows you to monitor your flights and schedule daily if desired. Keeping close tabs on your bookings means you're always aware of your travel status. 📅
- For experienced travelers, this system helps prevent surprises like running out of travel funds unexpectedly. 💰
- It's also useful to track any service charges, time savings, or fees you may have incurred, helping you manage your travel budget better. ⌚ 💰

Booking flights online with SKA is a smarter, faster, and more convenient way to plan your travels! ✈️ ✨

More Benefits of Online Flight Ticket Booking with SKA ✈️ 🌐

Convenience Anytime, anywhere 🕒 📱

- Book your flights 24/7 from the comfort of your home, office, or on the go using any device—desktop, tablet, or smartphone.
- No more rushing to airport counters or dealing with long queues; your entire booking is just a few clicks away.

Instant Access to Information 🔍 ✈️

- View up-to-date flight schedules, delays, cancellations, and gate changes in real-time.
- Receive timely email and SMS alerts about your bookings and travel updates directly from SKA.

Multiple Payment Options 💳 📱

- Pay securely using multiple methods including bank transfers, credit/debit cards, and QR code payments.
- Choose the payment method that best fits your convenience and comfort.

Easy Modifications and Cancellations 🔄 ✖️

- Modify your bookings or cancel tickets anytime online without hassle.
- Get automatic notifications and refunds processed smoothly when applicable.

Personalized Travel Experience 👤 🎯

- Save your preferences and travel history for faster bookings next time.
- Access special offers and loyalty rewards tailored just for you.

Secure and Confidential 🔒 🛡️

- Protect your personal and payment information with SKA's advanced security features.
- SKA never asks for your full bank details outside secure payment gateways to keep your data safe.

Environmentally Friendly Option 🌱 🌍

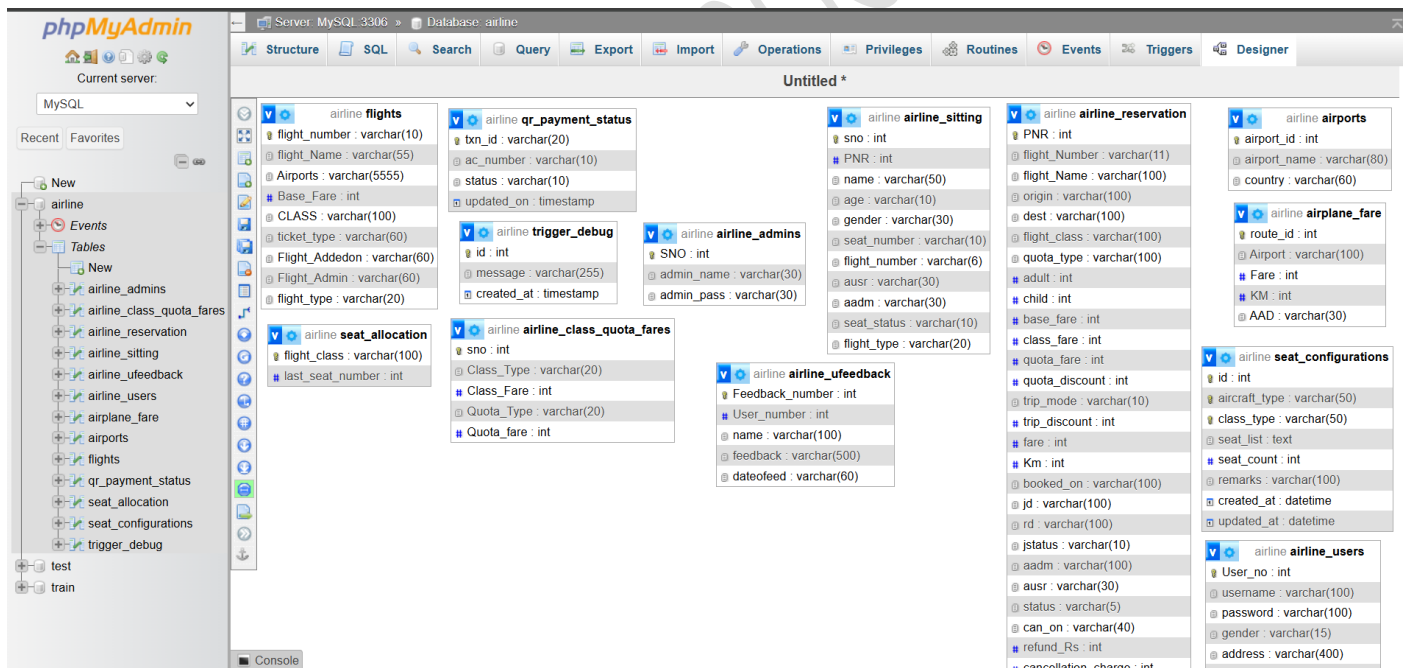
- Reduce paper waste by using e-tickets instead of printed tickets.
- Travel greener by embracing digital solutions powered by SKA.

Booking online with SKA Flight Ticket Booking System is more than just purchasing a ticket—it's the start of a smooth, secure, and personalized travel journey.

The SKA Flight Booking System's database is the core component that powers the entire application by securely managing and organizing all essential data. It includes multiple interconnected tables designed to handle user profiles, admin credentials, flight schedules, seat layouts, ticket bookings, payments, and feedback.

Each table stores specific information—such as passenger details, flight types, fare structures, and transaction records—allowing for efficient querying and seamless data integrity across the platform. The relational design supports dynamic features like real-time seat availability, automated PNR generation, and flexible payment processing.

By structuring data meticulously, the database ensures fast access, reliable updates, and smooth synchronization between backend processes and frontend user interfaces, ultimately delivering a robust and responsive flight booking experience.



The screenshot shows the phpMyAdmin interface for a MySQL database named 'airline'. The left sidebar lists the following tables: airline_admins, airline_class_quota_fares, airline_reservation, airline_sitting, airline_ufeedback, airports, flights, qr_payment_status, seat_allocation, seat_configurations, and trigger_debug. The main area displays the structure of several tables:

- airline flights**: flight_number (varchar(10)), flight_Name (varchar(55)), Airports (varchar(5555)), Base_Fare (int), CLASS (varchar(100)), ticket_type (varchar(60)), Flight_Admin (varchar(60)), flight_type (varchar(20)).
- airline qr_payment_status**: txn_id (varchar(20)), ac_number (varchar(10)), status (varchar(10)), updated_on (timestamp).
- airline trigger_debug**: id (int), message (varchar(255)), created_at (timestamp).
- airline airline_admins**: SNO (int), admin_name (varchar(30)), admin_pass (varchar(30)).
- airline airline_class_quota_fares**: sno (int), Class_Type (varchar(20)), Class_Fare (int), Quota_Type (varchar(20)), Quota_fare (int).
- airline airline_sitting**: sno (int), PNR (int), name (varchar(50)), age (varchar(10)), gender (varchar(30)), seat_number (varchar(10)), aadm (varchar(30)), ausr (varchar(30)), seat_status (varchar(10)), flight_type (varchar(20)).
- airline airline_reservation**: PNR (int), flight_Number (varchar(11)), flight_Name (varchar(100)), origin (varchar(100)), dest (varchar(100)), flight_class (varchar(100)), quota_type (varchar(100)), adult (int), child (int), base_fare (int), class_fare (int), quota_fare (int), quota_discount (int), trip_mode (varchar(10)), trip_discount (int), fare (int), Km (int), booked_on (varchar(100)), jd (varchar(100)), rd (varchar(100)), jstatus (varchar(10)), aadm (varchar(100)), ausr (varchar(30)), status (varchar(5)), can_on (varchar(40)), refund_Rs (int), cancellation_charge (int).
- airline airports**: airport_id (int), airport_name (varchar(80)), country (varchar(60)).
- airline airplane_fare**: route_id (int), Airport (varchar(100)), Fare (int), KM (int), AAD (varchar(30)).
- airline seat_configurations**: id (int), aircraft_type (varchar(50)), class_type (varchar(50)), seat_list (text), seat_count (int), remarks (varchar(100)), created_at (datetime), updated_at (datetime).
- airline airline_users**: User_no (int), username (varchar(100)), password (varchar(100)), gender (varchar(15)), address (varchar(400)).

THE SKA FLIGHT BOOKING SYSTEM APPLICATION CALLS/FLOW

airlines.py <-- Main entry, initializes and registers all blueprints

```
|
|
|  └─ Admin Section
|
|    └─ admin_login_bp    <-- Login functions
|
|    └─ admin_logout_bp   <-- Logout logic
|
|    └─ admin_dashboard_bp <-- HTML renderings (see below)
|
|    └─ admin_bp          <-- Route connectors
|
|  admin_routes.py -----> Imports blueprints and submodules:
|
|  |
|
|  └─ new_flight_bp      --> new_flight.html
|
|  └─ add_airport_bp     --> add_airport.html
|
|  └─ add_fare_bp        --> add_fare.html
|
|  └─ change_fare_bp     --> change_fare.html
|
|  └─ ticket_admin_bp    --> ticket_admin.html, ticket_admin1.html, ...
|
|  └─ ticket_download_admin_bp --> ticket_download_admin.html
|
|  └─ user_booking_history_bp --> user_booking_history.html
|
|  └─ airplane_airports_bp --> airline_airports_su.html
|
|  └─ admin_can_ticket_bp --> admin_can_ticket.html, admin_can_ticket1.html, ...
|
|  └─ payment_gateway_tkt_can_admin_bp
|
|  └─ airline_fund_bp     --> airline_fund_su.html
|
|  └─ (other submodules...)
|
|  └─ User Section
|
|    └─ user_login_bp
|
|    └─ user_logout_bp
|
|    └─ user_dashboard_bp
|
|    └─ user_bp
|
|  user_routes.py -----> Handles:
```

- | ├── ticket_user_bp --> ticket_user.html, ticket_user1.html, ...
- | ├── payment_gateway_bp --> payment_gateway_su.html
- | ├── ticket_userdb_bp
- | ├── user_can_ticket_bp --> user_can_ticket.html, user_can_ticket1.html, ...
- | ├── user_journey_history_bp --> user_journey_history_su.html, ...
- | ├── user_feedback_bp --> user_feedback.html, user_feedback_su.html
- | ├── user_update_mbl_num_bp --> user_update_mbl_num.html, ...
- | ├── user_update_password_bp --> user_update_password.html, ...
- | └── qr_payment_bp --> qr_payment_su.html
- └── Guest Section
- | ├── guest_dashboard_bp
- | └── guest_bp
- | guest_routes.py -----> Handles:
- | |
- | ├── user_register_bp --> user_register.html, register_success.html
- | ├── feedback_index_bp --> feedback_index.html, feedback_index_su.html
- | ├── plane_schedule_bp --> plane_schedule.html, plane_schedule_su.html
- | ├── pnr_index_bp --> pnr_index.html, pnr_index_su.html
- | └── ticket_index_bp --> ticket_index.html, ticket_index_su.html
- | guest_dashboard_bp: also maps routes directly to HTML
- | ├── airline_admins.html
- | ├── instructions.html
- | └── newsevents_index.html
- |
- └── Other Modules
- ├── seat_avl_bp --> seat_avl.html
- ├── payment_qr_bp
- └── (future modules...)

THE SKA FLIGHT BOOKING SYSTEM ENTIRE APPLICATION ENVIRONMENT

Airlines/

- |
- | — admin/
- | | — add_airport.py
- | | — add_fare.py
- | | — admin_can_ticket.py
- | | — admin_can_ticket1.py
- | | — admin_delete_feedback.py
- | | — admin_login.py
- | | — admin_routes.py
- | | — admin_view_feedback.py
- | | — airline_fund.py
- | | — airplane_airports.py
- | | — airports.py
- | | — change_fare.py
- | | — logout.py
- | | — new_flight.py
- | | — payment_gateway_tkt_can_admin.py
- | | — pnr_enq_admin.py
- | | — routes.py
- | | — seat_avl.py
- | | — ticket_admin.py
- | | — ticket_admindb.py
- | | — ticket_download_admin.py
- | | — usernumber.py
- | | — user_booking_history.py
- | | — utils.py

```
|  └─ __init__.py
|
|  └─ Extra Programs/
|
|  └─ Application_Issues_bugs
|
|  └─ brbr.txt
|
|  └─ check things in project.txt
|
|  └─ dateand time.txt
|
|  └─ detailed program.txt
|
|  └─ Email.txt
|
|  └─ from flask import Blueprint, render.txt
|
|  └─ good logic.txt
|
|  └─ mix.txt
|
|  └─ next steps.txt
|
|  └─ Old trigger dynamic.txt
|
|  └─ guest/
|
|  └─ feedback_index.py
|
|  └─ guest_routes.py
|
|  └─ plane_schedule.py
|
|  └─ pnr_index.py
|
|  └─ routes.py
|
|  └─ ticket_index.py
|
|  └─ user_register.py
|
|  └─ guest/__pycache__/
|
|  └─ feedback_index.cpython-312.pyc
|
|  └─ guest_routes.cpython-312.pyc
|
|  └─ plane_schedule.cpython-312.pyc
```


- | └─ pnr_index.cpython-312.pyc
- | └─ routes.cpython-312.pyc
- | └─ routes_py.cpython-312.pyc
- | └─ seats_avl.cpython-312.pyc
- | └─ ticket_index.cpython-312.pyc
- | └─ user_register.cpython-312.pyc
- |
- | └─ MYSQL/
- | | └─ airline (30).sql
- | | └─ airline 28.sql
- | | └─ airline 33.sql
- | | └─ airline 34.sql
- |
- | └─ Paper Work/
- | | └─ IMG_20240905_173915450.jpg
- | | └─ IMG_20240905_173943677.jpg
- | | └─ IMG_20240905_173954705.jpg
- | | └─ IMG_20240905_174000415.jpg
- | | └─ IMG_20240905_174005665.jpg
- | | └─ IMG_20240905_174011513.jpg
- |
- | └─ routes/
- | | └─ db_config.py
- | | └─ email_cnf_can.py
- | | └─ oracle_db_config.py
- | | └─ payment_gateway_Qr.py
- | | └─ qr_code_generator.py
- |

```
└─ routes/__pycache__/
|   └─ db_config.cpython-312.pyc
|   └─ email_cnf.cpython-312.pyc
|   └─ email_cnf_can.cpython-312.pyc
|   └─ oracle_db_config.cpython-312.pyc
|   └─ payment_gateway_Qr.cpython-312.pyc
|   └─ qr_code_generator.cpython-312.pyc
|
└─ sample tickets/
|   └─ SKA AIRLINES Boarding Pass (11).pdf
|   └─ SKA-AIRLINES Boarding Pass (8).pdf
|   └─ SKA-AIRLINES Boarding Pass (9).pdf
|
└─ Screenshots/
|   └─ Screenshot 2025-07-30 080941.png
|   └─ Screenshot 2025-07-30 081116.png
|   └─ Screenshot 2025-07-30 081129.png
|   └─ Screenshot 2025-07-30 081142.png
|
└─ static/
|   └─ aeroplane9.jpg
|   └─ airport1.jpg
|   └─ airport2.jpg
|   └─ airport3.jpg
|   └─ airport4.jpg
|
└─ templates/
|   └─ adminlogin.html
```

- | └─ airline_admins.html
- | └─ base.html
- | └─ error.html
- | └─ index.html
- | └─ test.html
- | └─ ticket_index.html
- | └─ userlogin.html
- | └─ admin/
 - | | └─ add_airport.html
 - | | └─ add_fare.html
 - | | └─ admin_can_ticket.html
 - | | └─ admin_can_ticket1.html
 - | | └─ admin_can_ticket2.html
 - | | └─ admin_can_ticket_su.html
 - | | └─ admin_delete_feedback.html
 - | | └─ admin_delete_feedback_su.html
 - | | └─ admin_instructions.html
 - | | └─ admin_news_events.html
 - | | └─ admin_view_feedback_su.html
 - | | └─ airline_airports_su.html
 - | | └─ airline_fund_su.html
 - | | └─ airport_success.html
 - | | └─ change_fare.html
 - | | └─ change_fare_su.html
 - | | └─ fare_success.html
 - | | └─ new_flight.html
 - | | └─ pnr_enq_admin.html
 - | | └─ pnr_enq_admin_su.html

| | ├── seat_avl.html
| | ├── seat_avl_su.html
| | ├── success.html
| | ├── ticket_admin.html
| | ├── ticket_admin1.html
| | ├── ticket_admin2.html
| | ├── ticket_admin3.html
| | ├── ticket_admin_su.html
| | ├── ticket_download_admin.html
| | ├── ticket_download_admin_su.html
| | ├── user_booking_history.html
| | ├── user_booking_history_su.html
| | └── welcome_admin.html
| ├── guest/
| | ├── feedback_index.html
| | ├── feedback_index_su.html
| | ├── instructions.html
| | ├── newsevents_index.html
| | ├── plane_schedule.html
| | ├── plane_schedule_su.html
| | ├── pnr_index.html
| | ├── pnr_index_su.html
| | ├── register_success.html
| | ├── ticket_index.html
| | ├── ticket_index_su.html
| | └── user_register.html
| ├── partials/
| | └── seat_availability.html

| └─ user/
| | └─ payment_gateway_su.html
| | └─ qr_payment_su.html
| | └─ ticket_user.html
| | └─ ticket_user1.html
| | └─ ticket_user2.html
| | └─ ticket_user3.html
| | └─ ticket_user4.html
| | └─ ticket_user_su.html
| | └─ user_can_ticket.html
| | └─ user_can_ticket1.html
| | └─ user_can_ticket2.html
| | └─ user_can_ticket_su.html
| | └─ user_feedback.html
| | └─ user_feedback_su.html
| | └─ user_journey_history1_su.html
| | └─ user_journey_history_can1_su.html
| | └─ user_journey_history_can_su.html
| | └─ user_journey_history_cnf1_su.html
| | └─ user_journey_history_cnf_su.html
| | └─ user_journey_history_su.html
| | └─ user_update_mbl_num.html
| | └─ user_update_mbl_num_su.html
| | └─ user_update_password.html
| | └─ user_update_password_su.html
| | └─ welcome_user.html
|
└─ user/

| ├── logout.py
| ├── payment_gateway.py
| ├── payment_gateway_ticket_cancellation.py
| ├── qr_payment.py
| ├── routes.py
| ├── ticket_user.py
| ├── ticket_userdb.py
| ├── user_can_ticket.py
| ├── user_can_ticket1.py
| ├── user_can_ticket2.py
| ├── user_feedback.py
| ├── user_journey_history.py
| ├── user_journey_history1.py
| ├── user_journey_history_can.py
| ├── user_journey_history_can1.py
| ├── user_journey_history_cnf.py
| ├── user_journey_history_cnf1.py
| ├── user_login.py
| ├── user_routes.py
| ├── user_update_mbl_num.py
| ├── user_update_password.py
| ├── utils.py
| ├── __init__.py
|
| ├── __pycache__/
| ├── airline.cpython-312.pyc
| ├── airports.cpython-312.pyc
| ├── flight_name.cpython-312.pyc

```
|  └─ flight_number.cpython-312.pyc
|  └─ usernumber.cpython-312.pyc
|  └─ user_numbers.cpython-312.pyc
|
|  └─ airline.py
|  └─ airports.py
|  └─ Application_Issues_bugs/
|  └─ filelist.txt
|  └─ flight_number.py
|  └─ models.py
|  └─ Project Descriptions.txt
|  └─ run.py
|  └─ versions.txt
|  └─ __init__.py
|
|  └─ .idea/
|  └─ .gitignore
|  └─ .name
|  └─ Airline.iml
|  └─ misc.xml
|  └─ modules.xml
|  └─ workspace.xml
|  └─ inspectionProfiles/
|  └─ profiles_settings.xml
|
└─ .vscode/
    └─ launch.json
```

THE SKA FLIGHT BOOKING SYSTEM VERSIONS

The SKA Flight Booking System has evolved through multiple versions, each improving features, performance, and user experience. This progression reflects ongoing enhancements in technology, security, and functionality to meet growing traveler and airline needs effectively.

THE SKA AIRLINES VERSIONS AND APPLICATION DEVELOPMENTS

SKA Airlines  Application Changelog 

The latest version is always the newest ✨; airline0 is old 📦, and airline1 is newer than airline0 .

For reference, the last version has all the most recent updates .

airline3: Up to the index page, changes are applied there .

airline4: Up to the index page, with updates to ticket_index for guest users   .

The file routes.py manages HTML routes , and common .py files are used throughout the website .

Airline v4 

Data now retrieves from DB tables in the right places , such as routes_html.py inside the needed module, Progressed up to the airport_fare section .

Version 7 

Added plane quota feature  when adding a new plane; tracking capacity is much easier now! 

Version 8 

Implemented new user registration with error flashes  when users try to register duplicates or refresh the page.

Checked pages for flash messages: Whenever you add things like airports or fares, you get an alert  for duplicates, If the endpoint is user_register_success.html and the page is reloaded, you're redirected  to the homepage with an error, preventing duplicate DB entries  .

Version 9 

All guest-related index functions completed, with tests , ticket index, and user feedback included .

If flights for a chosen origin-destination pair don't exist, there's an alert popup .

Fixed journey date logic : now the return date can't be before the journey date .

User-side reservation is paused , while admin-side implementation is underway .

Version 10 🔄

Shared data for HTML pages is now passed centrally through Python routes ⚙️, no more repetitive transfer.

Fares automatically update ⬆️, and all passenger names are shown in uppercase 🔄.

Version 11 🧩

Admin-side booking features auto-assign seat numbers based on class selection 🎫 by Python code—all logic, zero SQL!

All tickets and tables now use a responsive design 📱.

Added user number at reservation time; registered users can re-download their ticket from the portal 🔄.

Version 12 🎯

Refined application responsiveness: focus is now on downloading tickets instead of printing 🖨️ ➡️ 💾.

UI and fare logic now support roundtrips: base fare is doubled ($\text{base_fare} \times 2$) ➡️, and dynamic updates are clearly visible!

Fixed issues with previous double-calculation logic; updates now show live 🔄.

Version 13 👤👛

Admin entering a username now reveals the user number instantly 👁️.

Developed almost all admin features, including a robust PNR enquiry/audit system 🔍 for all bookings.

Admin and user booking histories are linked 📊; also added textbox for country code when adding new airports 🌐.

Version 14 📅

If the journey date is reached, journey status updates to “completed” automatically ✅.

When booking is “In Progress,” the status is set to “IP” instantly ⚡.

Cancelled tickets are clearly flagged 🚩. Event handlers with SQL keep everything up to date.

Plane schedules are available to all users 👥, and entering a plane number instantly reveals the plane name ✈️.

Version 15 🧩

Built PNR index module; user index page is now mobile-friendly 📱.

Plane schedule and PNR index pages are shared between main user and index pages 🔗.

When downloading tickets, each user only sees their own—privacy protected! 🔒.

Version 16 🏠

Fully developed the Bank Gateway 🏦 integrated with User Ticket Booking 🎫 with proper logic and data flow.

Implemented up to user ticket download 📄.

Version 17 ⚠️

Added caution alerts 🚨 for users and admin when entering the user name during booking.

Developed User Journey History tracking 📋 : routes from index -> user_journey_history.py -> user -> user_journey_history_su.html -> user_journey_history1.py -> user_ticket_download_su.html

Used session management to ensure data is accessed only for the particular user 🔑.

Version 18 ❌ 🚫

Developed fully user cancellable tickets with Bank Gateway integration 🏦 🏠.

Reduced separate views for user to download CNF (confirmed) or CAN (canceled) tickets with all journey statuses.

Removed the old ticket download module 🔥 (no need to enter user PNR again).

Users can select a group of CNF, CAN, or both from the list for easy download 📄.

Version 19 🗣️ 📱

Developed User Feedback module 📝 and enhanced user mobile update 📱.

Version 20 🔑 🔄

Developed User Password Update Module 🔒.

Improved some code usability and maintainability 🛠️.

Version 21 ⚠️ 📋

Developed Admin Cancellation Ticket Module 🚫 but pending payment mode handling.

Problem: Admin booked tickets without bank account details cause issues on cancellation (empty account info).

Planned: Added module to require account number at admin booking or cancellation time (mandatory if digital payments involved) 🏦.

Version 22 💰

Completed Admin side cancellation with payment types: Cash and Digital Payment 💵 📱.

Pending: Dynamic linking of account user number and name.

Version 23 📄 💻

Developed full Admin cancel ticket module with automatic fetching of user account number and name in Python code 🤖.

Added JavaScript logic for both user and admin sides to display ticket details dynamically:

For CAN tickets ➡️ show cancellation fare info 🔴

For CNF tickets ➡️ show confirmed details only ✅

Reduced extra code for cancellation ticket handling 🎉.

Version 24 📅 💻

Developed date handling completely in Python code (not HTML forms) 📅 💻.

Reduced ticket download modules by unifying CNF and CAN ticket handling using JavaScript visibility toggles (hide empty cancellation details) 👻 👁️.

Enhanced:

Admin side feedback control with deletion on selection 🗑️

Maintained accurate Airline Fund Details 💰 for SKA Airlines.

Improved code layout and structure for better maintenance 🛠️.

Version 25 🪑

Focused on seat availability based on user/admin seat selection 🚫 🪑.

Created seats_avl.py in admin folder, shared with admin, user, and guest.

Used JSON + MySQL join queries for realistic seat booking and layout visualization 🎨.

Version 26 🔄

Added submodule to check/modify flight search details on the same page 👤 💻 👤 💻 for admin and guest users.

Enables quick rechecking with different plane classes, dates, and fares without page reload 🔄.

Version 27 📅

Developed seat number auto-allocation based on flight_number, flight_class, with seat resets upon reaching max seats 🔄 seats reset.

Version 28 🔍 🪑

Improved seat allocation trigger considering flight_number, flight_class, and journey date (jd).

Example:

Flight: AA12AA, Class: First, JD: 15/Sep/2024 → Seat: 1A

Flight: AA12AA, Class: First, JD: 16/Sep/2024 → Seat: 1A

Flight: AA12AA, Class: First, JD: 15/Sep/2024 (next seat) → Seat: 1B

Allocation dynamically resets per flight, class, and journey date.

Attempted integration: linked SKA Airlines to SKA Bank at admin ticket booking time only.

User-side ticket booking and cancellation fully linked with bank gateway 🏦.

Admin side ticket cancellation done ✅, but booking-side bank account detail input is optional.

Fund increments to SKA Airlines bank account (account 106) handled if provided.

Version 29 💾 🔒

Backed up and inserted all transactions into bank-side database with full control and auditing 🔒.

Issue noted: Cancelling single passenger tickets works, but multiple passenger reservation ticket cancellations fail ❗.

Version 30 🚀 🔥 (Most Important Version!)

Solved all previous issues; implemented correct seat allocation trigger for cancelled seats → seats freed up as available based on flight number, journey date, and flight class 🚶 ✅.

Fixed issue where only single tickets could be cancelled, now multiple ticket cancellations work smoothly 🎉.

Bug fix: replaced cursor.fetchone() with cursor.fetchall() for multiple records handling 🐸 🐛.

Added "Number of Passengers" display in public PNR status 🧑 🤝 🧑.

Fixed user journey history to accurately show confirmed, canceled, and all statuses 📄.

Removed problematic submodule showing user numbers on admin side during booking — avoiding counter ticket confusion 🚫.

Implemented counter ticket cancellation logic: tickets booked by admin must be cancelled only on admin side as counter tickets ⚠️.

Added "All rights reserved" notices on specified pages ©.

✅ PHASE 1: Previously Completed Modules (Theoretical Summary) 🚀 ✈️

1. Multi-Step Ticket Booking System ✈️ 🛫 (Admin & User Side)

Goal: Seamlessly streamline flight ticket booking across multiple pages for a smooth user experience.

ticket_admin1.html ✈️ : Captures initial booking inputs like flight number, origin 🌐, destination 🌐, and dates 📅.

ticket_admin2.html 🛫 : Selects travel class (Economy, Business), quota type 📄 (e.g., Senior, Student), trip type (One-way/Round-trip 🔄), and number of passengers 👤 👤 👤 👤.

ticket_admin3.html 👤 : Collects passenger details — name, age 🎂, gender 🧑, and more.

ticket_admin_final.html 📄 : Displays a comprehensive booking summary for final confirmation ✅.

Note: Sessions 🔄 seamlessly carry data across pages, ensuring no details are lost during the booking flow!

2. Fare Calculation System 💰 📊

Goal: Dynamically calculate ticket fares based on smart business logic.

Base Fare: Standard flight fare 📄

Class Fare: Additional charge based on travel class 📄 (e.g., Economy, Business).

Quota Discounts: Special discounts 🏷️ for categories like seniors 👴, students 🎓.

Trip Discounts: Extra savings for round trips 🔄.

Real-time fare updates powered by JavaScript ⚡ keep passengers informed as they add travelers or change selections — no page reloads needed!

3. Seat Allocation Logic 🪑 ✨

Goal: Automate seat assignment for every booking to maximize efficiency.

Seats stored in airline_sitting table 📄, e.g., First Class seats from 1A to 3D.

Seats are assigned sequentially 🎯, looping or managing overflow smoothly when full.

Candidate for implementation via SQL triggers 📄 or Python logic 🐍 — both approaches explored depending on scalability & flexibility needs.

4. Ticket Cancellation Logic 🔄❌ (User/Admin)

Goal: Enable smooth ticket cancellation, seat freeing, and refund processing.

Cancellation forms available for both admin and users 🧑🔑👤, Upon cancellation: Updates the reservation's status field 🚦 (e.g., canceled). Calculates and processes refund amount refund_Rs 💰. Frees or flags the seat in the airline_sitting table for reuse ♻️.

THE PHASE 2

✅ Phase 1 Recap — Locked & Completed

These are already implemented, tested, and finalized:

✔ Feature	Details
Multi-Step Ticket Booking	Admin + User Booking Flow
Dynamic Fare Calculation (JS)	Class, Quota, Trip Mode based
Round Trip Fare Logic	Fare doubling + Return Date check
Seat Allocation Trigger	CNF/Seat assignment via Trigger
Seat Reuse Logic	Based on Flight No + JD/RD
Ticket Cancellation + Refund	Refund logic based on status/date
PDF Ticket + Cancel PDF	Using reportlab or fpdf
Admin Dashboard (Text)	CRUD, Search, Manage
Passenger Manifest	Based on PNR & Flight filter
Payment Simulation (Manual)	Form input only

PHASE 2 DEVELOPMENT PLAN

Unleashing dynamic, modern features for SKA Airlines! ✨

1. QR Code Integration in Ticket PDFs

Purpose: Fast airport check-in & digital ticket verification—COMPLETED 

`pip install qrcode[pil]` # QR Code generation

Libraries Used: qrcode, base64, reportlab/fpdf

QR Code Data Example:

json

```
{  
  "PNR": "12345",  
  "Passenger": "John Doe",  
  "Flight No": "AI-202",  
  "Seat": "2A",  
  "Date": "2025-07-21"  
}
```

Workflow:

Generate QR Code → Base64 encode → Embed in ticket PDF

Optionally preview QR in HTML before download 

2. Email Notification System

Purpose: Instantly update users via email on bookings and cancellations  

When to Send:

On Booking  (attach PDF ticket)

On Cancellation  (with refund details or summary)

<https://myaccount.google.com/apppasswords> to Generate the app password to use in the email system

Libraries & Install Commands:

text

pip install Flask flask-mail reportlab

Email Package: Flask-Mail (SMTP-based)

Session Management:

text

pip install Flask-Session

python

```
from flask_session import Session
```

```
app.config['SESSION_TYPE'] = 'filesystem' # Or use 'redis', 'sqlalchemy'
```

```
Session(app)
```

To Do:

Configure mail server and authentication 🔑

Trigger email on ticket booking/cancellation events 📧

3. 🏠 QR Code Payment Gateway (Simulated)

Purpose: Simulate secure payments with QR before final booking 💰

Sample QR Payment Data:

json

```
{  
  "Amount": 5600,  
  "Reference": "PNR-12345",  
  "Payee": "SKA Airlines",  
  "Timestamp": "2025-07-08 11:23:12"  
}
```

Steps:

Show payment QR on page

Simulate "Payment Done" → Confirm booking

4. 📊 Graphical Admin Dashboard

Purpose: Bring insights to life with eye-catching visuals!

Key Charts:

Bookings per Day 📅

Revenue per Flight/Class 💰

Cancellation vs Confirmed Trends 📈

Libraries:

Frontend: Chart.js, Plotly.js

Backend: Flask serves JSON endpoints

To Do:

Dashboard HTML

AJAX endpoints for live data

Note: Reservation funds visualization already implemented ✅

5. ✨ Advanced Seat Allocation by Flight Type

Purpose: Automatically manage seat allocation by aircraft type 🚦

Features:

Flight types (Boeing, Airbus) + seat range config for FC, BC, PECO, ECO

Auto-assign seats according to chosen flight type ✈️

Implementation:

Add flight_type field in flights table

On selecting flight, auto-populate seat/class limits (JS or backend)

To Do:

Module: flight_type_seats.py

Embed logic in JS/frontend or backend

6. 🛠️ Common DB Configuration File

Purpose: Centralized & secure database connection 🗝️

File: db_config.py

python

```
import mysql.connector
```

```
def get_connection():
```

```
    return mysql.connector.connect(
```

```
        host='localhost',
```

```
        user='root',
```

```
        password='tiger',
```

```
        database='airline'
```

```
    )
```

Usage:

python

```
from db_config import get_connection
```

Status: DONE ✅

7. 💻 Dynamic Fare Calculation (From DB)

Purpose: No more hardcoded fares—fetch dynamically from DB 📁

Current: Static fare in HTML—OUTDATED 🚫

New: Calculate on backend from class, quota, base fare, origin, destination, distance

To Do:

Revise DB schema if needed

Implement logic to fetch accurate fare on every booking

Status: Successfully implemented! 🎉

8. 🗨️ Admin Fare Update & Visualization

Purpose: Equip admin with powerful fare editing & analysis tools! 🛠️

Features:

Form-based UI: edit fares for class/km/origin-destination

Optional: auto-calculate fare increases by percentage

Action Items:

Fetch/update from fare_config table

Build responsive admin HTML page

9. 🌐 Dynamic IP Address / Domain Binding

Purpose: Make SKA Airlines accessible by domain name instead of raw IP 🌐

Outcome: Users can reach the app at <https://ska-airlines.com> vs. <http://xxx.xxx.xxx.xxx>

Status: DONE ✅

10. 🕒 Enhanced Time Logic (Optional Future Step)

Purpose: Add robust Departure/Arrival Time management in booking system

What's Next?

Integrate time pickers in booking UI

Use backend logic to ensure valid, timezone-aware times

Installations Required

`pip install qrcode[pil]` # QR Code generation

`pip install Flask flask-mail` # Email sending

`pip install Flask-Session` # Session management

`pip install reportlab` # PDF generation

`pip install mysql-connector-python`

`pip install mysql-connector-python` # MySQL database connector

🔥 VERSION 31

Implemented fully importable QR code generation on tickets 🚀 ✨

Developed seamless ticket layouts integrating dynamic QR codes for fast scanning and verification.

Made the code modular and reusable wherever needed. 🧩 ✅

🚀 VERSION 32

Centralized Database Access Created 🗄️ 🔑

Developed a unified DB connection module for consistent and easy import across all Python programs.

Simplified database handling and improved maintainability. 💻 🛠️

💳 VERSION 33

Introduced Advanced Payment via QR Code on Payment Gateway ⚡ 🌐

Integrated complex payment workflow involving three bank-side JSP programs:

uselqr.jsp, qr_codeaccept.jsp, authorize_payment.jsp

These communicate with our Flask backend via IP and port. 🌐 🖱️

Created new transaction table under airlines DB to capture txn_id and validate payment status dynamically.



Pending: Full ACID-compliant payment method processing under payment modules. ➡️ SOON 🛠️

💳 VERSION 34

Dynamic Fare Calculation from DB Implemented 🇮🇹 💰

Replaced static HTML fares with dynamic backend logic fetching from airline_class_quota_fares table.

Fare calculation now smartly considers:

Origin & Destination ✈️

Travel Class & Quota 🚗

Distance 🗺️

Flight base fare adjusted by flight priority for speciality flights 🚀

Timestamp auto-update added for auditing in airlines_users:

```
sql
```

```
ALTER TABLE airlines_users
```

```
MODIFY COLUMN DOR TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP;
```

VERSION 35

User-Side Email System Launched for Ticket Booking & Cancellation  

Sent professional, grammatically polished emails for both confirmed (CNF) and cancelled (CAN) bookings.

Corrected ticket cancellation info wording on printed tickets for clarity and professionalism.  

VERSION 36

Enhanced Email Logic to Avoid Duplicate Sends on Refresh  

Prevented repeated emails on accidental browser refresh during booking or cancellation.

Fixed unknown error page on user ticket cancellation related to refunds.

Positioned email dispatch before ticket_su and ticket_can_su methods in ticket_userdb.py and user_can_ticket2.py.

Admin Side Improvements:

Added extra email field in admin booking for capturing user emails. 

Fetches & uses user email during admin cancellations for realistic communication. 

Overall: Nearly all application issues resolved; future updates will focus on minor fixes and optimizations.



VERSION 37

Fixed User Ticket Journey History Display on Cancellations  

Ensured cancelled tickets properly reflect in user journey history with clear status messages. 

Skipping detailed notes on Version 37 to proceed to next significant update.

VERSION 38

Highly Dynamic Seat Allocation Based on Flight Number, Class, Journey Date & Flight Type  

Introduced flight types (e.g., Boeing, Airbus) selectable by admin during flight creation, saved in flights table.  

Advanced seat allocation logic extended to respect flight type-specific seat layouts.

Frontend restricts seat selection flow based on chosen flight type for accurate bookings. 

Modified airline_sitting table: added flight_type column for seat trigger logic consistency. 

Print & Display:

Tickets now show Flight Type info along with flight charges for transparency.  

Seat Availability Logic Improved:

seats_avl.py updated for robust seat availability checks — correctly handling switched origins and destinations.  

Example:

Flight AA12AA, Class: First, Total Seats: 12

Origin: Hyderabad, Destination: New York, Date: 01/Aug/2025

Booking status now accurately reflects seat availability—even if origin/destination are swapped.

Technical Enhancements:

Refined MySQL query logic in seat availability module for precision and error-free operation.  

Entire codebase polished and structured for maximum robustness and dynamic behavior.  

This version represents a major milestone with zero errors, seamless integrations, and enhanced user/admin experience!  

Version 39 & 40 Updates

Journey Date is Today—No Cancellation Allowed

Tickets with a journey date matching today's date will not be displayed for cancellation and cannot be cancelled.

Note:

To view, print, or download your ticket, please use My Journey History where you'll find and print or view your confirmed tickets.

To View Your Confirmed Tickets:

 /  View/Print Confirmed Tickets

Dynamic Seat Availability

Seats are now dynamically checked and displayed in real-time from the database for accuracy during booking and management.

Enhanced Admin Functionalities

Admins can now view and manage flight seats layouts dynamically based on flight type for future flights.

 Admin access lets you log into the backend database to change settings like flight seats, airports, class fares, flight fares, and more—admin-only features for enhanced control!

Version 41 Update

Critical Security Fix at Payment Gateway

Previously, the payment gateway QR code (UPI/Pay QR) was generated on the frontend (HTML/JS).

This allowed anyone to open Inspect Element → Edit QR value → Pay to their own account — breaking payment integrity.  

Fix Implemented:

Now, sessions securely store important payment data like total_fare, txn_id, transaction mode on the backend.

The QR code is generated server-side with these session values, so it cannot be tampered with via frontend inspection.

Secure Payment Flow:

When the user reaches the payment page, backend saves total_fare, txn_id, and trm in session/DB.

Frontend requests /generate_payment_qr with no sensitive parameters.

Server builds and returns a secure QR as an image.

Even if a user edits HTML, the QR is always generated from backend session data, not from the frontend input.

Benefits:

Users cannot modify QR code via Inspect Element.

Payment integrity is fully protected.

Your payment status polling (check_payment_status) continues to work smoothly with secure txn_id.

Version 42

Version 42 Updates

Admin Side Validation:

Implemented and fixed issues where, while adding a new flight, the system now alerts the admin to check and fill in all required flight information. ⚠️ ✈️

Dynamic Year Display:

Added automatic current year display for All Rights Reserved © [Year] on specific key pages:

HTML Index

User Login

Admin Login

Admin Dashboard

User Dashboard

Register Module

✂️ The year is dynamically generated by the system without any manual update needed.

Flight Types from Database:

Major enhancement — flight types like Boeing 777, 747, Airbus A320, Domestic, etc., are now retrieved dynamically from the flight_types database table instead of being hardcoded in HTML.

This improves the admin page experience when adding new flights and cleans up frontend code significantly.

🕒 Added time management for flights operating between origin and destination, with fly time tracked in hours.

🔧 Debugged the code and introduced a new flight_legs table to store departure, arrival timings, and fly duration.

⚡ Included 3 automatic triggers in flight_legs to handle operations like maintaining leg_sequence and calculating fly_time.

📅 Updated airline_reservation table with new columns to hold arrival_date, arrival_time, and fly_time.

🔗 Modified the necessary code for full dynamic behavior, checked issues, and corrected them.

📁 Added a new program guest/flight_date_hours.py that automatically adjusts the arrival date if a flight departs late (e.g., departs at 20:00 and arrives next day at 10:00).

🚀 Streamlined the flow across all modules so the entire application process runs smoothly without issues.

Version 44 Highlights

Admin Flight Addition & Airport Pair Splitting:

When an admin adds a new flight to the airline DB, selected airports are now automatically split into pairs and inserted into the `airplane_fare` and `flight_legs` tables.

➡ Robust airport pair splitting and duplicate prevention for both tables.

Manual Fare and Distance Entry:

After flight addition, admins log in to the MySQL backend to manually check the tables and enter KM distance.

💰 Fare is auto-calculated with a trigger as $KM \times 12$ per route!

Accurate Departure & Arrival Times:

Fixed missing departure/arrival time entries and reflections.

🌙 Arrival date now updates correctly for overnight flights—improved accuracy!

Controlled Database Access:

Limited access granted to `airline_admins` for DB table control, allowing official fare adjustments and admin actions.

SQL Query Safeguards:

In admin `new_flight.py`, added logic to avoid duplicate inserts into the `airplane_fare` table—existing airport pairs are skipped.

Admin Instructions & Change Management:

Provided updated instructions to guide admins through new processes and project changes.

Flight Timings Submodule:

Flight search now displays precise timings at origin (Departure) and destination (Arrival), with correct date handling if the flight crosses into the next day.

Compared to previous versions, this release is more dynamic, efficient, and powerful! ⚡ 🔥

Overall, this version is more enhanced and advanced! 🎉 ✨

Version 45

In this version, all mistakes have been corrected, including replacing terms like Airline Boarding Pass with Flight E-Ticket across all modules   

-----THE PHASE 3-----

Concentrated on the UI Fixes entire application to meet the Mobile and proper layout

Version 46

In this version, the Flight Boarding Pass Generator module has been developed for the admin side  .

The admin logs in at the airport, and the concerned admin can generate a boarding pass using the PNR number for today's and upcoming journeys only.

Fixed issues where flight types were displaying incorrectly on the admin side during ticket booking.

Fixed the issue where flights with the origin Hyderabad and destination New Delhi were showing incorrect timings and flight duration  .

SKA Airlines — Phase 3

Under Phase 3 of the SKA Airlines Project, there have been huge UI-related developments to improve the UX (User Experience) and add dynamic content, while maintaining our existing project logic.

This phase focuses on using:

✅ Bootstrap + Bootstrap Icons + Aviation.css (stored in static/css)

to build a modern UI that is fully compatible with all devices such as PCs, laptops, and mobile devices



🚀 Version 47

In this version, huge UI development has been introduced 🎨 ✨.

🌐 Index Side

Developed a huge UI upgrade to make the webpage look more dynamic and attractive using Bootstrap and icons ✨ 🎯.

Modified the instructions file and added more important information 📄 ✅.

🔧 Admin Side

Improved and corrected the admin UI to make it more dynamic and responsive, compatible with all screen resolutions (PC, Laptop, Mobile) 🖥️ 📱 📄.

Developed a dynamic UI for the Admin Boarding Pass Generation module with exact content for printing or downloading 🖨️ 📄 📄.

Developed logic to restrict admins from cancelling confirmed tickets if the departure time is within 30 minutes of the flight time ⌚ 🚫 🛔.

Admins can now generate a boarding pass only if the departure time is at least 30 minutes before the flight   .

Fixed some known issues while improving the UI   .

User Side

Huge UI upgradation has been done on the user side as well   .

Modified all user pages to look more dynamic and modern   .

Developed a query to prevent users from cancelling tickets unless it is at least 8 hours before departure    .

Developed a Web Check-in Boarding Pass Generator module   :

Tickets are displayed in the user dashboard in webchecking_boardingpass.html

Boarding pass generation is allowed only if the departure time is between:

 48 hours before departure and

 30 minutes before flight departure  

Added important user instruction points in the Web Check-in Boarding Pass program   .

Fixed some known issues during dynamic UI development   .

Version 48

In this version, flight booking restrictions have been implemented ✈️🕒.

👤 Index & User Side

Flight bookings are allowed only up to 3 hours before the scheduled flight departure time 🕒🚫.

📅 Return Date Rule

The return flight departure date is always greater than the booking date — no issues found ✅📅.

🔧 Admin Side

Flight bookings are allowed only up to 1 hour before the scheduled flight departure time 🕒🚫.

🔧 Fixes & Improvements

Fixed known issues in the application:

Admin boarding pass generation time limit changed to 20 minutes before departure 🕒🕒.

User-side boarding pass generation time limit increased to 30 minutes before departure 🕒✈️.

✅ Overall, all known issues in the application have been fixed and improved.

🚀 Version 49 – Release Notes

Seat Availability Fix:

Fixed seats_avl.py to accurately display real-time seat availability on the booking page.

This is a major and impactful improvement ensuring correct seat allocation during booking.

Payment System Migration:

Migrated the Bank database from Oracle to MySQL for all payment gateway operations, including ticket booking and cancellation.

Bank Module Integration:

Created a dedicated bank module within the project to handle banking user and admin ticket booking/cancellation processes seamlessly.

JSP to Flask Migration:

Converted all bank-side .jsp files to Flask (.py) with MySQL database queries for improved performance, maintainability, and system consistency.

Dynamic QR Payment View:

Updated bank-side HTML pages to support dynamic transaction views, including secure QR-based payment processing.

Email Enhancements:

Enhanced email notifications by adding relevant information regarding boarding passes and ticket cancellations.

 Overall, all known issues in the application have been fixed and improved.

Important Credentials DB Mysql

Root tiger

SKA Projects ska_Projetcts

airline_admin ska_airline_admin

Version 50 – Release Notes

QR Transaction Security Enhancement:

Restricted the bank-side QR transaction flow to prevent duplicate payments.

If a transaction with the same txn_id is already completed, the system automatically redirects the user to the home page after payment.

Any further attempt to process the same transaction is blocked to prevent duplicate ticket booking and bank debits.

Dynamic Ticket Layout Improvements:

Updated ticket layouts to support dynamic views for both Admin and Guest users, ensuring a more structured and role-based display.

Bug Fixes & Stability Improvements:

Resolved several known and minor issues across the project to improve overall performance and system stability.

Version 51 – Release Notes

Common Ticket Template Implementation:

Introduced a unified base_ticket.html template for both Admin and User sides, covering:

Ticket download

Ticket history

Admin PNR download

Admin audit view

Additionally, created base_ticket2.html for handling ticket layouts specifically during the booking stage.

These common templates now serve all ticket style layouts across the project, ensuring consistency and maintainability.



Boarding Pass Layout Fix:

Fixed boarding pass generation for both Admin and User sides, ensuring proper layout formatting during printing and download operations.



Bug Fixes & Stability Improvements:

Resolved several known issues to enhance overall system stability and performance.



Version 52 – Release Notes



Duplicate Airport Selection Fix (Admin):

Fixed an issue in the Add New Flight module where the Airport field could accept duplicate entries during insertion. For example, if an admin selected airports like HYD, NLD, NY, MBI, HYD, the system previously allowed HYD twice. This has now been corrected so that each airport can appear only once in the route sequence.



Automatic Fare Calculation Trigger:

Updated the flight fare triggers so that the ticket fare is automatically calculated when the distance (KM) is entered. A new database trigger has been introduced to handle this process seamlessly.



Duplicate Entry Restriction – airplane_fare Table:

Implemented restrictions to prevent duplicate records in the airplane_fare table. The system now blocks duplicate entries whether the airport order is ordered or reversed.

Duplicate Prevention – flight_legs Table:

Added constraints to ensure that duplicate route legs cannot be inserted within the same flight number, improving route data integrity.

Admin Program Update:

All the above improvements and validations were implemented in the new_flight.py module on the Admin side.

New Banking Module – User Bank Account Creation:

Introduced a new feature in the payment gateway system allowing unregistered users to request a bank account. Users can now submit an account request, which can be approved by the SKA Bank cashier or bank administrator.

New Database Table:

Created a new table userp in the Bank database to store and manage these bank account requests.

General Fixes:

Resolved several known issues to improve overall system stability and reliability.

Version 53

 Updated flight numbers to align with airline names, and added support for new aircraft types: Airbus A380 and Boeing 737.

 Developed backend logic in new_flight.py to automatically generate unique flight numbers based on the selected airline.

 Modified affected database tables (flights and flight_legs) to match the updated flight number system.

⚙️ Upgraded the seat allocation trigger to dynamically assign seats based on the maximum available class configuration for each flight type.

🛋️ Seat Configuration (Max Capacity per Flight)

First Class: 16

Business Class: 100

Premium Economy: 150

Economy: 550

🌐 Added new international routes including:

Japan, Singapore, Qatar, South Korea, France, Spain, Italy, Canada, China, Russia, New Zealand, Australia, Thailand, Malaysia, Denmark, Netherlands, Germany, Finland, Sweden, Norway, Belgium.

🇮🇹 Inserted all corresponding routes with fares and timings into the flight_legs table.

-----THE PHASE 4-----

Version 54 🚀

1. Flight Search and Booking Enhancements

Flight search has been improved to support both country codes and country names during ticket booking.

Implemented modules:

Admin side:

Ticket booking

New flight creation

Fare management

User side:

Flight ticket booking completed

Guest user side:

Flight search implemented

Ticket booking enabled

Flight schedule added with search by flight name

2. Seat Allocation System (Database-Driven Implementation)

The seat allocation logic has been redesigned to use a database-driven approach instead of hardcoded values.

A new table seat_master has been created to store all possible seat numbers, covering the maximum seating capacity of aircraft.

Before (Old Approach)

Iterated through up to 600 values 🤔

Used string parsing

Heavy cursor usage

Difficult to maintain

Now (Improved Approach)

Uses a single SELECT query ⚡

Leverages indexed tables

Easier to maintain and extend

More suitable for production environments

Indexes Added

```
CREATE INDEX idx_seat_master_class ON seat_master(class_type, row_no, seat_col);CREATE INDEX  
idx_sitting_lookup ON airline_sitting(seat_number, seat_status);
```

Final System Behavior

The updated flow works as follows:

User books a ticket

Trigger is executed automatically

The system checks:

Flight class

Flight number

Journey date

The first available seat is selected from seat_master

Seat is assigned automatically

Key Improvements

No loops

No string manipulation

No hardcoded seat lists

THE CORRECTED AND ADVANCED allocate_seat_number trigger with no hardcoded seat numbers directly from the db table seat_master

```
DROP TRIGGER IF EXISTS allocate_seat_number;
```

```
DELIMITER $$
```

```
CREATE TRIGGER allocate_seat_number
```

```
BEFORE INSERT ON airline_sitting
```

```
FOR EACH ROW
```

```
BEGIN
```

```
--  Variables
```

```
DECLARE class_type VARCHAR(100);
```

```
DECLARE journey_date VARCHAR(100);  
DECLARE flight_num VARCHAR(11);  
DECLARE candidate_seat VARCHAR(10);
```

```
--  Get reservation details
```

```
SELECT flight_class, jd, flight_Number  
INTO class_type, journey_date, flight_num  
FROM airline_reservation  
WHERE PNR = NEW.PNR;
```

```
/* =====
```

```
 SINGLE STEP: Pick NEXT AVAILABLE SEAT IN ORDER
```

```
(handles both cancelled + fresh seats automatically)
```

```
===== */
```

```
SELECT sm.seat_number  
INTO candidate_seat  
FROM seat_master sm  
WHERE sm.class_type = class_type
```

```
--  exclude already occupied seats
```

```
AND NOT EXISTS (  
    SELECT 1  
    FROM airline_sitting s  
    JOIN airline_reservation r ON s.PNR = r.PNR  
    WHERE r.flight_class = class_type  
        AND r.jd = journey_date  
        AND r.flight_Number = flight_num  
        AND s.seat_number = sm.seat_number  
        AND s.seat_status IN ('ACTIVE','REUSED')
```

)

-- ❌ avoid duplicates within same PNR

AND sm.seat_number NOT IN (

SELECT seat_number

FROM airline_sitting

WHERE PNR = NEW.PNR

)

ORDER BY sm.row_no, sm.seat_col

LIMIT 1;

-- ✅ assign seat

IF candidate_seat IS NOT NULL THEN

SET NEW.seat_number = candidate_seat;

SET NEW.seat_status = 'ACTIVE';

END IF;

END\$\$

DELIMITER ;

A clean, scalable, and production-ready seat allocation system.

In this version, duplicate ticket bookings have been successfully restricted for both Admins and Users. ✅

After booking a ticket successfully, if the Admin or User clicks the browser back button, the system will no longer allow duplicate ticket booking again. Instead, it will display the following message:

> “Ticket is already booked. Please book a new ticket from the beginning.” ⚠️

Also, payment gateways have been restricted to prevent duplicate debit or credit transactions if the ticket has already been booked successfully. 💳🚫

This fix is mainly implemented to prevent duplicate bookings in the system and improve booking security and reliability. 🔒💡

For this implementation, the following files were developed:

* `booking\_guard.py`

* `booking\_back\_guard.html`

Added the following code in the main `airline.py` application to redirect Admins and Users properly to their respective dashboards and to avoid direct access to booking pages without login authentication. 👤🛩️

```
```python
```

```
Added the booking flow guard for Admin, User, and Guest
```

```
in the 404 error page to avoid direct access of booking pages without login
```

```
@app.errorhandler(404)
```

```
def handle_404(_error):
```

```
 path = (request.path or "").lower()
```



```
if path.startswith('/admin'):
 return render_template(
 'common/booking_back_guard.html',
 actor_label='Admin',
 home_url=url_for('home'),
 booking_url=url_for('admin_dashboard_bp.ticket_admin'),
 dashboard_url=url_for('admin_login_bp.welcome_admin')
), 404
```

```
if path.startswith('/user'):
 return render_template(
 'common/booking_back_guard.html',
 actor_label='User',
 home_url=url_for('home'),
 booking_url=url_for('user_dashboard_bp.ticket_user'),
 dashboard_url=url_for('user_login_bp.welcome_user')
), 404
```

```
return render_template(
 'common/booking_back_guard.html',
 actor_label='Guest',
 home_url=url_for('home'),
 booking_url=url_for('guest_dashboard_bp.ticket_index'),
 dashboard_url=None
) , 404
```

...

In this version, 2 major operational modules were developed on the admin side to help monitor, audit, and analyze flight bookings based on flight number and journey date. 📊 ✈️

♦ Module 1 — Flight Passenger Manifest & Audit Report

Developed an admin module that displays all flight reservation tickets based on the selected flight and journey date.

Admins can directly audit passenger booking details, and each selected PNR is linked to the Admin PNR Enquiry system for detailed verification and investigation. 🗂️ 🔍

♦ Module 2 — Flight Booking Summary Dashboard

Developed an operational dashboard module that displays total flight bookings for the selected journey date.

The dashboard provides flight-wise booking summaries, passenger load details, revenue insights, and route information. From this module, admins can directly navigate to the Flight Passenger Manifest & Audit Report for detailed passenger-level auditing. 📊 ✈️ 👮

Version 57 ✈️

In this version, fixed the layout issues at the admin side, including ticket\_admin\_download\_su and pnr\_enq\_admin\_su.html pages. 🛠️

Also fixed the issue with ticket bookings occurring at the same time on the user side, and updated the HTML templates to make them more compact and responsive for mobile view 📱 ✨

Additionally, added type="number" for the passenger age field on both the user side and admin side while booking tickets, improving input validation and user experience 📅 ✅

## Version 58 ✈️

In this version, fixed the known issues and improved overall system stability 🛠️✅

Introduced a new feature during ticket booking for Admin, User, and Guest modules. In the flight search results, users and admins can now view the seat availability for the next 6 days 📅📱

Also added a new flight type: **Private Jet** ✈️💎 for luxury and VIP travel bookings.

## ✈️ Version 59

Adjusted and improved most ticket layout issues for mobile resolution devices 📱

Fixed known issues in the application 🛠️

## ✈️ Version 60

Fixed the main issue where the JSTATUS was not automatically updating to COMPLETED after the journey arrival date and time had passed 🕒

Example: If the arrival time was 13:10, 18/May/2026, the status will now automatically update without requiring a MySQL scheduler.

This enhancement mainly helps while using the SQLite database 💾

Modified the program logic to ensure the status updates correctly based on arrival date and time logic in journey\_status\_updater.py ⚙️

Fixed additional known issues in the application 🛠️

## ✈️ Version 61

✈️ In this version, the class selection dropdown has been removed once the user selects a flight number, making the booking flow smoother and more user-friendly.

🇮🇳 Improved the flight selection UI — when a user/admin selects a flight, the system now instantly displays the available seats, and the Book button is integrated directly inside the flight selection section for faster booking access.

📱🔒 Resolved a major QR payment security issue on the user side where mobile payment pages were opening directly when the URL was pasted into a mobile browser. This issue has now been fixed by implementing enhanced validation logic and adding extra columns in the qr\_payment\_status table for better payment session tracking and protection.

🔒 Developed secure password hashing for both the admin users and bank-side operations. Passwords are now stored in hashed format to improve authentication security and protect sensitive account data.

📄 Modified the FIRST CLASS fare to ₹6500.

Overall, this is an advanced and more secure version of the airline booking system. 🚀🌟

If any major issues are found, revert back to **\*\*Version 54-59\*\*** for stability.

☀️ Version 62 – SKA Airlines Integrated Wallet & Flight Operations System Update ✈️👛

🏠 Integrated Wallet System (Userreal-based)

Developed an integrated wallet system using the userreal table, linked with the airline\_user number, for the user side. The MASTER ACID is 106.

For the Admin Side, the following modules were developed:

+ Add Money

- Deduct Money
- 📄 View Transactions
- 💰 Refund History
- 📊 View Balance
- 💳 User Wallet Transactions
- 🎁 Wallet Offers (common for Admin / User / Guest)
- 👤 Admin User Management System

Developed an admin control module to manage users effectively:

- 🚫 Admin can block user accounts
- ⚠️ If a blocked user tries to log in, the system displays:  
“Please visit the SKA Airlines office until further admin action.”
- 🔒 This is handled via the airline\_users table using user\_status values like ACTIVE / BLOCKED

Additionally:

- 🔍 Admin can view blocked users history
- 📄 Searchable bar implemented for easy filtering
- ✈️ Flight Operational Management Module

Developed a complete flight operations control system:

- ✈️ Cancel flights
- ⌚ Delay flights
- Set ON\_TIME status

These updates reflect in the flight table, including:

flight\_status

cancelled\_from

cancelled\_to

cancellation\_remarks

⚙️ Automatic Flight Status Updater (flight\_status.py)

🔄 Automatically updates flights

When the cancelled\_to date is completed, the system automatically changes status back to ON\_TIME

👁️ Flight Status Visibility (Common Module)

Displays flight operational status (Cancelled / Delayed / On Time)

Visible to Guest / User / Admin

🧹 QR Payment Cleanup Module (qr\_payment\_cleanup.py)

🗑️ Automatically deletes unwanted or expired records

Cleans up the qr\_payment\_status table regularly

👤 User Side Wallet Modules

Developed user-side wallet features:

+ Add Money

💰 View Balance

📋 Transaction History

💎 Refund History

🎁 Wallet Offers

🚫 Flight Booking Restrictions Logic

Cancelled flights are automatically restricted based on dates

✗ Such flights will NOT be visible for booking (User / Admin / Guest)

⌚ They reappear only after cancellation period ends or admin reactivates ON\_TIME status

🌐 Guest User Module

✈ Displays flight operational status (Cancelled / Delayed / On Time)

🚫 Prevents booking of restricted flights

📄 Separate module for viewing cancelled flights

🔍 Includes searchable bar for easy access

🚀 Overall Summary

This version is a highly powerful and stable upgrade 💪 ✨ with:

Strong wallet integration 💳

Advanced admin controls 👮

Automated flight status engine 🚂

Clean payment maintenance system 🛠

Improved guest visibility & booking safety ✈

# 🚀 Version 65 (SQLite) | Version 63 (MySQL) – Airport Name Management & Booking Logic Enhancements ✈ 🚂

In this version, introduced a new **Airport Name Display Management System** to improve how airport names are displayed throughout the application for Admin, User, and Guest operations. 🌐 ✈

To support this enhancement, a new column named **real\_airport\_name** was added to the **airports** table. 📄

This column stores the actual airport name and is used for displaying user-friendly airport information during:

\* 🇺🇸 Ticket Booking

- \*  User Operations
- \*  Guest User Operations
- \*  Admin Management Modules

---

## ## Airport Management Enhancements

Developed a new sub-module on the **\*\*Admin Side\*\*** that allows administrators to:

- \*  Add new airports
- \*  Enter the actual airport name
- \*  Automatically save airport information into the `airports` table
- \*  Maintain consistent airport name display throughout the system

---

## ## Booking, Ticket & Boarding Pass Logic Fixes

Implemented and fixed logic related to:

- \*  User-side ticket booking process
- \*  Ticket downloading functionality
- \*  Admin-side ticket management
- \*  Boarding pass generation
- \*  Ticket cancellation windows
- \*  Boarding pass cancellation workflows



These improvements ensure that airport names are displayed accurately and consistently across all booking and travel-related operations. ✅

---

## ## ⚙️ New Business Logic Module – `airport\_map.py`

Introduced a new module:

📄 `**`airport_map.py`**`

inside the `**routes**` folder.

This module centralizes airport mapping and business-rule validation logic, making the system:

- \* 🧠 Easier to maintain
- \* 🔄 More consistent across modules
- \* ⚡ More scalable for future airport additions
- \* ✅ Fully compatible with application business rules

---

## ## 🌟 Key Improvements in Version 65

- \* ✈️ Introduced Airport Name Display Management System
- \* 🏠 Added `real\_airport\_name` column to the `airports` table
- \* ➕ Developed Admin Airport Creation Sub-Module
- \* 🛒 Fixed ticket booking and downloading logic

- \* 🇺🇸 Improved boarding pass and cancellation workflows
- \* ⚙️ Added centralized `airport\_map.py` business logic module
- \* 🌐 Enhanced airport name consistency across Admin, User, and Guest interfaces
- \* 🚀 Improved maintainability and scalability of airport-related operations

# ✈️ Version 66 (SQLite) | Version 64 (MySQL)

## ## 🚀 Flights Database Management System Enhancement

In this version, the Flights DB module has been significantly enhanced. Previously, administrators relied on direct local database access to maintain flight-related master data. Since direct database maintenance is not practical after online deployment, a dedicated **\*\*Admin Flight Database Management Module\*\*** has now been developed.

The new module allows administrators to view, edit, update, and manage important flight database records directly through the web application.

### # ✈️ Flights DB Main Menu

The Flights DB section now contains the following modules:

Module	Purpose	Editable Fields	Status
1 Manage Flight Type Seats	Manage aircraft seating capacities	seat_count	Completed

	2	Flight Class & Quota Fares		Manage fare amounts		class_fare, quota_fare	
✓		Completed					
	3	Manage Airplane Fare & KM		Maintain flight distance		km only	
✓		Completed					
	4	View Airports		Airport master data		View Only	
✓		Completed					
	5	View Registered Flights & Fare		Flight registration details		base_fare	
✓		Completed					
	6	Manage Flight Timings		Flight schedules		departure_time, arrival_time, arrival_day_offset	
✓		Completed					
	7	Add New Flight Type		Register new aircraft models		New Records	
✓		Completed					

---

## # 🎨 Common UI Framework Introduced

To maintain a consistent user experience across all Flights DB modules, reusable components have been developed.

### ### Reusable Files

**\*\*CSS\*\***

\* `static/css/flights\_db.css`

**\*\*JavaScript\*\***

\* `static/js/flights\_db.js`

**\*\*Base Template\*\***

\* `flights\_db\_base.html`

All Flights DB pages now utilize these common reusable resources.

---

## # 🔍 Common Features Added to All Modules

### ### Header Information Card

Each module now includes:

- \* Bootstrap Icon
- \* Module Title
- \* Module Description
- \* Professional Card Layout

### ### Search Facility

A reusable search component has been implemented across all management pages, allowing administrators to quickly locate records.

### ### Update Controls

All editable modules now provide a standardized:

- ✅ Update Changes button

for saving modifications.

#### ### Common Footer

All Flights DB pages now include a reusable footer displaying:

© SKA Airlines {{ current\_year }} All Rights Reserved.

---

### # 🚗 Module 1 – Manage Flight Type Seats

#### ### Purpose

Maintain aircraft seating capacities used during ticket booking and seat allocation.

#### ### Editable Field

\* Seat Count

#### ### Read-Only Fields

\* Flight Type ID

\* Aircraft Name

---

### # 💰 Module 2 – Flight Class & Quota Fares

### ### Purpose

Manage fare amounts associated with travel classes and passenger quotas.

### ### Editable Field

- \* Fare Amount

Supported Tables:

- \* Class Fare

- \* Quota Fare

---

#  Module 3 – Manage Airplane Fare & KM

### ### Purpose

Maintain route distance information.

### ### Editable Field

- \* KM

### ### Automated Fare Calculation

Fare values are automatically calculated using the company pricing formula:

**\*\*Fare = KM × 12\*\***

Direct fare editing has been restricted to maintain pricing consistency.

---

## # Module 4 – View Airports

### ### Purpose

Display airport master data in a centralized view.

### ### Information Displayed

- \* Airport Code
- \* Airport Name
- \* Real Airport Name
- \* City
- \* Country

### ### Future Enhancements

- \* Export to PDF
- \* Export to Excel

---

## # Module 5 – View Registered Flights & Fare

### ### Purpose

Maintain registered flight fare configurations.

### ### Displayed Information

- \* Flight Number

- \* Origin

- \* Destination

- \* Aircraft

- \* Base Fare

### ### Editable Field

- \* Base Fare

---

## # 🕒 Module 6 – Manage Flight Timings

### ### Purpose

Maintain flight schedules and automatically calculate journey duration.

### ### Editable Fields

- \* Departure Time

- \* Arrival Time

- \* Arrival Day Offset



### ### Automatic Duration Calculation

Example:

Departure Time: 22:00

Arrival Time: 01:30

Arrival Day Offset: 1

Result:

3 Hours 30 Minutes

The calculated duration is automatically stored within the database.

---

## # Module 7 – Add New Flight Type

### ### Purpose

Register new aircraft models that can be assigned to flights and schedules.

### ### Fields

\* Aircraft Code

\* Aircraft Name

- \* Seat Count
- \* Manufacturer
- \* Description

### ### Duplicate Protection

The system now validates duplicate aircraft entries before insertion.

If a duplicate flight type exists, the system displays:

⚠ Flight Type Already Exists

and prevents duplicate record creation.

---

### # 🛡 Data Integrity Improvements

#### ### POST → REDIRECT → GET Pattern Implemented

All update operations now utilize the Post-Redirect-Get pattern to prevent accidental form resubmission during browser refresh.

Benefits:

- \* Prevents duplicate updates
- \* Prevents duplicate insertions
- \* Improves application reliability

### ### Unique Constraint Recommendations

The following fields are protected against duplication:

- \* Aircraft Code
- \* Flight Number
- \* Airport Code

---

#  Summary

Version 66 introduces a complete web-based Flights Database Management System that replaces dependency on direct database maintenance tools. Administrators can now safely manage aircraft, fares, airports, flight schedules, and related operational master data through a centralized, user-friendly interface while maintaining data integrity and consistency across the SKA Airlines Reservation System.

 Version SQLite 67 | MySQL 65

✨ In this version, an important enhancement was developed for the User Registration module. User registration is now completed successfully only after entering a valid OTP sent to the registered email address.

 Fixed known issues and improved overall code reusability and maintainability across the application.

Overall, Version 65 significantly improves airport data management, booking accuracy, and system-wide consistency while providing a more user-friendly airport display experience. ✨ 

## # 🚀 Version 68 (SQLite) | Version 66 (MySQL) – Connecting Flights System 🌐

In this version, developed a new **Connecting Flight Suggestion System** to improve booking flexibility when no direct flights are available between an origin and destination. ✈️ ➡️ ✈️

---

### ## ✈️ New Feature: Step-wise Connecting Flights System

When a direct flight is not available, the system now automatically suggests **multi-step flight routes** based on available connecting airports. 🌐

This feature intelligently calculates and suggests:

- \* ✈️ Intermediate hub airports
- \* ⌚ Connection time validation between flights
- \* ⌚ Minimum layover time: **40 minutes**
- \* 🕒 Maximum layover time: **up to 6 hours**

This ensures realistic and safe connection planning for passengers. ✅

---

### ## 🛠️ New Modules Developed

To implement this feature, the following components were created:

- \* 📄 `connect\_flight.py` (backend logic)

- \* 🌐 `connect\_flight.html` (UI for displaying connection routes)

---

## ## 🔄 Integration Across System

The `connect\_flight.py` module has been integrated into:

- \* 👤 Guest User ticket booking flow
- \* 🛠️ Admin ticket booking system

Now, when no direct flights are found:

- \* ✈️ The system automatically suggests step-wise connecting flights
- \* 🕒 Users can book individual flight segments
- \* 💡 This avoids the need to redesign the entire application architecture

---

## ## ⚙️ Bug Fixes & Improvements

- \* 🐛 Fixed known issues across the flight booking system
- \* 🔄 Improved fallback logic when no direct flights exist
- \* ✈️ Enhanced reliability of flight search and routing engine
- \* 🧠 Improved system decision-making for alternative routes

---

## ## 🌟 Key Improvements in Version 68

- \* ✈️ Introduced Connecting Flight Recommendation System
- \* 🕒 Added intelligent multi-airport route suggestions
- \* ⌚ Implemented layover constraints (40 mins – 6 hrs)
- \* 🛠️ Developed `connect\_flight.py` and `connect\_flight.html`
- \* 👤 Integrated into both Admin and Guest booking flows
- \* 🔄 Improved system scalability without redesign
- \* 🐛 Fixed existing known system issues
- \* 🌍 Enhanced overall booking flexibility and user experience

---

Overall, Version 68 significantly improves flight availability handling by introducing smart connection-based routing, making the system more powerful, flexible, and user-friendly. 🚀 ✈️

## 🚀 Version 69 (SQLite) | Version 67 (MySQL)

🔒 As part of this version's security enhancements, a CAPTCHA verification layer has been added as an additional login security measure.

✅ CAPTCHA validation has been integrated into both `admin\_login.py` and `user\_login.py`.

🎨 The login page UI templates have also been updated to support and align with this new security feature.

🔧 To implement this functionality, a new module named `login\_captcha\_generator.py` was developed and placed inside the `security` folder.

🚀 Version 70 (SQLite) | Version 68 (MySQL)

🔒 As part of the login security enhancement, account lockout protection has been implemented.

✅ If a user or admin reaches **\*\*5 or more failed login attempts\*\***, the account will be automatically locked for **\*\*24 hours\*\***.

🔧 To support this feature, two new modules were developed:

\* `admin\_login\_attempts.py`

\* `user\_login\_attempts.py`

📦 These modules have been integrated into:

\* `admin\_login.py`

\* `user\_login.py`

📖 Additionally, two new columns were added to both the `airline\_admins` and `airline\_users` tables:

\* `login\_attempts`

\* `login\_locked\_until`

🚫 The Login button is now automatically disabled when the failed login attempts reach the configured threshold and the account is locked, preventing further login attempts until the lockout period expires.

🔒 This enhancement provides stronger protection against brute-force attacks and unauthorized access attempts.

✈️ Version SQLite 71 | MySQL 69

🚀 In this version, a new flight search module has been implemented, allowing users to search for flights using airport names.

📋 The module displays a list of flights originating from the selected airport, along with their departure times and arrival times at the destination airport.

🔧 Fixed known issues in the guest user module `flight_operational_status.py`:

Corrected an issue where cancelled flights were incorrectly displayed as ON\_TIME.

Improved flight status display accuracy.

Fixed various UI-related issues for a better user experience. ✨

✅ Overall improvements have been made to flight search functionality, flight status accuracy, and the guest-side user interface. 🎉

✈️ Version 72 (SQLite) | Version 70 (MySQL)

✨ In this version, two new Python programs were developed to dynamically fetch and display airline information from the database:

📌 `news_flights_aircrafts_airports.py`



Retrieves the latest 2 newly added flights ✈️

Retrieves the latest 2 newly added airports 🏢

Retrieves the latest 2 newly added aircraft types 🛩️

📌 total\_flights\_routes.py

Retrieves the total number of active flights ✈️

Retrieves the total number of flight routes 🌐 operated by SKA Airlines

🔧 These programs have been imported into the required routes and the main application to display dynamic information on the homepage/index page.

✅ Removed hardcoded news and statistics data.

🎯 The News & Events section now automatically displays the latest flight, airport, aircraft, and route information directly from the SKA Airlines database, ensuring that users always see accurate and up-to-date operational information. ✈️ 🌐 🛩️

🚀 \*\*Version 72 (SQLite) | Version 70 (MySQL) — Continued\*\*

✨ In this version, various UI/UX improvements and issue fixes have been implemented across the SKA Airlines application.

### 🧠 UI Improvements

\* Fixed multiple UI issues across the application for a cleaner and more consistent user experience.

\* Added the **"Fly Beyond Limits"** caption throughout the UI to strengthen SKA Airlines branding and identity.

### ♻️ Reusable Copyright Component

\* Created a reusable template:

\* `templates/common/airline_copyright.html`

\* This component displays the `SKA Airlines copyright footer` along with the current year and can now be included across multiple templates, reducing duplicate code and improving maintainability.

### ### ✈️ Flight Schedule Integration

\* Added a `Flight Schedule` button within the ticket booking window whenever the list of available flights is displayed.

\* This allows users to quickly view flight schedules while booking tickets.

### ### 🐛 Major Seat Availability Fix

\* Fixed a major issue in the ticket booking module where `seat availability checks` were not functioning correctly when full airport details were used.

\* The issue occurred when airport selections included:

- \* Airport Name

- \* City Name

- \* Country Information

\* Seat availability now works correctly with complete airport information, ensuring accurate booking and seat allocation.

✅ These enhancements improve usability, maintainability, and the overall booking experience while ensuring accurate flight and seat availability information throughout the SKA Airlines system.

## 🗣 ChatBot Introduced to the Airline Application (Guest Side)

In this version, a new AI-powered ChatBot has been introduced to the airline application for the guest user side.

💡 The chatbot is designed to be intelligent and interactive. It responds based on predefined suggestions, provides time-based greetings, and can remember the user's name throughout the current session.

Example:

User: "Hi, I'm Sai Kiran"

ChatBot: "Hello, Sai Kiran! 🙌"

The chatbot will then continue using the user's name during the session to provide a more personalized experience.

## 📁 Project Structure

### Backend Structure

airline/

├─ chat\_bot/

├─ guest/

### Templates Structure

templates/

├─ chat\_bot/

└— guest/

This separation keeps the chatbot logic and templates organized, making the project easier to maintain and preventing the codebase from becoming cluttered.

#### Current Features

 Time-based greetings (Good Morning / Good Afternoon / Good Evening)

 Session-based user name memory

 Suggestion button support

 Airline information assistance

 Flight-related query support

 News & Events integration

 PNR lookup support

 Flight Schedule information

 Airport Schedule information

 Wallet information

 Feedback support

The chatbot is capable of handling a large amount of information and serves as a centralized assistance system for the guest-side index page.

#### SOON Next Version

The next version will focus on implementing and integrating the ChatBot for:

 User Dashboard / Logged-in User Side

 Admin Side

This will provide a consistent and personalized chatbot experience across the entire airline application.



 Version 74 (SQLite) / Version 72 (MySQL)

 Continuing from Version 73 (SQLite) / Version 71 (MySQL), this release focuses on the implementation and expansion of the SKA Airlines Chatbot across the airline project.

 Chatbot Enhancements

Implemented the chatbot on the User Side and Admin Side.

Added session-based greetings and personalization.

The chatbot can accurately understand and respond to various requests and scenarios.

Introduced support for numerous keywords, realistic phrases, and common typing mistakes for a smoother experience.

 Code Improvements

All chatbot modules were designed with reusability, maintainability, and modularity in mind.

The chatbot architecture is optimized to be fast, responsive, and smooth.

Organized the codebase to make future expansion and maintenance easier.

 UI, Backend, and Database Improvements

Resolved various issues across the UI, backend, and database layers.

Improved overall stability and performance.

Enhanced the user experience with dynamic buttons and context-aware responses.

 Project Status

This is one of the most feature-rich and stable versions of the project so far. 

 The development of SKA Airlines continues, and more features and improvements will be introduced in future versions.

🔧 This continued release focuses on improving session security and enhancing the chatbot user experience.

### 🔒 Session Management Improvements

Fixed a major issue related to session persistence. Previously, if a user or admin logged into the application on a device and forgot to log out, the session could remain active indefinitely, especially when the application was hosted.

To address this issue:

💡 Developed `session_manager.py` and placed it inside the security folder.

⚙ Integrated it with `airline.py` alongside `utc_time.py`.

🕒 Implemented automatic session expiration based on inactivity.

#### 👤 Admin Session

If an administrator forgets to log out, the system will automatically terminate the session after 20 minutes of inactivity.

#### 👤 User Session

If a user forgets to log out, the session will automatically expire after 2 hours of inactivity.

This enhancement improves both security and resource management, particularly in hosted environments.



### 🤖 Chatbot Improvements

✨ Updated the chatbot interface across all sides of the project.

📄 Replaced the previous suggestion buttons with a dropdown-style suggestion menu, providing a cleaner and more responsive experience.

🚀 Improved overall usability and interaction with the SKA Airlines Assistant.

## Overall

This continued version significantly strengthens the application's security, session handling, and user experience, making SKA Airlines more reliable and production-ready.  

### Version 74 (SQLite), 72 (MySQL) — Continue 2

In this continued version, several improvements and security enhancements were implemented for the SKA Airlines Chatbot system.  

 Chatbot template protection was introduced by developing dedicated routes.py files for the Admin and User modules. This ensures that pages such as chat\_bot\_window.html are accessible only to authorized sessions.

 Additional backend security was implemented through chat\_bot\_routes.py. Even if someone attempts to access the chatbot endpoints directly through JavaScript or manually invoke API routes, both the templates and backend routes are protected using the corresponding utils.py authentication mechanisms for Admin and User modules.

 An extra layer of security and safety was added to the chatbot architecture, providing enhanced protection against unauthorized access and improving overall reliability.

 The chatbot was further refined to deliver a more realistic, responsive, and user-friendly experience across Guest, User, and Admin modules.

 Various known issues were also fixed, resulting in a smoother, faster, and more stable chatbot system.

 As always, the project continues with a focus on reusability, maintainability, performance, and continuous improvement.  

----- End of Changelog -----

-----

## Issues Encountered and Resolutions ⚠️✅

During the development of the SKA Flight Booking System, several challenges arose, including:

- Data synchronization complexities between flight schedules and real-time seat availability, which were resolved by implementing efficient database transaction controls and caching mechanisms. 🔄📱
- Payment gateway integration hurdles, especially ensuring secure, ACID-compliant transactions, overcome by adopting robust API protocols and thorough testing. 💳🔒
- User authentication security risks, mitigated by strengthening password policies, session management, and encryption techniques. 🗝️🛡️
- Handling scalability for growing user base and flight data, addressed by modular system design and optimizing query performance. 📈⚙️

These solutions helped enhance the system's reliability, security, and performance, laying a strong foundation for future upgrades and maintenance.

### ⚠️ ISSUES & BUGS DURING SKA AIRLINES APPLICATION DEVELOPMENT 🛩️🐛

#### Issue 1: Journey Date vs Return Date 📅➡️⬅️BACK

Problem: Return date must be after the journey start date.

Status: Resolved ✅ with JavaScript validation ensuring the return date is always later than the journey date.

#### Issue 2: Date Format Consistency 📅

Problem: Date formats should appear as 12/Jul/2024 for consistency and clarity.

Status: Fixed ✅ by enforcing date input and display formatting via JavaScript.

#### Issue 3: Round Trip Discount 🚫🔄

Problem: When trip mode is Round Trip, a discount of 1% should apply on the total trip cost.

Status: Resolved ✅ using JavaScript logic to automatically calculate and apply the discount.



#### Issue 4: Automatic Airport Name Retrieval ✈️ 🏠

Problem: Airport names were hardcoded and not dynamically fetched.

Status: Solved ✅ by fetching airport data from the database using Python (airport.py globally and respective routes.py modules for guest/admin).

#### Issue 5: Realistic Ticket Layout & Seat Allocation 🎫 🪑

Problem: Seat assignment during booking was not automatic or realistic.

Status: Fixed ✅ with backend Python code implementing automatic seat allocation at booking time.

#### Issue 6: Reservation Data Transfer via Sessions 🔁 📄

Problem: Booking details were not preserved properly across multi-step forms.

Status: Resolved ✅ by managing data flow through Flask sessions, maintaining state across booking steps.

#### Issue 7: Dynamic Passenger Selection 👤 👤 👤 👤 ➕

Problem: Passengers couldn't book tickets for multiple people simultaneously.

Status: Fixed ✅ by introducing dynamic JavaScript forms allowing multiple passenger entries in one booking.

#### Issue 8: Quota Reservation Discounts 🏆 🎫

Problem: Defence students discount (Rs 1000) not applied correctly, especially for round trips.

Status: Resolved ✅ — Defence quota discount now applies Rs 1000 for one-way and Rs 2000 for round trips.

#### Issue 9: Automatic Fare & Class Updates 💰 ⚙️

Problem: Fare details (base fare, quota fare, class fare) did not auto-update when adding passengers.

Status: Fixed ✅ with JavaScript dynamically recalculating and updating all fare components instantly.

#### Issue 10: MySQL Trigger on airline\_reservation Table 🔔 🗄️

Problem: Needed DB-level triggers to synchronize seat assignments with reservations.

Status: Resolved  through a MySQL trigger, updating the airline\_sitting table and displaying all PNRs with passengers accordingly.

#### Issue 11: Single PNR for Multiple Passengers

Problem: Passengers booking together did not share the same PNR.

Status: Fixed  — multiple passengers now share one PNR with distinct seats during a single booking.

#### Issue 12: Realistic Plane Seat Layouts

Problem: Seat allocation was not class-based or reflecting realistic flight seating.

Status: Resolved  by Python code dynamically managing seat layout based on flight class selection.

#### Issue 13: Ticket Download Instead of Printing

Problem: Printing tickets was inconvenient and non-digital.

Status: Solved  via an HTML template allowing users to download tickets to their devices instead of printing.

#### Issue 14: Correct Round Trip Charge Calculation

Problem: Charges were not doubling for round trips; quota discounts also miscalculated.

Status: Fixed  through JavaScript validation ensuring base fare doubles for round trip selections and discounts adjust accordingly.

#### Issue 15: Auto-fetch User Name Using User Number

Problem: Admin had to manually look up user names after entering user numbers.

Status: Resolved  — When admin enters user number, the username is automatically fetched from the database dynamically (airline\_users.py with proper routes to HTML). Also applied to admin's user booking history views.

#### Issue 16: User Number vs Name in Admin Audit

Problem: Need dynamic display of user name when entering user number during admin audit or booking history review.

Status: Resolved  — Username now dynamically displays in the text box in admin ticket booking and user booking history audit.

#### Issue 17: Mapping User Booking History with Admin PNR Enquiry

Problem: Need to reduce redundant code by mapping user booking history to admin PNR enquiry based on user number input.

Status: Done  — Module developed to hold all user ticket bookings linked with PNR, accessible via admin enquiry for quick audit.

#### Issue 18: Auto-Clear Journey & Return Dates via SQL Trigger or Query

Problem: Need to automatically clear journey and return dates after completion.

Status: Business logic changed — Instead of deleting dates, jstatus is updated to "completed" once journey and return dates pass. 

#### Issue 19: Auto-Use Bank Account Details on Ticket Cancellation

Problem: Avoid asking user for bank account details again upon cancellation; refund should go automatically to the account used at booking.

Status: Implemented  — Added account number column in airline\_reservation table and automated refund process to same account at cancellation.

#### Issue 20: Fare Auto-Population in Guest/Index Ticket Booking Page

Problem: Fare was not auto-fetched with base fare in index ticket booking.

Status: Resolved  — Fare now auto-populates on the ticket booking page for guests and users.

#### Issue 21: Combined Plane Schedule for Guest, User, and Admin

Problem: Separate plane schedule modules caused code duplication.

Status: Fixed  — Developed unified module combining guest, user, and admin plane schedules for efficient code maintenance.

Issue 22: Ticket Cancellation Module for Admin & User with Refund 🟢❌

Problem: Need cancellation modules for both admin and user with automatic refund crediting.

Status: Implemented ✅ — Both user and admin ticket cancellation with refund processing in place.

Issue 23: Dynamic Flight Name Display on Plane Schedule by Flight Number ✈️📄

Problem: Flight name should dynamically display when flight number is entered in scheduling.

Status: Done ✅ — Flight names now fetch and display dynamically on input of flight number.

Issue 24: Form Close Button Should Not Trigger Empty Popups ❌❌

Problem: Closing forms with buttons caused unwanted empty popup forms.

Status: Fixed ✅ — Close buttons use correct type="button" and proper form attributes to avoid empty popups, plus UI fixes.

Issue 25: Global PNR Status Common for Index & User Pages 🗑️🌐

Problem: Need common PNR status view usable by guest, user, and admin for downloading tickets and viewing bookings.

Status: Developed ✅ — Shared module allows common access yet restricts downloads to respective users only.

Issue 26: Add “New User? Please Register” Link on User Login Page 🔑🔒

Problem: Login page lacked direct link to registration for new users.

Status: Fixed ✅ — Hyperlink/button added redirecting new users directly to registration page.

Issue 27: Capitalize Passenger Names Automatically ✎️🔠

Problem: Passenger names were entered inconsistently with regards to case.

Status: Resolved ✅ — Python code converting passenger names to uppercase before saving.

Issue 28: Passenger Details Alert in Booking 🚨📄

Problem: Users/admin need to be cautioned to enter passenger details exactly as per government ID.

Status: Fixed  — Alert message implemented on booking page. Plans for master list integration for added data validation.

Issue 29: Confirmation Popup on Ticket Cancellation  

Problem: Users should confirm cancellation intent via alert popup.

Status: Done  — Confirmation alert implemented before processing cancellation.

Issue 30: Combined User Journey History & Ticket Download Module  

Problem: Required module to show user journey list with direct ticket download without entering PNR again.

Status: Developed  — Users access complete journey with ticket download options seamlessly.

Issue 31: Separate Modules to Sort CNF, CAN & Complete Journey Tickets   

Problem: Need modular separation for Confirmed (CNF), Cancelled (CAN), and full ticket journeys.

Status: Completed  — Separate modules created with clear logic for respective ticket statuses.

Issue 32: User Sessions to Store Number, Name, Feedback, Mobile & Password Update  

Problem: Data such as user number and name should be stored in session for easier retrieval and to enhance code maintainability.

Status: Implemented  — User sessions now hold essential data, accessed via Python backend to maintain high data integrity.

Issue 33: Admin Side Ticket Cancellation Requires Extra User Account Handling   

Problem: Admin ticket cancellation needs to capture and validate user account number properly.

Status: Resolved  — Admin side cancellation module enhanced with user account number retrieval and validation.

Issue 34: Auto-Generate Fare Based on Distance (KM)  

Problem: Fare calculation should dynamically reflect distance between airports (e.g., Rs 12 per KM).

Status: Implemented  — SQL trigger created for automatic fare generation based on KM distance between origin and destination.

Issue 35: Seat Allocation Trigger with Cancelled Seats Fix  

Problem: Trigger logic not correctly handling cancelled seats (seats not properly freed/reassigned).

Status: Fixed  — Corrected trigger logic to handle seat availability accurately on cancellations.

Issue 36: Cancelling Confirmed Tickets Fails for Multiple Passengers (>2)  

Problem: Ticket cancellation fails when more than two passengers booked under same PNR.

Status: Resolved  — Fixed by replacing fetchone() with fetchall() in seat handling logic.

Issue 37: Journey Status Should Be “In Progress” (IP) Until Ticket Cancelled  

Problem: Journey status not reflecting “In Progress” until ticket cancellation.

Status: Implemented  — Python code updated to set journey status to “IP” during booking lifecycle.

Issue 38: Auto-Fetch Bank Account Number for Refunds  

Problem: Bank account used in booking should auto-fill in cancellation refund process.

Status: Fixed  — Implemented code to retrieve and use user’s bank account during refund.

Issue 39: Match Transaction Scenarios With Banking Logic  

Problem: DB transaction records didn’t correctly reflect debit/credit flow for bookings and cancellations.

Status: Fixed  — Updated transaction entries so that booking debits user account & credits airline account, cancellation reverses credit/debit accordingly.

Issue 40: Admin-Booked Ticket Cancellation Logic    

Problem: Tickets booked by admin must only be cancellable by admin (counter ticket cancellation), without involving user number fetching.

Note: Admin retains power to cancel user-booked tickets.

Status: Resolved  — Removed user number requirement for admin-booked ticket cancellations to streamline process.

#### Issue 41: Professional “All Rights Reserved” Footer Addition ©®

Requirement: Add a consistent footer on every page displaying:

SKA AIRLINES ALL RIGHTS RESERVED © ®

Status: Implemented  — Footer with copyright and registered mark added site-wide.

#### Issue 42: Improve Code Reusability

Goal: Create common reusable packages for HTML, including JS scripts and CSS, to avoid duplication and ease maintenance.

Status: Completed  — Modular includes implemented for reusable frontend assets.

#### Issue 43: Seat Allocation Trigger Fix for Multiple Cancelled Seats

Problem: Trigger logic failed to reassign cancelled seats when more than one seat is available for the same flight/class/journey.

Example: For First Class with 12 seats, 2 cancelled seats were not reassigned correctly.

Status: Fixed  — Trigger logic corrected to allocate cancelled seats properly and in order.

#### Issue 44: Phase 2 Development Module Summary

SKA Airlines is building dynamic & modern features:

 QR Code on Ticket PDF: Fast airport check-in & digital verification (Generation and Base64 encoding done)

 Email System / WhatsApp Notifications: Booking and cancellation updates

 QR Code Payment Gateway (Simulated): Realistic payment simulation via QR (done)

 Graphical Admin Dashboard (Optional): Visual insights for bookings, revenue, cancellations

🔧 Advanced Seat Allocation by Flight Type: Flight types (Boeing, Airbus) with seat configs per class; auto seat range setup done

🔧 Common DB Configuration File: Centralized DB connection (db\_config.py)

🔄 Dynamic Fare Calculation from DB: Fare fetching and calculation done dynamically, avoiding static HTML fares

📊 Admin Fare Update & Visualization Page: Admin can update fares by class/quota/km with optional fare increase calculations

🌐 Dynamic Domain/IP Configuration: Application accessible via domain instead of raw IP (done)

Issue 45: QR Code on Ticket Implementation 🚫

Requirement: Develop an importable Python module to generate QR codes containing ticket reservation details, embedded in ticket layout after booking.

Status: Completed ✅ — QR code generation module implemented and integrated in tickets successfully.

Issue 46: QR Code for User-Side Payment Gateway 🔄

Requirement:

Develop a dedicated QR code generator module that encodes transaction details like fare and bank URL.

When the user scans the QR, it opens a bank authentication window for login and payment authorization.

Upon successful payment, the bank backend notifies the ticketing system, triggering ticket generation and payment confirmation.

Maintain a transaction ID (txn\_id) tracked in the qr\_payment\_status table.

Pass user's bank account ID (acid) into the airline\_reservation table for automated refunds on cancellations.



Flow:

ticket\_user4.html (QR module) → use1qr.jsp (bank) → accept\_qr\_payment.jsp → authorised\_payment.jsp  
→ qr\_payment\_gateway.py → qr\_payment\_su.html → ticket\_userdb.py → ticket\_user\_su.html

Status: Debugging completed and issue resolved .

Issue 47: Email-Based Ticket Confirmation & Cancellation Notifications  

Requirement:

Implement an email system that automatically sends booking or cancellation confirmations to users with ticket details (flight number, passenger name, class, journey date, etc.).

Add an email\_id column to the airline\_users table to store user emails.

Create a dedicated email module linked to user-side booking and cancellation processes.

For admins, sending emails on ticket booking is optional but recommended.

Status: Implemented and fully operational .

Issue 48: Centralized Database Handling & Security  

Requirement:

Create centralized DB configuration for MySQL and Oracle to avoid exposing database credentials across multiple files/pages.

Facilitate easy maintenance and improved security.

Status: Successfully implemented .

#### Issue 49: Including Flight-Specific Charges in Fare Calculations

Requirement:

Include individual airline flight charges dynamically during booking fare calculation.

Status: Completed and integrated successfully .

#### Issue 50: Dynamic Fare Calculation from Database Based on Class & Quota

Requirement:

Fetch fares dynamically from DB tables storing class fares and quota discounts (e.g., Defence discount).

Allows admins to update fares in the DB without changing application code, enhancing business flexibility.

Example: First class fare = 1500 ₹.

Status: Fully implemented and tested .

#### Issue 51: Reflecting Bank Account ID in QR Payment Gateway Flow

Requirement:

Add ac\_number column in the qr\_payment\_status table to hold user bank account info.

After payment confirmation, insert the bank account ID into the corresponding record in the airline\_reservation table for reconciliation and refund processing.

This solves cancellation/refund issues when payment mode is QR code.

Status: Logic corrected and issue resolved .

#### Issue 52: Email Delivery Error on Ticket Cancellation Confirmation ✉️ ❌

Problem:

Cancellation confirmation emails were failing due to errors in passenger list formatting (specifically, incorrect access like `p['name']`).

The fix involved correctly fetching email and passenger details from the `airline_users` table for reliable email content.

Status: Bug fixed and emails are now delivered successfully ✅.

#### Issue 53: QR Code Missing in Ticket Cancellation Emails 📄 ✉️

Problem: When a user cancels a confirmed ticket, the email sent does not include the QR code—only textual information is delivered.

Root Cause: QR generation was implemented only for confirmation emails (`email_cnf_can.py`), not for cancellations.

Resolution: Redesigned email module to include QR codes for both ticket confirmations and cancellations.

Status: Done ✅ — QR code now generates correctly in cancellation emails.

#### Issue 54: Duplicate Emails Sent on Browser Refresh 🌐 🔄 ✉️

Problem: Refreshing the endpoint page causes the system to resend booking/cancellation emails again, causing duplicates.

Fix: Placed email-sending logic before success methods in backend Python code for user and admin booking/cancellation routes, preventing resends on page refresh.

Status: Resolved ✅.

#### Issue 55: Error Pages on Browser Refresh after Confirmation 🌐 ❌

Problem: Refreshing the confirmation pages after ticket generation causes error pages, particularly with cancellation endpoints for both user and admin sides.

Fix: Adjusted flow so that database confirmation data is properly passed to success pages, preventing errors on refresh. Status: Fixed ✅.

Issue 56: Add Email Column to airlines\_reservation Table for Admin Use 📧 📱

Requirement: Store users' email addresses in the reservations table to allow admins to send email notifications during ticket bookings.

If user email is null, no email sent (neutral handling).

Status: Implemented successfully ✅.

Issue 57: Fetch User Email ID during Admin Ticket Cancellations 📧 ⚙️

Requirement: Retrieve stored email from airlines\_reservation table to send cancellation notices when admin cancels tickets.

Status: Implemented and working ✅.

Issue 58: Insert User Email into Reservation Table during User Booking 📧 🇺🇸

Requirement: Store email in email\_user column during user-side bookings for use during admin-side cancellation notifications.

Benefits: Enables admins to notify users if admin cancels user-booked tickets, improving communication and trust.

Status: Completed ✅.

Issue 59: Dynamic Advanced Seat Allocation by Flight Details type 🪑 ✈️

Requirement: Implement dynamic seat allocation factoring in flight number, name, class, journey date, origin, destination, and flight type (e.g., Boeing seats allocation to Boeing flights).

Current Status:

Existing 3 scenarios had limitations; seats\_avl.py restricts reservation seats at frontend.

Old trigger allocates seats based on flight type but needs enhancement to handle larger seat configurations (i.e., Boeing 747 FC with 14 seats, etc.).

Next Steps:

Upgrade trigger logic to handle larger seat layouts per aircraft type.

Add submodule in admin for selecting aircraft type while adding a flight, storing flight type in the flights table.

Store flight\_type also in airlines\_sitting table for accurate seat availability tracking.

Pass flight type info in tickets and emails (including QR codes) during reservation to ensure consistency.

Outcome:

A single, robust trigger plus frontend logic (seats\_avl.py) enables seamless and accurate seat allocation while handling cancellations dynamically.

Status: Fully implemented and all related issues resolved .

Issue 60: Display Flight Type & Flight Charge on Tickets  

Requirement: Add Flight Type and Flight Charge details visibly on the ticket, QR code, and email confirmations.

Status: Fixed  — Flight Type and Flight Charge now clearly displayed on tickets and included within QR codes and emails.

Issue 61: Increase Seat Allocation Trigger for Largest Flight Layouts  

Requirement: Enhance the seat allocation database trigger to support the highest-capacity aircraft like Boeing 747, which have large seat layouts.

Status: Completed  — The trigger was successfully scaled to handle the largest layouts seamlessly. Impressive performance ensured! 

Issue 62: Add Exact Seat Layouts in seats\_avl.py for Frontend Validation  

Requirement: Integrate precise seat layouts within the seats\_avl.py module to validate seat availability and booking on the frontend, preventing overbooking or invalid selections.

Status: Resolved  — The program now validates seat layout accurately during booking.

Issue 63: Add Flight Type Column in airlines\_reservation Table  

Requirement: Add a new column in the airlines\_reservation DB table to store the flight type, enabling more accurate data retrieval especially during ticket cancellation workflows.

Status: Done  — Flight Type column added and integrated successfully.

#### Issue 64: Correct Display of Seat Availability with Origin-Destination Consideration

##### Problem:

When origin and destination are swapped (e.g., Delhi ↔ New York on flight AA12AA), the seat availability was incorrectly shown as the same for both directions.

Requirement: Seat availability must reflect exact journey direction and other parameters like journey date, flight type, etc.

Fix: Updated seats\_avl.py to consider origin and destination properly during seat availability checks, giving accurate display of available seats.

Status: Issue fixed  — Seat availability now reflects exact journey data, ensuring correctness and user trust.

#### Issue 65: Tickets Displaying Past Journey Dates

##### Requirement:

Restrict displaying **past journey dates** in user ticket history using the query:

sql

```
SELECT * FROM airline_reservation
```

```
WHERE ausr = %s AND status='CNF' AND jstatus='IP'
```

```
AND STR_TO_DATE(jd, '%d/%b/%Y') >= CURDATE() //For the Todays date it will not allow to cancel the ticket
```

To print or view the ticket then user needs to go to the My Account/ My Journey History

```
ORDER BY PNR DESC, (uno,)
```

Status: Done  — Successfully resolved the issue. Past journey tickets are no longer displayed.

#### Issue 66: Dynamic Seat Allocation Fetching in seats\_avl.py

##### Requirement:

Fetch available seats dynamically from the database table aircraft\_seats\_layout.

Admins will be able to update seat details via a frontend HTML module under development.

Status: Done  — Dynamic seat availability fetching implemented and working smoothly.

### Issue 67: Display Flight Class and Quota Fares 💰✈️

#### Requirement:

Display fare details including flight class and quota charges by accessing backend database table with secure, read-only admin credentials. Database user creation example:

sql

```
CREATE USER 'airline_admin'@'localhost' IDENTIFIED BY 'airlineadmin';
```

```
GRANT SELECT ON airline.* TO 'airline_admin'@'localhost';
```

```
FLUSH PRIVILEGES;
```

For seat management by flight type, the following privileges are required:

sql

```
GRANT SELECT, INSERT, UPDATE, DELETE ON airline.my_table TO 'admin'@'localhost';
```

```
FLUSH PRIVILEGES;
```

(Note: DELETE permission is optional. If issues arise, airline admin should coordinate with the MySQL database developer/admin.)

**Status:** Ongoing — Database permissions configured to securely manage fare and seat data.

### 🐛 Issue 68: QR Code on Payment Gateway Editable via HTML Inspect 🐛

There was a major bug where the QR code on the payment gateway could be manipulated by editing the HTML elements through Inspect Element.

For example, the total fare displayed as Rs. 12,000/- 💰 could be changed to Rs. 10/- by tampering with the HTML, breaking payment security. 🔒 ❌

#### 🔒 Fix Implemented: Secure Server-Side QR Code Generation Using Sessions

Instead of generating QR code values from editable frontend HTML/JS, all sensitive values come directly from backend sessions:

python

```
txn_id = session.get('txn_id')
```

```
total_fare = session.get('total_fare')
```

```
trm = session.get('trm', 'Db SKA-AIRLINES')
```

```
if not txn_id or not total_fare:
```

```

 abort(400, "Invalid session or missing transaction")

ip_address = get_local_ip()

Secure QR generation with server-side values only
bank_url = f"http://{ip_address}:8181/bank/uselqr.jsp?amount={total_fare}&mode={trm}&txn_id={txn_id}"
img = qrcode.make(bank_url)

buffer = io.BytesIO()
img.save(buffer, format='PNG')
buffer.seek(0)

return send_file(buffer, mimetype='image/png')

```

Simplified flow in routes.py:

```

python

@user_dashboard_bp.route('/ticket_user4', methods=['GET', 'POST'])
@login_required
def ticket_user4():
 step4_data = session.get('step4_data', {})
 combined_data1 = {**step4_data}
 txn_id = str(uuid.uuid4())[:8]
 total_fare = combined_data1.get("total_fare1") # safer than dot attr
 trm = "Db SKA-AIRLINES"

 # Store securely in session
 session['txn_id'] = txn_id
 session['total_fare'] = total_fare
 session['trm'] = trm

 return render_template('user/ticket_user4.html', txn_id=txn_id, combined_data1=combined_data1)

```

In payment\_gateway\_Qr.py, the QR is generated using session values only, ensuring security.

Backend trusts only session data, so end users cannot modify QR code contents via Inspect Element.  



### ⚠ Summary of Critical Security Fix

Payment QR generation moved from insecure frontend (HTML/JS) to secure backend sessions.

Users cannot tamper with QR values by inspecting or editing HTML.

Payment integrity and transaction safety fully restored.

Payment status polling (check\_payment\_status) continues to work smoothly using secure txn\_id. 🔄 💳

### 🐛 Issue 69: Tickets Displaying Past Journey Dates 🛫 📅

Requirement:

Restrict displaying past journey dates in user ticket history using the following query:

sql

```
SELECT * FROM airline_reservation
```

```
WHERE ausr = %s AND status='CNF' AND jstatus='IP'
```

```
AND STR_TO_DATE(jd, '%d/%b/%Y') > CURDATE()
```

```
ORDER BY PNR DESC, (uno,)
```

Status: Done ✅ — Successfully resolved the issue. Past journey tickets are no longer displayed in the user's ticket history.

### Issue 70: Add Hyperlink for Ticket Cancellation Redirect 🔗 🛫

When users want to cancel a ticket, display a message like:

To view, print, or download your ticket, please use My Journey History, where you can find, print, or view your confirmed tickets.

🔗 To View Your Confirmed Tickets:

🖨 / 👁 [View/Print Confirmed Tickets]

Issue Resolved: ✅

A hyperlink was added to smoothly navigate users to their journey history for ticket printing or downloading.

## Issue 71: Automatic Display of Current Year on Web Pages

The year on the web pages should automatically update based on the system year, eliminating the need for manual edits annually.

This includes replacing hardcoded years like “All Rights Reserved 2025” with a dynamic year display.

### Implementation Details:

Created a separate JavaScript file with the code:

xml

```
<script>document.getElementById('year').textContent = new Date().getFullYear();</script>
```

Imported this file where needed across the project pages.

Applied only in specific key pages/modules:

HTML Index

User Login

Admin Login

Admin Dashboard

User Dashboard

Register Module

 Current Year is dynamically displayed without manual intervention.

### Impacted modules:

Index, admin login, user login, welcome admin, welcome user, guest ticket, user register, guest plane schedule, news and events, feedback.

 Excluded: No need to add this in ticket pages as copyright year is already declared.

## Issue 72: Dynamic Retrieval of Flight Types from Database

### Requirement:

Create a new database table named `flight_types` to store different types of flights dynamically. 

This new table replaces the hardcoded flight types on HTML pages where admins add new flights.

The table should be editable, allowing admins to add new flight types as needed in the future. 

 **Status:** Issue fixed — Flight types are now dynamically retrieved from the database table, ensuring flexibility and easier future updates.

## Issue 73: Flight Class and Quota Validation While Adding New Flight

### Problem:

When the admin adds a new flight, if they miss selecting the flight class or quota, the system was accepting null values, causing data inconsistency. 

### Fix Implemented:

Added JavaScript validation to check that the admin selects flight class and quota (along with other mandatory fields).

If any required field is missed, the admin is immediately notified to fill it properly before submitting.  

 **Status:** Issue resolved — Admin cannot submit incomplete flight details without selecting essential flight class and quota.

#### Issue 74: Time Module — Departure & Arrival Times Not Inserted 🕒 ✈️

##### Problem:

While developing the time module, the departure time and arrival time were not being inserted into the database table. ⚠️

##### Reason:

Needed to add extra database columns for jd (departure time), arrival time, and flytime to store these values. 📁 ⌚

##### Status:

✅ Issue cleared — Added the necessary columns and data now inserts correctly!

#### Issue 75: Fly Time Not Reflecting with Departure & Arrival Times 🕒 ✈️ ⌚

##### Problem:

The flight departure time and arrival time were not reflecting the correct fly time from the flight\_legs table.

##### Reason:

Added the fly\_time field, which is now correctly calculated and displayed based on the flight's departure and arrival times. 🛠️ 📁

##### Status:

✅ Issue resolved — Fly time is now accurately synced with the departure and arrival times!

#### Issue 76: Arrival Date Not Updating for Overnight Flights 🌙 ✈️ 📅

##### Problem:

When a flight departs at night (e.g., 9 PM) and arrives the next morning (e.g., 9 AM), the arrival date was not changing—it remained on the previous day. ⚠️

##### Reason:

The flight time was not correctly considering the transition to the next day.

🛠️ To fix this, an extra program flight\_date\_hours.py was added in the guest folder to properly handle date and time calculations.

##### Status:

✅ Issue resolved — The arrival date now updates correctly to the next day for overnight flights!

### Issue 77: Departure Date Not Displaying in Ticket Booking Steps 📅✈️

#### Problem:

The departure date was not appearing during the next steps of ticket booking. ⚠️

#### Reason:

The session variable for departure date was missing, causing it not to carry forward between booking steps.



#### Fix:

✅ Session management was updated so the departure date persists until the ticket is fully booked, ensuring a seamless user experience and smooth UI flow!

### Issue 78: Reservation Table Not Capturing Arrival & Departure Times 📅🕒

#### Problem:

The reservation side table was not storing arrival and departure times. ⚠️

#### Reason:

The relevant fields were missing due to an incorrect or missing table index in the hidden table setup.



#### Status:

✅ Issue fixed — Corrected the table index, and now arrival and departure times are properly captured in the reservation table!

### Issue 79: Email Not Sending for Ticket Booking and Cancellation 📧✖️

#### Problem:

Emails were not being sent on both user and admin sides during ticket booking and cancellation. ⚠️

#### Reason:

Required parameters were not being passed correctly in the email sending functions. 📧🔴

#### Status:

✅ Issue fixed! Emails are now sent properly when tickets are booked or cancelled.

Additional details like departure at origin, arrival time, and fly time have been included in all relevant modules: Admin side: Ticket booking, PNR audit User side: Ticket booking, ticket cancellation, ticket download, journey history (confirmed, cancelled, and total)

Issue 80: Spelling Mistake - "Depature" Corrected to "Departure" ✏️ ✔️

Problem:

The word "Depature" was misspelled instead of "Departure".

This typo occurred in both the admin side ticket booking and the email program, causing inconsistencies.



Status:

✔️ Corrected the spelling to "Departure" in the admin side and email modules for accuracy and professionalism.

Issue 81: Develop Program for Airport Pairs Handling on New Flight Addition ✈️ 📝

Problem:

When an admin adds a new flight to the database, selecting multiple airports, the system needs to:

Insert the flight group into the flights table.

Split the selected airports into separate airport pairs.

Insert these pairs into both flight\_legs and airplane\_fare tables without duplications.

Reason:

Ensures consistent data entry where pairs of airports represent flight legs and fare routes properly, avoiding duplicates and preserving data integrity.

Fix Implemented:

Backend Python program at admin side splits the comma-separated airport list, forming pairs:

python

```

airports_list = [a.strip() for a in selected_airports.split(',') if a.strip()]

for i in range(len(airports_list)):
 for j in range(i+1, len(airports_list)):
 origin = airports_list[i]
 dest = airports_list[j]

 # Insert into flight_legs
 leg_query = """
 INSERT INTO flight_legs (flight_number, origin_airport, destination_airport, departure_time,
 arrival_time)
 VALUES (%s, %s, %s, '00:00:00', '00:00:00')
 """
 cursor.execute(leg_query, (flight_number, origin, dest))

 # Insert into airplane_fare avoiding duplicates
 origin, dest = sorted([origin, dest]) # Alphabetical order to prevent duplication
 route_str = f"{origin},{dest}"

 fare_query = """
 INSERT INTO airplane_fare (Airport, Fare, KM, AAD)
 VALUES (%s, NULL, NULL, %s)
 ON DUPLICATE KEY UPDATE Airport = Airport
 """
 cursor.execute(fare_query, (route_str, fadm))

```

The program ignores duplicate airport pairs, ensuring no redundant entries.

The flight\_legs table uses flight number as a unique key to prevent duplicate flights.

⚙ Additional Notes:

After adding flights, admins should manually enter KM in the airplane\_fare table.

Fare is auto-calculated as  $KM \times 12$  (automatically generated based on distance).

Admins must update flight origin and destination times with day codes in the flight\_legs table if times exceed 24 hours.

Fly time is automatically calculated based on these times.

Issue 82: Develop Trigger to Calculate Fly Time and Date Change on flight\_legs Table 🕒 ✈️ 📅

Problem:

Need to automatically calculate fly time (duration) and handle date changes in the flight\_legs table to avoid repeated calculations in Python code.

Example: If a flight departs at 5:00 PM and arrives the next morning at 10:00 AM, the arrival date must correctly advance to the next day. ⚠️

Reason:

Centralizing calculation in a database trigger improves efficiency and consistency, preventing issues with incorrect date calculations from Python side.

Status:

✅ Fixed with a database trigger that calculates fly\_duration between departure and arrival times.

✨ Additionally, a Python program named flight\_date\_hours.py was developed in the Guest folder to maintain accurate flight times and handle date transitions dynamically based on flight schedules.

This ensures the arrival date correctly updates to the next day when required.



### Issue 83: Develop Module to Display Flight Departure and Arrival Times in Plane Schedule

#### Problem:

Need to display flight departure time at origin and arrival time at destination in the plane schedule results, providing clear timing details alongside flight information.

#### Reason:

This enhancement will show users the exact timings for flights, including flight number, origin, destination, class, available quota, and more for better clarity.

#### Status:

✅ Fixed!

Added the module in plane\_schedule.py that fetches and displays:

Flight timings for origin and destination

Flight number, airport, flight name, class, quota availability from the flights table

Corresponding times and fly time from the flight\_legs table

### Issue 84: Flight Times Not Retrieving for Multi-Leg Flights

#### Problem:

When a flight has multiple legs, the system was not displaying all origins and destinations along with their respective times and fly time. ⚠️

#### Reason:

Missing retrieval of all leg details caused incomplete information display in the plane schedule.

#### Status:

✅ Fixed in the plane\_schedule.py Python code by:

Adding cursor.fetchall() to fetch all flight legs data.

Adding {% if time\_data %} condition in the template to display the complete information.

Now, all details including flight number, airports, timings, and fly time for each leg are properly displayed.

## Key Issues Summary (74-84)

- 🕒 Fixed missing departure/arrival time inserts and reflections.
- 🌙 Arrival date updates correctly for overnight flights.
- 💾 Fixed session handling so departure date persists through booking steps.
- ✉ Email notifications fixed to send on booking/cancellation with full details.
- ✎ Typo “Depature” corrected to “Departure”.
- ✈ Robust airport pair splitting on flight add, preventing duplicates in flight\_legs and airplane\_fare tables.
- 🔄 Added trigger and python program for automatic fly time calculation and date rollover.
- 🕒 Multi-leg flight timings fully displayed after fixing fetchall and template logic.

## Final Note:

The entire app’s latest stable and corrected version is Version 44 🚀 — incorporating all these fixes and improvements.

## Conclusion

The SKA Flight Ticket Booking System represents a significant step forward in providing a modern, efficient, and secure online platform for flight reservations. It addresses the critical needs of today's travelers by offering a seamless booking experience that is accessible anytime and anywhere, reducing the hassles of traditional ticketing methods.  

With features like secure user authentication, flexible payment methods including QR code and bank account payments, and comprehensive admin controls for flight, fare, and ticket management, the system ensures both user convenience and operational effectiveness.

   The integration of real-time seat availability and dynamic flight management ensures customers receive up-to-date information, empowering them to make informed travel decisions.  

Moreover, the system design emphasizes strong security measures, data integrity, and user privacy, which are essential in maintaining trust and reliability in online transactions. The modular architecture, powered by Python Flask and MySQL, facilitates scalability and easy maintenance, making the system future-ready.  

Looking ahead, planned feature enhancements such as dynamic flight type fetching, advanced seat layout configuration, and robust payment gateway integration with ACID compliance will elevate the platform's flexibility and transactional security. These advancements will not only improve administrative efficiency but also enhance the overall user experience, making flight booking more personalized and responsive to customer needs.  

In conclusion, the SKA Flight Ticket Booking System is well-positioned to transform how airlines and passengers interact in the digital age. By blending technology, usability, and security, it offers a comprehensive solution that supports the evolving dynamics of air travel, ensuring smoother journeys and greater satisfaction for all stakeholders involved.



## Future Look of SKA Flight Ticket Booking System 🚀 ✨

### Feature Developments & Enhancements 🧑‍💻 📁 💻

- Enhanced Payment Gateway Support (Optional):  
Integrate the user-side payment gateway with payment\_mode=ACID support to provide strong transactional guarantees, making payment processes more robust and secure. This supports reliable online payments across multiple modes. 💳 🏠

These future advancements will significantly improve the system's flexibility, usability, and security, keeping SKA Flight Ticket Booking System competitive and responsive to user needs.

## Author's Note

This entire project, SKA Flight Ticket Booking System, has been solely designed, developed, and documented by Adepu Sai Kiran. All concepts, code, designs, and content reflect my personal effort and dedication to delivering a comprehensive and high-quality flight booking platform.

## Contact Information

For any questions, feedback, or collaboration related to the SKA Flight Ticket Booking System, please feel free to reach out to me:

Full Stack Developer Name: Adepu Sai Kiran

GitHub: [Github.com/SaiKiran-Adepu](https://github.com/SaiKiran-Adepu)

Website: <https://saikiran-adepu.github.io/SKA-Re/>

I am happy to assist with any inquiries about the project or explore opportunities for further development and teamwork.

## Acknowledgements

**I sincerely thank my mentors, family members, friends, and Guruji for their constant support and encouragement throughout the development of this project.**

Thank you for reviewing the SKA Flight Ticket Booking System project documentation.

THE END

SKA FLIGHT TICKET BOOKING SYSTEM